Quantum Chemistry in the Age of Quantum Computing

Quantum Subspace Methods & Neural Quantum States



Quantum Computing Research Internship at Orange Innovation.

Author: Arthur Aimone

September, 2026

Contents

1 Introduction & Initial Directions **3**

2 Foundations of Classical Chemistry **5**

2.1 The Chemistry Hamiltonian . . . 5

2.2 First & Second Quantisation . . . 6

2.3 Basis Functions . . . 10

2.4 Methods for Ground State Approximation . . . 11

2.4.1 Hartree-Fock (HF) Method . . . 11

2.4.2 Multiconfigurational Self-Consistent Field (MCSCF) Method - Accounting for Static Correlations . . . 12

2.4.3 Configuration Interaction (CI) Method - Accounting for Dynamic Correlations . . . 12

2.4.4 Coupled Cluster (CC) Method . . . 13

3 Quantum Computational Chemistry **14**

3.1 Toolkit for Quantum Computing . . . 14

3.1.1 One- & Two-Qubit Gates . . . 14

3.1.2 Givens Rotations & the QR Decomposition . . . 18

3.2 Qubit Mapping of the One-electron Hamiltonian . . . 19

3.2.1 First approach - Direct Mapping . . . 19

3.2.2 Unitary Linear Transformations . . . 20

3.2.3 Second Approach - Unitary Linear Transformations . . . 22

3.3 Qubit Mapping of the Two-electron Hamiltonian . . . 27

3.4 Qubit Mapping of the Full Chemistry Hamiltonian . . . 28

4 Quantum Subspace Methods **30**

4.1 Krylov Subspace Method Construction . . . 30

4.2 Krylov Subspace Method on a QPU & Performance . . . 32

4.3 Sample-Based Quantum Diagonalisation (SQD) . . . 34

5 Machine Learning for Many-Body Quantum Systems **36**

5.1 The Sign-Problem in Quantum Monte Carlo . . . 36

5.2 Variational Monte Carlo & Stochastic Sampling . . . 37

5.2.1 Implementation - 2D Ising Model with the Metropolis-Hastings Algorithm . . . 43

5.3 Artificial Neural Networks . . . 44

5.4 Neural Quantum States & Stochastic Reconfiguration . . . 47

5.4.1 Implementation - Transverse Field Ising Model with a Jastrow Ansatz . . . 56

5.4.2 Implementation - Transverse Field Ising Model with NQS . . . 59

5.5 Advanced Topics in Neural Quantum States . . . 65

5.5.1 Symmetry-Aware Neural Quantum States . . . 65

5.5.2 Neural Quantum States for Fermionic Systems 66
5.5.3 Representation of Tensor Networks with Neural Quantum States 67
5.5.4 Neural Quantum States for Open Quantum Systems 68

A Further Topics **70**
A.1 Rayleigh Quotient for Energy Minimization 70
A.2 Density Matrix Embedding Theory (DMET) 70
A.3 Tensor Decompositions in a Nutshell 72
A.4 An Automatic Reduction in the Space of Active Orbitals 73

References **77**

1 | Introduction & Initial Directions

Quantum Computing (QC) is a revolutionary field of computation that leverages the principles of Quantum Mechanics to process amounts of information in a distinct way to Classical Computing. It emerged in the 1980s from the observation that classical computers face fundamental difficulties when simulating quantum systems. Richard Feynman was among the first to propose the idea of using quantum systems for computation, while David Deutsch later formalised the concept of a universal quantum computer. By exploiting fundamentally quantum phenomena such as superposition and entanglement, quantum computers present a promise to solve certain problems more efficiently than classical computers, providing an exponential speed-up over these classical methods in some instances. The field subsequently progressed from a theoretical concept towards practical implementations, with a major milestone given by the development of Shor's algorithm in 1994.
The potential of QC in medicine began to emerge more clearly with advances in quantum algorithms and hardware during the 2010s. A major breakthrough was the development of quantum algorithms for molecular simulation, motivated by the difficulty of classically modelling complex molecular systems and thus opening the possibility of applying QC to drug discovery. Another promising direction is the emergence of Quantum Machine Learning (QML), with applications to genomic data analysis, medical diagnostics and personalised medicine[1]. These developments have established healthcare as a promising area for quantum computing. However, current quantum technologies remain limited by hardware constraints, noise and scalability, making the integration of QC into medical applications an ongoing challenge[1]. This report explores this emerging intersection between quantum computing and medicine, with a core emphasis on the development of quantum simulation methods for the study of molecular systems.

Several initial directions were considered as potential starting points for the research. First, we settled the global theme of the research to be on the use of quantum computers for the design of new drug molecules. This is a very tough problem since, for the time being, designing a new compound roughly takes 10-20 years and costs billions of dollars. There already were some methods that have been developed mainly based on Artificial Intelligence models, such as the renowned `AlphaFold` system which predicts the structure of proteins and other biomolecules from their amino acid sequences. There was a first attempt in 2022 to make a quantum-approach system named `QFold`[2], however this was only presented as a proof-of-concept. Moreover, another idea was to centre the research on quantum-classical methods by mixing traditional Machine Learning models with the help of a quantum computer. This is for instance the case of the `QCA-MolGAN`[3] system which samples bitstrings from a quantum circuit that will then act as a prior distribution for a Generative Adversarial Network (GAN is a Machine Learning model) with the aim to design novel drug molecules with desired target properties. In this report, we will focus on the core problem of creating novel drug molecules which can be understood via *Protein-Ligand Interactions*. The situation is sketched in Figure (1.1) and can be described as follows: our immune system is responsible to generate a response to some potential malignant cells. Among them, cancer cells are usually treated by our immune system with the proliferation of T-cells, for which their role is to destroy infected cells and regulate immune responses. We now mention that on the surface of a T-cell lives some proteins which are enormous molecules of hundreds of atoms, whereas a cancer cell has some ligands (binding molecules) on its surface. However, it turns out that specific proteins and ligands may bind together, where for instance it is the case of the Programmed Cell-death Protein (PD-1) and the Programmed Death-Ligand

1 (PD-L1). If this binding occurs, then the power of the T-cell to destruct the cancer cell is inhibited. Thus, developing an effective cancer treatment requires finding a molecule capable of disrupting the PD-1/PD-L1 interaction, thereby restoring the T-cell's ability to destroy cancer cells. It turns out that understanding this binding requires evaluating the corresponding *binding energy*. For protein-ligand interactions, the energy of interest is the binding energy E_{Bind} which is given by the difference between the energy of the Protein/Ligand Complex $E_{\text{P/L Complex}}$ and the sum of the energies of the protein E_{Protein} with the one of the ligand E_{Ligand}. Formally we have:

$$E_{\text{Bind}} = E_{\text{P/L Complex}} - E_{\text{Protein}} - E_{\text{Ligand}}. \tag{1.1}$$

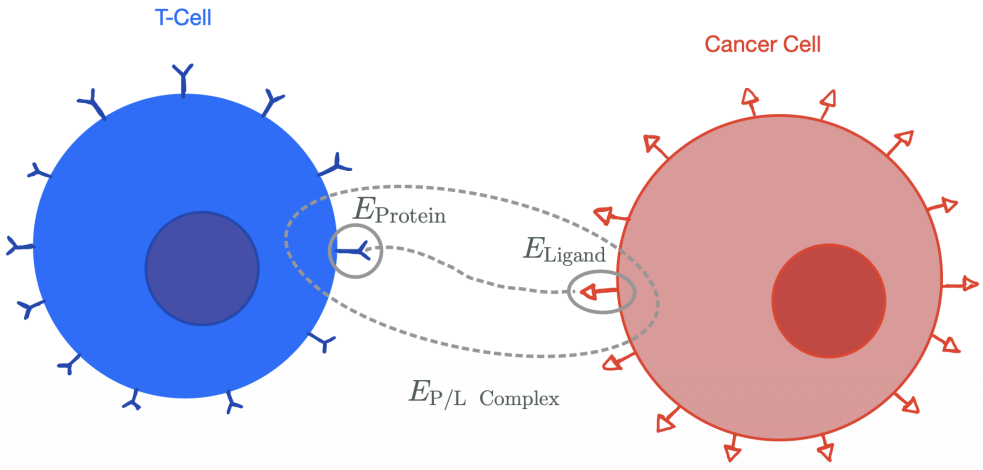


Figure 1.1: Representation of Protein-Ligand Interactions.

Thus, at the level of this simplified description, the problem reduces to evaluating the energies of the complex and its constituent fragments. After some research, it turned out that several crucial problems in Quantum Chemistry could be interpreted as the estimation of an energy. Of particular interest is the so-called *ground-state energy* E_0, which corresponds to the lowest level of energy of a system. In fact, the ground-state energy plays a central role in describing important chemical phenomena, such as bond formation and dissociation. However, computing the (ground-state) binding energy accurately remains a challenging problem for classical computers, as protein–ligand complexes can contain tens of thousands of atoms, making their exact high-accuracy quantum-mechanical description computationally intractable for current methods. For this reason, we initially investigated several approaches to approximately estimate the ground-state energy, including molecular fragmentation techniques such as *Density Matrix Embedding Theory (DMET)* (see [4] for an application to Protein-Ligand complexes). However, these methods quickly proved prohibitively demanding for the available computational resources. Consequently, we decided to focus on ground-state energy estimation for small-scale molecular systems.

In this report, we investigate different methods to estimate the ground-state energy of a quantum-mechanical system. We first briefly review the foundations of Quantum Mechanics, Quantum Chemistry, and Quantum Computing in order for the reader to be able to follow the later sections. We then investigate *Quantum Subspace Methods*, which seek to simplify the energy estimation problem by focusing the analysis on a smaller part of the original problem. We also explore *Neural Quantum States*, a machine-learning approach designed to represent and study complex many-body quantum systems. Throughout the report, the theoretical developments are complemented by numerical implementations, allowing the different methods to be tested on concrete examples.

Note that solving the ground state problem for a completely general local Hamiltonian is QMA-complete. We then present the steps required to implement the fermionic (chemistry) Hamiltonian on a quantum computer.

2 | Foundations of Classical Chemistry

In this chapter, we present the foundational ideas of Quantum Chemistry that will be the basis for the later chapters. For this chapter, we closely follow the (very detailed) book on *Molecular Electronic-Structure Theory* [5] by T. Helgaker, P. Jorgensen and J. Olsen and also [6] by S. McArdle et al. on *Quantum Computational Chemistry*.

2.1 The Chemistry Hamiltonian

We first describe the electronic structure of a molecule via the Chemistry Hamiltonian. First, the Hamiltonian of a molecule consisting of K nuclei and N electrons is

$$H = \underbrace{-\sum_i \frac{\hbar^2}{2m_e}\nabla_i^2 - \sum_I \frac{\hbar^2}{2M_I}\nabla_I^2}_{(*)} \underbrace{-\sum_{i,I} \frac{e^2}{4\pi\epsilon_0}\frac{Z_I}{|\mathbf{r}_i - \mathbf{R}_I|} + \frac{1}{2}\sum_{i\neq j}\frac{e^2}{4\pi\epsilon_0}\frac{1}{|\mathbf{r}_i - \mathbf{r}_j|} + \frac{1}{2}\sum_{I\neq J}\frac{e^2}{4\pi\epsilon_0}\frac{Z_I Z_J}{|\mathbf{R}_I - \mathbf{R}_J|}}_{(**)}, \quad (2.1)$$

for $M_I, \mathbf{R_I}$ and Z_I the mass, position and atomic number of the Ith nucleus and $\mathbf{r}_i$ is the position of the i-th electron. We now describe the two blocks of terms:
$(*)$ Kinetic terms of the electrons and nuclei respectively.
$(**)$ Coulomb repulsion between the electrons and nuclei, the electrons themselves, and the nuclei themselves respectively.

The Hamiltonian in atomic units $M_I' = M_I/m_e$ (with unit of length $a_0 = 1$ Bohr, unit of mass the electronic mass m_e, unit of energy being 1 Hartree $= e^2/(4\pi\epsilon_0 a_0) = 27.211$ eV) becomes:

$$H = -\sum_i \frac{1}{2}\nabla_i^2 - \sum_I \frac{1}{2M_I'}\nabla_I^2 - \sum_{i,I}\frac{Z_I}{|\mathbf{r}_i - \mathbf{R}_I|} + \frac{1}{2}\sum_{i\neq j}\frac{1}{|\mathbf{r}_i - \mathbf{r}_j|} + \frac{1}{2}\sum_{I\neq J}\frac{Z_I Z_J}{|\mathbf{R}_I - \mathbf{R}_J|}. \quad (2.2)$$

Finally, since we are predominantly interested in the electronic part of the Hamiltonian (the parts involving $\mathbf{r}$) for the study of arrangements and interactions of electrons within a molecule, thanks to the Born-Oppenheimer approximation (i.e. since nuclei are much heavier than electrons, they move more slowly - thus consider the electrons in a molecule to be moving in the field of fixed nuclei) we set:

$$H_e = -\sum_i \frac{1}{2}\nabla_i^2 - \sum_{i,I}\frac{Z_I}{|\mathbf{r}_i - \mathbf{R}_I|} + \frac{1}{2}\sum_{i\neq j}\frac{1}{|\mathbf{r}_i - \mathbf{r}_j|}. \quad (2.3)$$

The **aim** is to find the energy eigenstates $|E_i\rangle$ and the corresponding energy eigenvalues E_i of the electronic Hamiltonian H_e.

2.2 First & Second Quantisation

From the Pauli exclusion principle, we know that the electronic wavefunction must be antisymmetric under the exchange of two electrons. We thus characterise the antisymmetrisation in two ways: (i) **first quantisation** where variables like position and momentum are promoted to being operators (*quantisation*), and (ii) **second quantisation** where we consider the quantisation of fields rather than variables. Note that whether a simulation is carried out in first or second quantisation is distinct from the *basis set* or *grid based* method is being used.

Remark 2.2.1. *A key difference however is that using a basis is known as a Galerkin discretisation, which ensures that the energy converges to the correct value from above as the number of basis functions tends to infinity. This property does not hold for grid based methods.*

• **First Quantisation**: Let us briefly present the first quantisation with **basis set methods** (we omit grid based methods as they will not be useful in our context). The **idea** is to project the Hamiltonian onto M basis wavefunctions $\{\phi_p(\mathbf{x}_i)\}$ where $\mathbf{x}_i = (\mathbf{r}_i, \sigma_i)$ is the spatial and spin coordinate of the i-th electron. These basis functions approximate electron spin-orbitals.

Remark 2.2.2. *The grid-based method directly stores the wavefunction without exploiting any knowledge we may have about the general spatial form of the orbitals.*

Let us then write the many electron wavefunction as a **Slater determinant** (i.e. an antisymmetrised product of the single electron basis functions):

$$\psi(\mathbf{x}_0, \ldots, \mathbf{x}_{N-1}) = \frac{1}{\sqrt{N!}} \det \begin{pmatrix} \phi_0(\mathbf{x}_0) & \phi_1(\mathbf{x}_0) & \ldots & \phi_{M-1}(\mathbf{x}_1) \\ \phi_0(\mathbf{x}_1) & \phi_1(\mathbf{x}_1) & \ldots & \phi_{M-1}(\mathbf{x}_1) \\ \vdots & \vdots & \vdots & \vdots \\ \phi_0(\mathbf{x}_{N-1}) & \phi_1(\mathbf{x}_{N-1}) & \ldots & \phi_{M-1}(\mathbf{x}_{N-1}) \end{pmatrix}. \tag{2.4}$$

Swapping the positions of any two electrons is equivalent to interchanging two rows of the Slater determinant, which changes the sign of the wavefunction. This provides the correct exchange symmetry for the fermionic wavefunction. Note, the number M of spin-orbitals is considered to be larger than the number N of electrons in the system (so the electrons can occupy N of the spin-orbitals in a Slater determinant).

• **Second Quantisation**: The information encoded by a Slater determinant can be compressed by moving to the second quantised formalism (so first quantisation is rarely used in classical computational chemistry). As before, we consider the basis set methods and project the Hamiltonian onto M basis wavefunctions $\{\phi_p(\mathbf{x}_i)\}$ and consider many-electron wavefunctions that must be antisymmetric under the exchange of any two electrons. As we saw, the Slater determinant is only determined through the occupation of the spin-orbitals and so we can write a shorthand of these determinants using

$$\psi(\mathbf{x}_0, \ldots, \mathbf{x}_{N-1}) = |f_{M-1}, \ldots, f_p, \ldots, f_0\rangle =: |\mathbf{f}\rangle\,, \tag{2.5}$$

where $f_p = 1$ when ϕ_p is occupied (i.e. it is present in the Slater determinant) and $f_p = 0$ when ϕ_p is empty. The state $|\mathbf{f}\rangle$ is known as the **occupation number (ON) vector** and the space of all such vectors is known as the **Fock space**. The determinants $|\mathbf{f}\rangle$ form an orthonormal basis for the Fock space of the system, and thus the eigenstates of the true Hamiltonian can be expressed as linear combinations of these Slater determinants written in the form

$$|\Psi\rangle = \sum_{\mathbf{f}} \alpha_{\mathbf{f}} |\mathbf{f}\rangle\,, \tag{2.6}$$

for $\alpha_{\mathbf{f}}$ some complex coefficients which are defined to be **determinant amplitudes**. Now, recall the actions of the fermionic creation operator $a_p^\dagger$ for electron excitation and annihilation operator a_p for electron de-excitation:

$$a\,|n\rangle = \sqrt{n}\,|n-1\rangle\,, \tag{2.7}$$

$$a^\dagger\,|n\rangle = \sqrt{n+1}\,|n+1\rangle \tag{2.8}$$

and they obey the fermionic anti-commutation relations:

$$\{a_p, a_q^\dagger\} = a_p a_q^\dagger + a_q^\dagger a_p = \delta_{pq}, \tag{2.9}$$

$$\{a_p, a_q\} = \{a_p^\dagger, a_q^\dagger\} = 0. \tag{2.10}$$

Let us now write the actions of these fermionic operators on the determinants, where $\oplus$ is the mod 2 addition and $S = \sum_{j=0}^{p-1} f_j$:

$$a_p\,|f_{M-1}, f_{M-2}, \ldots, f_0\rangle = \delta_{f_p,1}(-1)^S\,|f_{M-1}, f_{M-2}, \ldots, f_p \oplus 1, \ldots, f_0\rangle\,, \tag{2.11}$$

$$a_p^\dagger\,|f_{M-1}, f_{M-2}, \ldots, f_0\rangle = \delta_{f_p,0}(-1)^S\,|f_{M-1}, f_{M-2}, \ldots, f_p \oplus 1, \ldots, f_0\rangle\,. \tag{2.12}$$

Note that the phase term $(-1)^S$ captures the exchange anti-symmetry of fermions. Moreover, the **spin-orbital occupation-number (ON) operator** is given by $\hat{n}_p = a_p^\dagger a_p$ where its action on the determinants is

$$\hat{n}_p\,|f_{M-1}, \ldots, f_p, \ldots, f_0\rangle = f_p\,|f_{M-1}, \ldots, f_p, \ldots, f_0\rangle\,, \tag{2.13}$$

and counts the number of electrons in a given spin-orbital. Adding together all ON operators in the Fock space yields a Hermitian operator $\hat{N} = \sum_{p=0}^{M-1} a_p^\dagger a_p$ returning the number of electrons in an ON vector:

$$\hat{N}\,|\mathbf{f}\rangle = \sum_{p=0}^{M-1} f_p\,|\mathbf{f}\rangle = N\,|\mathbf{f}\rangle\,, \tag{2.14}$$

and is thus named the **particle-number operator**. Note that a quantum system has a **vacuum state** usually denoted as $|\text{vac}\rangle := |0_0, 0_1, \ldots, 0_{M-1}\rangle$ such that it is always normalized $\langle\text{vac}|\text{vac}\rangle = 1$. Moreover, the action of the annihilation operator on the vacuum state is $a_p\,|\text{vac}\rangle = 0$ such that no electrons are to be destroyed in this state. On the other hand, an interesting property is that we can always write an ON vector of the Fock space in terms of the vacuum state and corresponding creation operators as follows:

$$|\mathbf{f}\rangle = \left[\prod_{p=0}^{M-1} \left(a_p^\dagger\right)^{f_p}\right] |\text{vac}\rangle\,. \tag{2.15}$$

The simplest number-conserving operators (in addition to the particle-number operators) are the **excitation operators** $\hat{X}_q^p := a_p^\dagger a_q$, where the action of an ON vector $|\mathbf{f}\rangle = |f_0, \ldots, f_{M-1}\rangle$ is:

$$\begin{aligned} a_p^\dagger a_q\,|\mathbf{f}\rangle = {} & \Gamma_p^{\mathbf{f}}\Gamma_q^{\mathbf{f}}\Lambda_{pq}^{\mathbf{f}}\,|f_{M-1}, \ldots, f_{q+1}, \delta_{pq}, f_{q-1}, \ldots, f_{p+1}, 1, f_{p-1}, \ldots, f_0\rangle\,\mathbb{1}_{\{p\le q\}} \\ & - \Gamma_p^{\mathbf{f}}\Gamma_q^{\mathbf{f}}\Lambda_{pq}^{\mathbf{f}}\,|f_{M-1}, \ldots, f_{p+1}, 1, f_{p-1}, \ldots, f_{q+1}, \delta_{pq}, f_{q-1}, \ldots, f_0\rangle\,\mathbb{1}_{\{p>q\}}\,, \end{aligned}$$

where:

$$\Gamma_p^{\mathbf{f}} := \prod_{q=0}^{p-2}(-1)^{f_q} \quad \text{and} \quad \Lambda_{pq}^{\mathbf{f}} := (1 - f_p + \delta_{pq})f_q. \tag{2.16}$$

Now, we wish to write the many-body Chemistry Hamiltonian in terms of fermionic operators to produce the so-called *second quantisation* formalism. From now on, whenever a superscript c is written we make reference

to the first quantisation formalism. In this setting, one-electron operators are written as

$$\mathcal{V}^c = \sum_{i=0}^{N-1} \mathcal{V}^c(\mathbf{x}_i), \tag{2.17}$$

where the summation is over the N electrons of the system. Since each term in this operator involves a single electron, the matrix elements should vanish whenever the Slater determinants differ in more than one pair of spin orbitals. Thus, in second quantization, the analogue of Eq. (2.17) has the structure

$$\hat{\mathcal{V}} = \sum_{p,q} \mathcal{V}_{pq} a_p^\dagger a_q, \tag{2.18}$$

since the excitation operators $a_p^\dagger a_q$ shift a single electron in an ON vector. To ensure flexibility in the model, the summation is over all pairs of spin orbitals and the order of the operators in each terms ensures that the one-electron operator $\hat{\mathcal{V}}$ produces zero if it is compiled with the vacuum state. To determine the parameters $\{\mathcal{V}_{pq}\}$, we must evaluate the matrix elements of $\hat{\mathcal{V}}$ between two ON vectors and compare with the so-called *Slater-Condon rules*[7] for matrix elements between determinants. We thus have three distinct cases (see [5, pp.9-10] for a proof):

1. The ON vectors are identical:

$$\langle \mathbf{f} | \hat{\mathcal{V}} | \mathbf{f} \rangle = \sum_p \mathcal{V}_{pp} \langle \mathbf{f} | a_p^\dagger a_p | \mathbf{f} \rangle = \sum_p f_p \mathcal{V}_{pp}. \tag{2.19}$$

2. The ON vectors differ in more than one pair of occupation numbers: let

$$|\mathbf{f}_1\rangle = |f_{M-1}, \ldots, 0_p, \ldots, 1_q, \ldots, f_0\rangle \quad \text{and} \quad |\mathbf{f}_2\rangle = |f_{M-1}, \ldots, 1_p, \ldots, 0_q, \ldots, f_0\rangle, \tag{2.20}$$

then we have:

$$\langle \mathbf{f}_2 | \hat{\mathcal{V}} | \mathbf{f}_1 \rangle = \Gamma_p^{\mathbf{f}_2} \Gamma_q^{\mathbf{f}_1} \mathcal{V}_{pq}. \tag{2.21}$$

3. The ON vectors differ in more than one pair of occupation numbers, and so $\langle \mathbf{f}_2 | \hat{\mathcal{V}} | \mathbf{f}_1 \rangle = 0$.
By comparison and identification with the Slater-Condon rules for one-electron operators, it must be that

$$\mathcal{V}_{pq} = \int \phi_p^*(\mathbf{x}) \mathcal{V}^c(\mathbf{x}) \phi_q(\mathbf{x}) d\mathbf{x}, \tag{2.22}$$

with the permutational symmetry $\mathcal{V}_{pq} = \mathcal{V}_{qp}$ for real spin orbitals. For the two-electron operator, the electronic-repulsion operator takes the form:

$$\mathcal{W}^c = \frac{1}{2} \sum_{i \neq j} \mathcal{W}^c(\mathbf{x}_i, \mathbf{x}_j). \tag{2.23}$$

A two-electron operator gives non-vanishing matrix elements between Slater determinants if they contain at least two electrons and if they differ in the occupations by at most two pairs of electrons. As a result, the second-quantization representation of the two-electron operator has the structure:

$$\hat{\mathcal{W}} = \frac{1}{2} \sum_{p,q,r,s} \mathcal{W}_{pqrs} a_p^\dagger a_r^\dagger a_s a_q. \tag{2.24}$$

The annihilation operators appear to the right of the creation operators in order to ensure that $\hat{\mathcal{W}}$ gives zero if applied on an ON vector with less than two electrons. Note also than if we anti-commute the creation and annihilation operators we find,

$$\sum_{p,q,r,s} \mathcal{W}_{pqrs} a_p^\dagger a_r^\dagger a_s a_q = \sum_{p,q,r,s} \mathcal{W}_{pqrs} a_r^\dagger a_p^\dagger a_q a_s = \sum_{p,q,r,s} \mathcal{W}_{rspq} a_p^\dagger a_r^\dagger a_s a_q. \tag{2.25}$$

We can thus choose the parameters $\mathcal{W}$ to be symmetric, such that $\mathcal{W}_{pqrs} = \mathcal{W}_{rspq}$. Now, as before, the values of the parameters $\{\mathcal{W}_{pqrs}\}$ can be determined by evaluating the matrix elements of $\mathcal{W}$ between two ON vectors and set the result equal to the matrix element between the corresponding Slater determinants. We thus have four distinct cases (see [5, pp.12-13] for a proof):

1. The ON vectors are identical:

$$\langle \mathbf{f}| \hat{\mathcal{W}} |\mathbf{f}\rangle = \frac{1}{2} \sum_{p,q,r,s} \langle \mathbf{f}| a_p^\dagger a_r^\dagger a_s a_q |\mathbf{f}\rangle = \frac{1}{2} \sum_{p,r} f_p f_r (\mathcal{W}_{pprr} - \mathcal{W}_{prrp}). \tag{2.26}$$

2. The ON vectors differ in one pair of occupation numbers: let

$$|\mathbf{f}_1\rangle = |f_{M-1}, \ldots, 0_p, \ldots, 1_q, \ldots, f_0\rangle \quad \text{and} \quad |\mathbf{f}_2\rangle = |f_{M-1}, \ldots, 1_p, \ldots, 0_q, \ldots, f_0\rangle\,, \tag{2.27}$$

then we have:

$$\langle \mathbf{f}_2| \hat{\mathcal{W}} |\mathbf{f}_1\rangle = \Gamma_p^{\mathbf{f}_2} \Gamma_q^{\mathbf{f}_1} \sum_r f_r (\mathcal{W}_{pqrr} - \mathcal{W}_{prrq}). \tag{2.28}$$

3. The ON vectors differ in two pairs of occupation numbers: let

$$|\mathbf{f}_1\rangle = |f_{M-1}, \ldots, 0_p, \ldots, 0_q, \ldots, 1_r, \ldots, 1_s, \ldots, f_0\rangle \quad \text{and} \quad |\mathbf{f}_2\rangle = |f_{M-1}, \ldots, 1_p, \ldots, 1_q, \ldots, 0_r, \ldots, 0_s, \ldots, f_0\rangle\,, \tag{2.29}$$

such that $p > q$ and $r > s$, then we have:

$$\langle \mathbf{f}_2| \hat{\mathcal{V}} |\mathbf{f}_1\rangle = \Gamma_p^{\mathbf{f}_2} \Gamma_q^{\mathbf{f}_2} \Gamma_r^{\mathbf{f}_1} \Gamma_s^{\mathbf{f}_1} (\mathcal{W}_{prqs} - \mathcal{W}_{psqr}). \tag{2.30}$$

4. The ON vectors differ in more than two pairs of occupation numbers, and so $\langle \mathbf{f}_2| \hat{\mathcal{W}} |\mathbf{f}_1\rangle = 0$.
Again by comparison and identification with the Slater-Condon rules for two-electron operators, it must be that

$$\mathcal{W}_{pqrs} = \int\int \phi_p^*(\mathbf{x}_1) \phi_r^*(\mathbf{x}_2) \mathcal{W}^c(\mathbf{x}_1, \mathbf{x}_2) \phi_q(\mathbf{x}_1) \phi_s(\mathbf{x}_2) d\mathbf{x}_1 d\mathbf{x}_2, \tag{2.31}$$

with the permutational symmetry $\mathcal{W}_{pqrs} = \mathcal{W}_{qprs} = \mathcal{W}_{pqsr} = \mathcal{W}_{qpsr}$ for real spin orbitals. Hence, for real spin orbitals there are a total of eight permutational symmetries in the two-electron integrals, whereas for complex spin-orbitals there is only one such symmetry. Finally, combining the different results, we obtain the **second quantised form of the electronic Hamiltonian**:

$$\hat{H} = \sum_{p,q} h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum_{p,q,r,s} h_{pqrs} a_p^\dagger a_q^\dagger a_r a_s + h_{\text{nuc}}, \tag{2.32}$$

with the one- and two-electron integrals given by:

$$h_{pq} = \underbrace{\int \phi_p^*(\mathbf{x}) \left(-\frac{\nabla^2}{2} - \sum_I \frac{Z_I}{|\mathbf{r} - \mathbf{R}_I|} \right) \phi_q(\mathbf{x}) d\mathbf{x}}_{\text{kinetic energy terms of electrons + Coulomb interaction with nuclei}}\,, \tag{2.33}$$

$$h_{pqrs} = \underbrace{\int\int \frac{\phi_p^*(\mathbf{x}_1) \phi_q^*(\mathbf{x}_2) \phi_r(\mathbf{x}_2) \phi_s(\mathbf{x}_1)}{|\mathbf{r}_1 - \mathbf{r}_2|} d\mathbf{x}_1 d\mathbf{x}_2}_{\text{electron-electron Coulomb repulsion}}\,, \tag{2.34}$$

$$h_{\text{nuc}} = \frac{1}{2} \sum_{p \neq q} \frac{Z_p Z_q}{R_{pq}}\,. \tag{2.35}$$

Remark 2.2.3. *This Hamiltonian contains up to M^4 terms depending on the basis functions used. Furthermore, if all $\binom{M}{N}$ determinants are included, the wavefunction is known as the* ***full configuration interaction wavefunction*** *(FCI wavefunction).*

2.3 Basis Functions

In Quantum Chemistry, the standard wavefunctions are all constructed from antisymmetric products of molecular orbitals (MOs). The description of these MOs is generated through the expansion of analytical functions known as *atomic basis functions*. As a result, the choice of basis functions for a molecular calculation is an important one, as it dictates the quality of the wavefunction. In this section, we briefly investigate the mathematical properties of atomic basis functions and provide several categories of bases.

There are two ways one can construct MOs, either numerically or algebraically by linear expansion in a set of basis functions. The numerical approach is usually flexible and accurate, however it quickly becomes intractable due to the large amount of grid points. For complex polyatomic systems, we are forced to take the algebraic route and expand the MOs $\{\phi_p(\mathbf{x})\}$ in a set of simple analytical one-electron functions:

$$\phi_p(\mathbf{x}) = \sum_\mu C_\mu^{(p)} f_\mu(\mathbf{x}), \tag{2.36}$$

for some functions $\{f_\mu(\mathbf{x})\}$ to be determined and coefficients $\{C_\mu^{(p)}\}$. In practice, for molecular systems defined by the Chemistry Hamiltonian, the orbitals in the matrix elements (2.33) and (2.34) must be expressed as linear combinations in the form of (2.36) where the functions $\{f_\mu(\mathbf{x})\}$ are atomic orbital centred on one of the nuclei. A physically motivated choice for these atomic orbitals is driven by the exact solution of the Schrödinger equation for the Hydrogen atom. These are the **Slater-Type Orbitals (STOs)**:

$$f_{n,l,m}(r,\xi,\theta,\phi) = N r^{n-1} e^{-\xi r} Y_l^m(\theta,\phi), \tag{2.37}$$

where n is the principal quantum number, l and m are the angular momentum quantum numbers, ξ is a parameter controlling the radial decay, r is the distance of the electron to the atomic nucleus, $N = (2\xi)^n \sqrt{2\xi/(2n)!}$, and $Y_l^m(\theta,\phi)$ are the spherical harmonics. The factor r^{n-1} ensures the correct cusp behaviour at the nucleus (see the *Kato's cusp condition*) and the exponential $e^{-\xi r}$ reproduces the asymptotic decay. However, dealing with multicentre integrals in the STO basis is a challenging problem, since one has to compute integrals of the form:

$$\int \frac{e^{-\xi_i|\mathbf{r}-\mathbf{R}_A|} e^{-\xi_j|\mathbf{r}-\mathbf{R}_B|}}{|\mathbf{r}-\mathbf{r}'|} d\mathbf{r}, \tag{2.38}$$

where $\mathbf{R}_A$ and $\mathbf{R}_B$ denote the positions of two distinct nuclei. In fact, these integrals do not generally admit closed form formulas. To circumvent this issue, one can adopt **Gaussian-Type Orbitals (GTOs)**:

$$g_{n,l,m}(\mathbf{r},\xi) \propto x^l y^m z^n e^{-\xi r^2}, \tag{2.39}$$

with non-negative numbers (l,m,n) such that $l+m+n$ corresponds to the orbital's total angular momentum, and $r^2 = x^2 + y^2 + z^2$. Although GTOs do not capture the nuclear cusp or the exact asymptotic decay of the electron density, they offer a significant computational advantage. Indeed, the product of two Gaussian functions centred at different nuclei is again a Gaussian:

$$e^{-\xi_i|\mathbf{r}-\mathbf{R}_A|} e^{-\xi_j|\mathbf{r}-\mathbf{R}_B|} = K e^{-(\xi_i+\xi_j)|\mathbf{r}-\mathbf{R}|^2}, \tag{2.40}$$

where

$$K = e^{-\frac{\xi_i \xi_j}{\xi_i + \xi_j}|\mathbf{R}_A - \mathbf{R}_B|^2} \quad \text{and} \quad \mathbf{R} = \frac{\xi_i \mathbf{R}_A + \xi_j \mathbf{R}_B}{\xi_i + \xi_j}. \tag{2.41}$$

Moreover, the Coulomb operator can be re-written as an integral over a Gaussian,

$$\frac{1}{|\mathbf{r} - \mathbf{r}'|} = \frac{2}{\sqrt{\pi}} \int_0^\infty e^{-t^2 |\mathbf{r} - \mathbf{r}'|^2} dt, \tag{2.42}$$

which enables all multicentre integrals to be reduced to combinations of Gaussian integrals that can be evaluated analytically. Now, while the STO basis is more accurate, in practice one tends to use the GTO basis. However, one can also approximate the STOs with a linear combination of Gaussian functions (GTOs). One caveat is that Gaussians tend to underestimate the values close to the nucleus, which gives significant deviations in the calculations for larger molecules, and so in practice one needs to *fit* a certain number N of Gaussian functions to Slater functions. This yields the **STO-nG** basis (n here is **not** the principal quantum number):

$$\phi_p^{\text{STO-}n\text{G}}(\mathbf{r}) = \sum_{j=1}^{n} \beta_j^{(p)} \phi_p^{\text{GTO}}(\mathbf{r}, \xi_j). \tag{2.43}$$

It is common practice to find the coefficients $\{\beta_j^{(p)}\}$ by fitting these Gaussian functions with a least squares method. We finally conclude by noting that there are multiple other basis sets, such as the X-YZg basis (for some numbers (X,Y,Z)) or also the cc-PVNZ basis for $N \in \{D, T, Q, 5, 6, \ldots\}$ (D = double, T = triple, Q = quadruple). We will not delve into other basis sets, however the interested reader can look at [5] in Chapters 6 & 8 or [6, pp. 17-19] for more details.

2.4 Methods for Ground State Approximation

In this final section, we briefly review four methods for the approximation of the ground state energy. For a complete description of the methods, one can refer to [6, pp. 14-16].

2.4.1 Hartree-Fock (HF) Method

In this subsection, we present the Hartree-Fock (HF) method for approximating the ground state wavefunction with a restricted number of Slater determinants. The **aim** is to find the dominant Slater determinant in the system wavefunction, where this is achieved by optimising the spatial form of the spin-orbitals to minimise the energy of the wavefunction.

Method Construction:
We consider a number of spin-orbitals M and a number of electrons in a system N, such that $M \gg N$ (i.e. N spin-orbitals are occupied and $M - N$ orbitals are unoccupied/virtual).
• Neglect the final Coulomb repulsion term in the electronic structure Hamiltonian (Eq. (2.3)) reducing the problem to one of the N independent electrons.
• Replace this final two-electron part by an effective one electron Fock potential V_{ij} which in an average sense incorporates the effects of the Coulomb repulsion among the electrons.
• In the second quantised formalism, use the orbitals to construct the Fock operator $\hat{f}$ and diagonalise this operator to obtain new orbitals:

$$\hat{f} = \sum_{i,j} (h_{ij} + V_{ij}) a_i^\dagger a_j, \tag{2.44}$$

$$V_{ij} = \sum_{k \in \text{occ}} (h_{ikkj} - h_{ikjk}), \tag{2.45}$$

where occ is the set of occupied orbitals. The term h_{ikkj} describes the Coulomb interaction of an electron with the charge distribution of the other electrons. The term h_{ikjk} describes exchange effects (Fermi correlations)

arising from the required anti-symmetrisation. (see [6, pp. 14] for discussions on the roles of dynamic and static correlations).

Summary
To perform HF, input a set of atomic orbitals which are localised around each atom. They are used to find the Fock operator which is then diagonalised to obtain new orbitals (i.e. linear combinations of the old orbitals). Repeat the process until the orbitals converge.

The new orbitals obtained are referred to as the **canonical orbitals**. This procedures generates single particle molecular orbitals from combinations of the single particle atomic orbitals. As mentioned in [6], the Slater determinant generated from a HF calculation is typically taken as the reference state for post-HF methods (see the next subsections), such as configuration interaction and coupled cluster, which seek to capture some of the dynamic correlation energy by including additional determinants, describing excitations above the HF state.

2.4.2 Multiconfigurational Self-Consistent Field (MCSCF) Method - Accounting for Static Correlations

Although the HF method is relatively simple, it performs poorly for strongly correlated systems. Systems with static strong correlation are defined as those where multiple Slater determinants are equally dominant, and thus the wavefunction is of *multiconfigurational* nature (e.g. excited states). To account for this, the MCSCF approach considers a wavefunction with several Slater determinants and variationally optimises both the molecular orbitals and the determinant amplitudes simultaneously. Concretely, we consider a wavefunction $|\Psi_{\text{MCSCF}}\rangle = e^{-\kappa}\sum_f \alpha_f |f\rangle$, where α_f are determinant amplitudes, $|f\rangle$ are Slater determinants, and $\kappa = \sum_{i,j} k_{ij} a_i^\dagger a_j$ is anti-Hermitian operator for which the exponential of κ produces a unitary operator which rotates the orbital basis. We then variationally minimise the energy by optimising both the amplitudes α_f and the entries $\{k_{ij}\}$ of the matrix κ. Although MCSCF is usually considered to be the best approximation to the exact wavefunction for a given number of determinants, it is not possible to perform a complete MCSCF calculation on all determinants since the latter grows exponentially with the system size. To circumvent this issue, one can use chemical intuition to select the most important Slater determinants and then perform MCSCF on this reduced space, or use a complete active space self-consistent field (CASSCF) method[8] by selecting the most *important* orbitals (the *active space*) to perform the MCSCF calculation. Again, we emphasize that MCSCF and CASSCF are the most efficient methods to treat systems with strong static correlation, however one caveat is that choosing the active space is highly subjective and lacks of a proper motivation to define the word *important*. Recent publications have presented a method for automated selection of the active space via the orbital entanglement metric named AutoCAS[9,10], where we give an account of the method in Appendix A.2. We finally note that, while MCSCF and CASSCF are efficient for the recovery of the static correlation energy, they are not suitable to recover the dynamic correlation energy and so we have to turn to other methods.

2.4.3 Configuration Interaction (CI) Method - Accounting for Dynamic Correlations

The CI method generates a correlated wavefunction by considering excitations above a reference state, typically taken to be the Hartree-Fock state from Subsection 1.4.1. Note that if all determinants are included, we recover the full configuration interaction (FCI) wavefunction:

$$|\Psi_{\text{FCI}}\rangle = \left(\mathbb{I} + \sum_{i,\alpha} C_{i\alpha} a_i^\dagger a_\alpha + \sum_{i>j,\alpha>\beta} C_{ij\alpha\beta} a_i^\dagger a_j^\dagger a_\alpha a_\beta + \ldots\right)|\Psi_{\text{HF}}\rangle\,, \tag{2.46}$$

where $|\Psi_{\text{HF}}\rangle$ is the Hartree-Fock state and C are parameters to be optimised according to the Rayleigh-Ritz variational principle, i.e. using the fact that $\langle\Psi| H |\Psi\rangle / \langle\Psi|\Psi\rangle \geq E_{\text{GS}}$, for the ground state energy E_{GS}, one can always optimise the energy to *get close* to E_{GS}. Simulating all the determinants is a classically hard problem, thus the CI method is generally restricted to a small number of excitation operators: single excitations (CIS), double excitations (CISD), and occasionally triple excitations (CISDT). However, this does not generally affect the quality of the approximation of the ground state energy since low excitations often dominate the ground state wavefunction in most systems. Although the CI method is effective at recovering dynamic correlations it often struggles with static correlations, unless if the initial state is a MCSCF state. This alternative option, named multiconfiguration configuration interaction (MCI), seeks to recover dynamic correlations on top of the static correlations described by the MCSCF state. Two limitations of this method are the following: (i) the method converges slowly to the FCI wavefunction as a result of its linear parametrization, and (ii) the energy obtained from a truncated CI calculation does not scale correctly with the system size.

2.4.4 Coupled Cluster (CC) Method

The CC method is similar in nature to the CI method in the sense that it also includes additional determinants to recover the correlation energy, however this time we use a product parametrization for the wavefunction:

$$|\Psi_{\text{CC}}\rangle = \left[\prod_{i,\alpha}\left(\mathbb{I} + C_{i\alpha}a_i^\dagger a_\alpha\right) \times \prod_{i>j,\alpha>\beta}\left(\mathbb{I} + C_{ij\alpha\beta}a_i^\dagger a_j^\dagger a_\alpha a_\beta\right) \times \dots\right] |\Psi_{\text{HF}}\rangle\,, \tag{2.47}$$

where $|\Psi_{\text{HF}}\rangle$ is the Hartree-Fock state and C are parameters. Note that the CC state may be recast in an exponential form:

$$|\Psi_{\text{CC}}\rangle = e^T |\Psi_{\text{HF}}\rangle\,, \qquad \text{where } T = \sum_j T_j. \tag{2.48}$$

Each component $\{T_j\}$ can be written as:

$$T_1 = \sum_{\substack{i\in\text{virt}\\ \alpha\in\text{occ}}} t_{i\alpha}a_i^\dagger a_\alpha\,, \qquad T_2 = \sum_{\substack{i,j\in\text{virt}\\ \alpha,\beta\in\text{occ}}} t_{ij\alpha\beta}a_i^\dagger a_j^\dagger a_\alpha a_\beta\,, \qquad \dots \tag{2.49}$$

where t are excitation amplitudes, *occ* denotes the occupied orbitals in the HF state and *virt* denotes unoccupied (virtual) orbitals in the HF state. As before, if all the excitation operators are included we recover the FCI wavefunction, at the cost of an exponential complexity. As a result, it is common practice to truncate the CC wavefunction to double excitations (CCSD). The reader should refer to [6] for details on the scaling of the complexity for the implementation of a CCSD wavefunction. One drawback of the CC method is that it does not obey the Rayleigh-Ritz variational principle, i.e. the fact that $\langle\Psi| H |\Psi\rangle / \langle\Psi|\Psi\rangle \geq E_{\text{GS}}$, for the ground state energy E_{GS}. Moreover, the conventional CC method described above is a single determinant reference state method. Consequently, it does not tend to perform well in the setting of multireference states, which are required to describe systems with strong static correlation. One possible method to bypass this issue is the so-called unitary coupled cluster[11,12] (UCC) which is both variational and suitable for multireference states. Also, although it is has an exponential cost on classical computers, the method is efficient on a quantum computer.

3 | Quantum Computational Chemistry

Let us present how we can encode a chemistry Hamiltonian into a qubit Hamiltonian for quantum computing implementations by following the steps in [13]. An encoding method is a map from the fermionic Fock space to the Hilbert space of qubits such that every fermionic state can be represented by a qubit state. For our purposes, we will concentrate on the so-called **Jordan-Wigner** (JW) encoding. The **idea** is simple, we store the occupation number of a spin-orbital in the $|0\rangle$ (unoccupied) or $|1\rangle$ (occupied) state of a qubit. Formally, for $q_p = f_p \in \{0, 1\}$, we consider the mapping

$$|f_{M-1}, f_{M-2}, \ldots, f_0\rangle \mapsto |q_{M-1}, q_{M-2}, \ldots, q_0\rangle \,. \tag{3.1}$$

Now, we will also have to encode the creation/annihilation operators which increase/decrease the occupation number of a spin-orbital by 1, and also introduce a multiplicative phase factor. We thus have:

$$a_p = Q_p \otimes Z_{p-1} \otimes \ldots \otimes Z_0 := |0\rangle\langle 1| \otimes Z_{p-1} \otimes \ldots \otimes Z_0 \overset{\text{Pauli}}{\equiv} \frac{X + iY}{2} \otimes Z_{p-1} \otimes \ldots \otimes Z_0, \tag{3.2}$$

$$a_p^\dagger = Q_p^\dagger \otimes Z_{p-1} \otimes \ldots \otimes Z_0 := |1\rangle\langle 0| \otimes Z_{p-1} \otimes \ldots \otimes Z_0 \overset{\text{Pauli}}{\equiv} \frac{X - iY}{2} \otimes Z_{p-1} \otimes \ldots \otimes Z_0. \tag{3.3}$$

The Q and $Q^\dagger$ operators change the occupation number of the target spin-orbital, while the string of Z operators recover the exchange phase factor $(-1)^S$ (i.e. the *parity of the state*). Let us now transcribe the fermionic electronic Hamiltonian (2.32) into qubits.

3.1 Toolkit for Quantum Computing

In this section, we present the fundamental tools for Quantum Computing that will be used in this work.

3.1.1 One- & Two-Qubit Gates

We recall the single-qubit Pauli operators:

$$\mathbb{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \; X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \; Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \; Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Recall also that any qubit can be prepared in a generic state $|\phi\rangle$ by initialisation in $|0\rangle$ and application of single-qubit Pauli rotations

$$R_\sigma(\theta) = \exp\left(-i\frac{\theta}{2}\sigma\right) = \cos\left(\frac{\theta}{2}\right)\mathbb{I} - i\sin\left(\frac{\theta}{2}\right)\sigma; \qquad \sigma \in \{X, Y, Z\}, \; \theta \in [0, 2\pi). \tag{3.4}$$

In matrix form, these rotations are given as follows:

$$R_X(\theta) = \begin{pmatrix} \cos(\theta/2) & -i\sin(\theta/2) \\ -i\sin(\theta/2) & \cos(\theta/2) \end{pmatrix}, \; R_Y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}, \; R_Z(\theta) = \begin{pmatrix} \exp(-i\theta/2) & 0 \\ 0 & \exp(i\theta/2) \end{pmatrix}.$$

Recall also the important Hadamard single-qubit gate:

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

As we know, single-qubit gates are often not enough for universal quantum computation since they cannot entangle qubits. We should then consider the combination of single- and two-qubit gates. One important example for us is the **canonical gate** defined as:

$$\mathrm{Can}(t_X, t_Y, t_Z) := \exp\Big(-i\frac{\pi}{2}(t_X \cdot X\otimes X + t_Y \cdot Y\otimes Y + t_Z \cdot Z\otimes Z)\Big) \equiv R_{XX}(\pi t_X)R_{YY}(\pi t_Y)R_{ZZ}(\pi t_Z), \tag{3.5}$$

where the parameters t_X, t_Y, t_Z lie in the so-called Weyl chamber $\{1/2 \geq t_X \geq t_Y \geq t_Z \geq 0\} \cup \{1/2 \geq (1-t_X) \geq t_Y \geq t_Z \geq 0\}$. Any 2-qubit gate can be decomposed into a canonical gate and single-qubit gates by the so-called KAK decomposition. The important example for us will be the decomposition of the $XX+YY$ gate, a key component in the simulation of fermionic systems:

$$V_{XX+YY}(\theta,\beta) := \big(\mathbb{I}\otimes R_Z(\beta)\big)\mathrm{Can}\left(\frac{\theta}{\pi}, \frac{\theta}{\pi}, 0\right)\big(\mathbb{I}\otimes R_Z(-\beta)\big). \tag{3.6}$$

Another gate that will be crucial in the construction of our circuits is the 2-qubit controlled-NOT gate given by:

$$\mathrm{CNOT} := \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \tag{3.7}$$

This gate applies the identity operator on the second qubit if the first qubit is $|0\rangle$, and applies the X operator if the first qubit is $|1\rangle$. Formally, for a Hilbert space with $\dim(\mathcal{H}) = 4$ (i.e. a 2-qubit system), this gate may be written in terms of the projection operators $|0\rangle\langle 0|$ and $|1\rangle\langle 1|$ as $\mathrm{CNOT} = |0\rangle\langle 0|\otimes\mathbb{I} + |1\rangle\langle 1|\otimes X$. Now, let us see how can this be adapted on a N-qubit space such that $\dim(\mathcal{H}) = 2^N$. In particular, we are interested in a CNOT gate that has j as the control qubit and k as the target qubit, denoted by $\mathrm{CNOT}_{j,k}$. **Starting the indexing from zero**, this is formally given by:

$$\mathrm{CNOT}_{j,k} = \big(|0\rangle\langle 0|_j \otimes \mathbb{I}_k + |1\rangle\langle 1|_j \otimes X_k\big) \otimes \left(\bigotimes_{w\neq\{j,k\}} \mathbb{I}_w\right) \tag{3.8}$$

$$= \Big(\mathbb{I}^{\otimes j}\otimes|0\rangle\langle 0|_j \otimes \mathbb{I}^{\otimes(N-j-1)}\Big) + \Big(\mathbb{I}^{\otimes j}\otimes|1\rangle\langle 1|_j \otimes \mathbb{I}^{\otimes(k-j-1)}\otimes X_k\otimes\mathbb{I}^{\otimes(N-k-1)}\Big), \tag{3.9}$$

where the identity matrix $\mathbb{I}$, the projectors and the NOT gate X are of appropriate dimension(s) for the space of N qubits. In this work, we set $\mathrm{CNOT} \equiv \mathrm{CNOT}_{0,1}$ when it is clear from the context.

Now, as we will see, the simulation of the time-evolution operator will be of central importance. Since this will involve a trotterization of the operator, we must look at how we can implement exponentials of Pauli operators. Considering multi-qubit Pauli operators $\hat{\Sigma} := \hat{\sigma}_{m_{N_q-1}}\otimes\ldots\otimes\hat{\sigma}_{m_0}$ for $\hat{\sigma}_m \in \{\mathbb{I}, X, Y, Z\}$, if we consider $\hat{\sigma}\neq\mathbb{I}$ we can write the rotation operators in the following way:

$$R_{\hat{\Sigma}}(\theta) := \exp\left(-i\frac{\theta}{2}\hat{\Sigma}\right) = \hat{V}^\dagger \exp\left(-i\frac{\theta}{2}Z\otimes\ldots\otimes Z\right)\hat{V} \tag{3.10}$$

such that,

$$\hat{V} = \bigotimes_{k=0}^{N_q-1} \hat{A}_{m_k} \quad \text{and} \quad \hat{A}_m^\dagger \hat{\sigma}_m \hat{A}_m = Z. \tag{3.11}$$

We can then introduce a ladder of CNOT gates that map $Z \otimes \ldots \otimes Z$ onto Z_{N_q-1} so that we finally have:

$$R_{\hat{\Sigma}}(\theta) = \hat{V}^\dagger \hat{W}^\dagger (R_Z(\theta))_{N_q-1} \hat{W} \hat{V}, \qquad \text{with } \hat{W} = \prod_{k=0}^{N_q-2} \text{CNOT}_{k,k+1}, \tag{3.12}$$

where $\text{CNOT}_{k,k+1}$ indicates a CNOT gate with control qubit k and target qubit $k+1$. It is given by (from Eq. (3.9)):

$$\text{CNOT}_{k,k+1} = \left(\mathbb{I}^{\otimes k} \otimes |0\rangle\langle 0|_k \otimes \mathbb{I}^{\otimes(N-k-1)}\right) + \left(\mathbb{I}^{\otimes k} \otimes |1\rangle\langle 1|_k \otimes X_{k+1} \otimes \mathbb{I}^{\otimes(N-k-2)}\right). \tag{3.13}$$

Let us briefly go back to the $XX+YY$ gate and see how it can be implemented on a quantum computer. In particular, since this gate depends on the canonical gate, with the help of Eq. (3.12) we describe the implementation of the R_{XX}, R_{YY} and R_{ZZ} gates. Let us start with the $R_{ZZ}(\theta)$ gate by looking at its action on computational basis states:

$$R_{ZZ}(\theta)\,|uv\rangle := e^{-\frac{i\theta}{2} Z\otimes Z}\,|uv\rangle = e^{-\frac{i\theta}{2}(-1)^{u\oplus v}}\,|uv\rangle\,.$$

Thus we see that the $R_{ZZ}(\theta)$ gate can be implemented by first using a CNOT gate to verify the parity of qubits u and v, then use a $R_Z(\theta)$ gate to apply the correct phase factor, and finally undo the effect of the first gate by applying another CNOT gate. This gives us:

$$R_{ZZ}(\theta) = \text{CNOT}\big(\mathbb{I} \otimes R_Z(\theta)\big)\text{CNOT}. \tag{3.14}$$

Remark 3.1.1. *By expanding $R_{ZZ}(\theta)$ using $\text{CNOT} = |0\rangle\langle 0| \otimes \mathbb{I} + |1\rangle\langle 1| \otimes X$ and Eq. (3.4), note the following:*

$$R_{ZZ}(\theta) = |0\rangle\langle 0| \otimes R_Z(\theta) + |1\rangle\langle 1| \otimes X R_Z(\theta) X = |0\rangle\langle 0| \otimes R_Z(\theta) + |1\rangle\langle 1| \otimes R_Z(-\theta).$$

Remark that $|0\rangle$ and $|1\rangle$ correspond to the basis states of the Pauli-Z operator. Then, a $R_{ZZ}(\theta)$ gate tells us to apply a rotation of a Bloch vector by an angle θ about the Z-axis of the Bloch sphere if the control qubit is $|0\rangle$ and to apply a rotation of angle $(-\theta)$ if the control qubit is $|1\rangle$.

Let us proceed with a circuit representation for the $R_{XX}(\theta)$ gate using Eq. (3.12). We are then looking for an operator $\hat{V}$ such that $R_{XX}(\theta) := \exp\big(-\frac{i\theta}{2} X \otimes X\big) = \hat{V}^\dagger \exp\big(-\frac{i\theta}{2} Z \otimes Z\big)\hat{V}$ where $\hat{V} = \hat{A}_0 \otimes \hat{A}_1$ and the relationship $\hat{A}_0^\dagger X \hat{A}_0 = \hat{A}_1^\dagger X \hat{A}_1 = Z$ holds. However, from the well-known matrix equality $HXH = Z$ we have that $\hat{A}_0 = \hat{A}_0^\dagger = \hat{A}_1 = \hat{A}_1^\dagger = H$. Thus we know that $\hat{V} = H \otimes H = \hat{V}^\dagger$, where along with Eq. (3.14) for the $R_{ZZ}(\theta)$ gate, we can conclude:

$$R_{XX}(\theta) = (H \otimes H) R_{ZZ}(\theta)(H \otimes H) = (H \otimes H)\text{CNOT}\big(\mathbb{I} \otimes R_Z(\theta)\big)\text{CNOT}(H \otimes H). \tag{3.15}$$

Remark 3.1.2. *By expanding $R_{XX}(\theta)$ using* $\text{CNOT} = |0\rangle\langle 0| \otimes \mathbb{I} + |1\rangle\langle 1| \otimes X$ *and Eq. (3.4), note the following:*

$$R_{XX}(\theta) = |+\rangle\langle +| \otimes HR_Z(\theta)H + |-\rangle\langle -| \otimes HR_Z(-\theta)H = |+\rangle\langle +| \otimes R_X(\theta) + |-\rangle\langle -| \otimes R_X(-\theta),$$

where we have used the conventional notation $H|0\rangle = |+\rangle$ and $H|1\rangle = |-\rangle$. Remark that $|+\rangle$ and $|-\rangle$ correspond to the basis states of the Pauli-X operator. Then, a $R_{XX}(\theta)$ gate tells us to apply a rotation of a Bloch vector measured in the X-basis by an angle θ about the X-axis of the Bloch sphere if the control qubit is $|+\rangle$ and to apply a rotation of angle $(-\theta)$ if the control qubit is $|-\rangle$.

Let us finally look for a circuit representation of $R_{YY}(\theta)$ again using Eq. (3.12). We are once again looking for an operator $\hat{V}$ such that $R_{YY}(\theta) := \exp\big(-\frac{i\theta}{2}Y \otimes Y\big) = \hat{V}^\dagger \exp\big(-\frac{i\theta}{2}Z \otimes Z\big)\hat{V}$ where $\hat{V} = \hat{A}_0 \otimes \hat{A}_1$ and the relationship $\hat{A}_0^\dagger Y \hat{A}_0 = \hat{A}_1^\dagger Y \hat{A}_1 = Z$ holds. The key idea is to remark the following:

$$\begin{aligned} R_X(-2\theta)YR_X(2\theta) = e^{i\theta X}Ye^{-i\theta X} &= \big(\cos(\theta)Y + i\sin(\theta)XY\big)\big(\cos(\theta)\mathbb{I} - i\sin(\theta)X\big) \\ &= \big(\cos^2(\theta) - \sin^2(\theta)\big)Y + i\sin(\theta)\cos(\theta)[X,Y] \\ &= \cos(2\theta)Y - \sin(2\theta)Z. \end{aligned}$$

Thus we require $\cos(2\theta) = 0$ and $\sin(2\theta) = -1$ giving us $\theta = -\pi/4$. As a result, we will consider $\hat{A}_0 = \hat{A}_1 = R_X(-\pi/2)$ and $\hat{A}_0^\dagger = \hat{A}_1^\dagger = R_X(\pi/2)$ in order to construct $\hat{V} = R_X(-\pi/2) \otimes R_X(-\pi/2)$ and $\hat{V}^\dagger = R_X(\pi/2) \otimes R_X(\pi/2)$. Along with Eq. (3.14) for the $R_{ZZ}(\theta)$ gate, we can conclude:

$$R_{YY}(\theta) = \big(R_X(\pi/2) \otimes R_X(\pi/2)\big)R_{ZZ}(\theta)\big(R_X(-\pi/2) \otimes R_X(-\pi/2)\big) \tag{3.16}$$

$$= \big(R_X(\pi/2) \otimes R_X(\pi/2)\big)\text{CNOT}\big(\mathbb{I} \otimes R_Z(\theta)\big)\text{CNOT}\big(R_X(-\pi/2) \otimes R_X(-\pi/2)\big). \tag{3.17}$$

Remark 3.1.3. *By expanding $R_{YY}(\theta)$ using* $\text{CNOT} = |0\rangle\langle 0| \otimes \mathbb{I} + |1\rangle\langle 1| \otimes X$ *and Eq. (3.4), note the following:*

$$\begin{aligned} R_{YY}(\theta) &= R_X\left(\frac{\pi}{2}\right)|0\rangle\langle 0|R_X\left(-\frac{\pi}{2}\right) \otimes R_X\left(\frac{\pi}{2}\right)R_Z(\theta)R_X\left(-\frac{\pi}{2}\right) \\ &\qquad + R_X\left(\frac{\pi}{2}\right)|1\rangle\langle 1|R_X\left(-\frac{\pi}{2}\right) \otimes R_X\left(\frac{\pi}{2}\right)R_Z(-\theta)R_X\left(-\frac{\pi}{2}\right) \\ &= |y_+\rangle\langle y_+| \otimes R_Y(\theta) + |y_-\rangle\langle y_-| \otimes R_Y(-\theta), \end{aligned}$$

where we have used the fact that $R_X(\pm\pi/2)R_Z(\theta)R_X(\mp\pi/2) = R_Y(\theta)$, and defined the notation $R_X(\pi/2)|0\rangle = (|0\rangle - i|1\rangle)/\sqrt{2} =: |y_+\rangle$ and $R_X(\pi/2)|1\rangle = (-i|0\rangle + |1\rangle)/\sqrt{2} =: |y_-\rangle$. Remark that $|y_+\rangle$ and $|y_-\rangle$ correspond to the basis states of the Pauli-Y operator with $Y = |y_+\rangle\langle y_+| - |y_-\rangle\langle y_-|$. Then, a $R_{YY}(\theta)$ gate tells us to apply a rotation of a Bloch vector measured in the Y-basis by an angle θ about the Y-axis of the Bloch sphere if the control qubit is $|y_+\rangle$ and to apply a rotation of angle $(-\theta)$ if the control qubit is $|y_-\rangle$.

Setting $\overline{C} := \text{CNOT}$, we can finally write an **overall circuit for the XX + YY gate**:

$$\begin{aligned} V_{XX+YY}(\theta,\beta) &= \big(\mathbb{I} \otimes R_Z(\beta)\big)R_{XX}(\theta)R_{YY}(\theta)\big(\mathbb{I} \otimes R_Z(-\beta)\big) \\ &= \big(\mathbb{I} \otimes R_Z(\beta)\big)H^{\otimes 2}\overline{C}\big(\mathbb{I} \otimes R_Z(\theta)\big)\overline{C}H^{\otimes 2}\left(R_X\left(\frac{\pi}{2}\right)^{\otimes 2}\right)\overline{C}\big(\mathbb{I} \otimes R_Z(\theta)\big)\overline{C}\left(R_X\left(-\frac{\pi}{2}\right)^{\otimes 2}\right)\big(\mathbb{I} \otimes R_Z(-\beta)\big). \end{aligned}$$

For future reference we also introduce the 2-qubit **controlled-Phase gate** $\text{cP}(\theta)$ which, as the name suggests,

applies a phase whenever the control qubit is $|11\rangle$. It can be verified that this gate may be implemented using the operator $\exp\big(i\theta(1-Z_u)(1-Z_v)/4\big)$.

$$\mathrm{cP}(\theta) := \mathrm{diag}\big(1,1,1,e^{i\theta}\big) = \left(R_Z\left(-\frac{\theta}{2}\right)\otimes R_Z\left(-\frac{\theta}{2}\right)H\right)R_{XX}\left(\frac{\theta}{2}\right)(\mathbb{I}\otimes H). \tag{3.18}$$

3.1.2 Givens Rotations & the QR Decomposition

In general, if one wants to implement the time-evolution operator of some Hamiltonian for Quantum Simulation, Trotterisation is often the method of choice due to its simplicity. However for this method to be precise, it usually requires a high number of small time-steps. As a result of this, Trotterisation tends to suffer from an accumulation of truncation errors. As we will shortly see, the implementation of operators having the form of Eq. (3.33) will become necessary. It was shown in [14] that such operators can be efficiently implemented using a QR decomposition of Givens rotations to counter the overhead of Trotterisation. In this subsection, we aim to briefly present Givens rotations before giving an efficient strategy for the decomposition of specific unitary transformations.

We formally define the **Givens rotation gate** (eliminating the (p,q)-th entry of a matrix) with the matrix representation (up to a sign convention)[15]:

$$\texttt{Givens}_{pq}(\theta_{pq}) = \begin{pmatrix} \mathbb{I}_{p-1} & & & & \\ & \cos(\theta_{pq}) & & \sin(\theta_{pq}) & \\ & & \mathbb{I}_{q-p-1} & & \\ & -\sin(\theta_{pq}) & & \cos(\theta_{pq}) & \\ & & & & \mathbb{I}_{N-q} \end{pmatrix}_{N\times N}, \tag{3.19}$$

where $\dim(\mathbb{I}_r) = \dim(\mathrm{diag}(1,\cdots,1)_r) = r$. We note, for reference[16], the interesting decomposition of fermionic Givens rotations expressed in terms of fermionic operators as a product of R_Z rotations and the so-called iSWAP gate

$$\mathrm{iSWAP} := R_{XX+YY}\left(-\frac{\pi}{2}\right) := \exp\Big(i\frac{\pi}{4}(X\otimes X + Y\otimes Y)\Big) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & i & 0 \\ 0 & i & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \tag{3.20}$$

given by:

$$\texttt{Givens}_{pq}(\theta_{pq}) \equiv \exp\big(\theta_{pq}\big(a_p^\dagger a_q - a_q^\dagger a_p\big)\big) = \sqrt{\mathrm{iSWAP}}^\dagger_{p,q} e^{-i\frac{\theta_{pq}}{2}Z_p} e^{i\frac{\theta_{pq}}{2}Z_q}\sqrt{\mathrm{iSWAP}}_{p,q}, \tag{3.21}$$

and where

$$\sqrt{\mathrm{iSWAP}}_{p,q} := \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{1}{\sqrt{2}} & \frac{i}{\sqrt{2}} & 0 \\ 0 & \frac{i}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}_{p,q}. \tag{3.22}$$

These gates play a crucial role in the implementation of some unitaries U since they can be decomposed using a QR decomposition of Givens rotations. The idea is to decompose the operator U into individual operators that can be implemented on a quantum circuit. To this end, we will decompose the unitary U into a sequence of Givens rotations using the QR decomposition

$$U = R\cdot\prod_{\langle pq\rangle}\texttt{Givens}_{pq}(\theta_{pq}), \tag{3.23}$$

such that the Givens rotations $\texttt{Givens}_{pq}(\theta_{pq})$ mixes the spatial orbitals p and q leaving the other orbitals untouched[17]. Here, the notation $\langle pq \rangle$ indicates that we will only iterate over required pairs of (p, q) for the decomposition and R is an upper-triangular matrix. Recalling that a Slater determinant may be represented by a matrix for which each row represents a single orbital, a Givens rotation induces a rotation between rows p and q of the matrix U allowing for the elimination of a single element in one of those rows[15]. Throughout the elimination procedure, the Givens rotations are always applied to neighbouring rows allowing for the elimination of elements in the second row. For instance, the following matrix demonstrates the elimination procedure via Givens rotations where the number describes the order of elimination:

$$\begin{pmatrix} * & * & * & * & * \\ 4 & * & * & * & * \\ 3 & 5 & * & * & * \\ 2 & 4 & 6 & * & * \\ 1 & 3 & 5 & 7 & * \end{pmatrix}. \tag{3.24}$$

In the above matrix, elements with the same number mean that they could be eliminated simultaneously while asterisks $(*)$ mark all upper-diagonal elements[14].

3.2 Qubit Mapping of the One-electron Hamiltonian

We study two different methods to map the one-electron Hamiltonian. A first approach demonstrates a direct mapping by explicitly substituting the qubit representations of the creation and annihilation operators. We then present a second approach by defining a diagonalization of the one-electron Hamiltonian in order to create new valid sets of orbitals. For now we consider spinless systems, however we bear in mind that in the subsequent chapters we will introduce spins to describe our systems.

3.2.1 First approach - Direct Mapping

We start with the sum $\hat{H}_1$ over the product of the creation and annihilation operators:

$$\begin{aligned}
\hat{H}_1 := \sum_{p,q} h_{pq} a_p^\dagger a_q &= \sum_p h_{pp} a_p^\dagger a_p + \sum_{p \neq q} h_{pq} a_p^\dagger a_q \\
&= \sum_p \frac{h_{pp}}{4} (X - iY)_p (X + iY)_p + \sum_{p \neq q} h_{pq} \frac{X_p - iY_p}{2} \otimes \left(\bigotimes_{k=p-1}^{0} Z_k \right) \otimes \frac{X_q + iY_q}{2} \otimes \left(\bigotimes_{j=q-1}^{0} Z_j \right) \\
&= \sum_p \frac{h_{pp}}{2} (\mathbb{I} - Z_p) + \sum_{p \neq q} \frac{h_{pq}}{4} \left(\bigotimes_{k=p-1}^{0} Z_k \right) \otimes \left(\bigotimes_{j=q-1}^{0} Z_j \right) \otimes \left[(X_p X_q + Y_p Y_q) + i(X_p Y_q - Y_p X_q) \right] \\
&= \sum_p \frac{h_{pp}}{2} (\mathbb{I} - Z_p) + \sum_{p < q} \frac{h_{pq}}{2} \left(\bigotimes_{k=p+1}^{q-1} Z_k \right) \otimes (X_p X_q + Y_p Y_q) \,,
\end{aligned}$$

where we have used the fact that terms of the form $a_p^\dagger a_q + a_q^\dagger a_p$ cancel cross-terms $X_p Y_q$ and $Y_p X_q$ in the imaginary part of the above expression (by symmetry). Of course, if one is interested in the time-evolution of this operator we can apply a Trotterisation to find:

$$e^{-i\Delta t \hat{H}_1} \mapsto \prod_\alpha e^{-i\Delta t P_\alpha} + \mathcal{O}(\Delta t^2). \tag{3.25}$$

As we will see shortly with another method, we can also implement $\hat{H}_1$ by first computing the parity of the Pauli-Z string before applying the so-called XY-gate $XY(t) := \exp\big(-i\frac{\pi}{2} t (X \otimes X + Y \otimes Y)\big)$. Interestingly, we

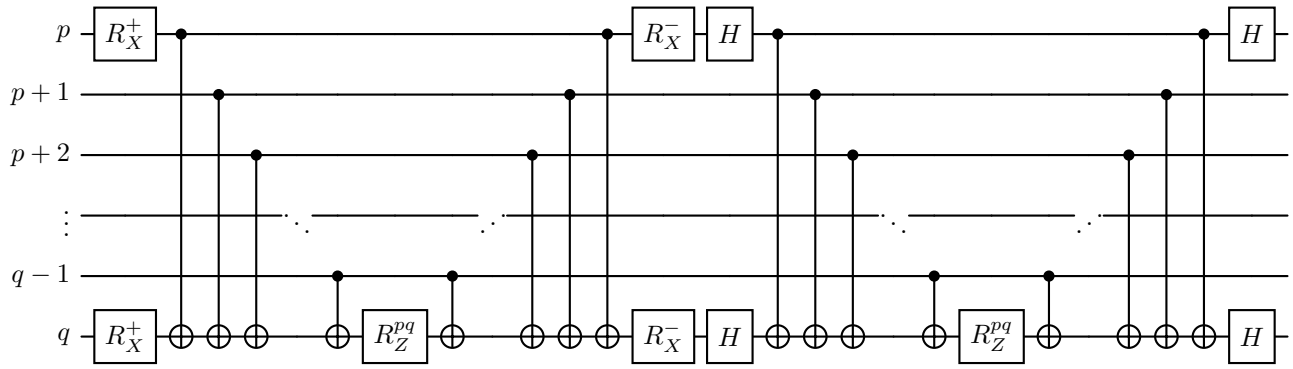


Figure 3.1: Quantum circuit for a single term (p, q) of the time-evolution of the $XX + YY$ term in $\hat{H}_1$ using Trotterisation by direct separate implementation of the Pauli strings $X_p Z_{p+1} \cdots Z_{q-1} X_q$ and $Y_p Z_{p+1} \cdots Z_{q-1} Y_q$, where $R_X^{\pm} := R_X(\pm\pi/2)$ and $R_Z^{pq} := R_Z(h_{pq})$. Above, we have used the facts $HXH = Z$ and $R_X(\pi/2) Y R_X(-\pi/2) = Z$ such that we only require the implementation of a Pauli-Z string. For the full implementation of the time-evolution of $\hat{H}_1$, one needs to account for all the indices $\{(p, q)\}_{p<q}$ and initialise the circuit with $\bigotimes_j R_Z(-h_{jj}\Delta t)$.

note that the XY-gate is locally equivalent to a Givens rotation gate (see [18, pp.39]) since:

$$\texttt{Givens}(\theta) = \left(T^\dagger \otimes T\right) XY \left(\frac{\theta}{\pi}\right) \left(T \otimes T^\dagger\right) \qquad \text{where } T = \begin{pmatrix} 1 & 0 \\ 0 & \exp(i\pi/4) \end{pmatrix}.$$

3.2.2 Unitary Linear Transformations

We first introduce a diagonalization of the matrix $h = \{h_{pq}\}_{p,q}$ such that $h = UVU^\dagger$ where U is a matrix of eigenvectors of h and V a diagonal matrix of its eigenvalues $\{\eta_p\}_p$. Now, let us define an operator $\widetilde{a}_p$ corresponding to a rotation of the creation operator a_p by U. We then have that $\widetilde{a}_p := U^\dagger a_p = \sum_r U^*_{pr} a_r$ and so $\widetilde{a}^\dagger_p := a^\dagger_p U = \sum_r U_{pr} a^\dagger_r$. Of course, it is clear that the inverse relations are $a_p = U\widetilde{a}_p = \sum_r U_{pr}\widetilde{a}_r$ and $a^\dagger_p = \widetilde{a}^\dagger_p U^\dagger = \sum_r U^*_{pr}\widetilde{a}^\dagger_r$. So:

$$\begin{aligned}
\hat{H}_1 = \sum_{p,q} h_{pq} a^\dagger_p a_q = \sum_{p,q} h_{pq} \left(\sum_r U^*_{pr}\widetilde{a}^\dagger_r\right)\left(\sum_w U_{qw}\widetilde{a}_w\right) &= \sum_{r,w}\left(\sum_{p,q} U^*_{pr} h_{pq} U_{qw}\right)\widetilde{a}^\dagger_r\widetilde{a}_w \\
&= \sum_{r,w} \left(U^\dagger h U\right)_{rw} \widetilde{a}^\dagger_r \widetilde{a}_w \\
&= \sum_{r,w} V_{rw}\widetilde{a}^\dagger_r\widetilde{a}_w \\
&= \sum_r \eta_r \widetilde{a}^\dagger_r \widetilde{a}_r,
\end{aligned}$$

where in the third line we have used the fact that if $h = UVU^\dagger$ then $V = U^\dagger h U$ and in the last equality that $V = \{\eta_r\}_r$ is diagonal. For the purpose of implementation on a QPU, we now want to define an operator $\widehat{K}$ such that $U = e^{\widehat{K}}$. Note that since we require U to be unitary, i.e. $UU^\dagger = \mathbb{I}$, then it must be that $\widehat{K}^\dagger = -\widehat{K}$. We want to define this new operator $\widetilde{a}_p$ as a transformation $\overline{a}_p = e^{-\widehat{K}} a_p e^{\widehat{K}}$ and $\overline{a}^\dagger_p = e^{-\widehat{K}} a^\dagger_p e^{\widehat{K}}$. Let us verify that this transformation is equivalent to $\widetilde{a}_p$ and $\widetilde{a}^\dagger_p$. To this end, we define the **orbital rotation operator** $\widehat{K} = \sum_{p,r} \kappa_{pr} a^\dagger_p a_r$ and use a Baker-Campbell-Hausdorff (BCH) formula

$$e^{-A} B e^{A} = B + [B, A] + \frac{1}{2!}[[B, A], A] + \frac{1}{3!}[[[B, A], A], A] + ... + \frac{1}{n!}[...[[B, A], A], ...] + ... \tag{3.26}$$

in order to derive an expansion for $\overline{a}_p$:

$$\overline{a}_p = e^{-\widehat{K}} a_p e^{\widehat{K}} = a_p + [a_p, \widehat{K}] + \frac{1}{2!}[[a_p, \widehat{K}], \widehat{K}] + \frac{1}{3!}[[[a_p, \widehat{K}], \widehat{K}], \widehat{K}] + ... + \frac{1}{n!}[...[[a_p, \widehat{K}], \widehat{K}], ...] + ... \quad (3.27)$$

To derive the individual commutators we use the fact that $\{a_p, a_r\} = 0$ and $\{a_p, a_p^\dagger\} = 1$ in order to find, for a fixed p:

$$[a_p, \hat{K}] := a_p\hat{K} - \hat{K}a_p = \sum_r \kappa_{pr}\big(a_p a_p^\dagger a_r - a_p^\dagger a_r a_p\big) = \sum_r \kappa_{pr}\big(a_p a_p^\dagger a_r + a_p^\dagger a_p a_r\big) = \sum_r \kappa_{pr}\{a_p, a_p^\dagger\}a_r = \sum_r \kappa_{pr} a_r. \quad (3.28)$$

$$[[a_p, \widehat{K}], \widehat{K}] = \sum_r \kappa_{pr}[a_r, \hat{K}] = \sum_{q,r} \kappa_{pr}\kappa_{rq} a_q = \sum_q [\hat{K}^2]_{pq} a_q. \quad (3.29)$$

And in general it is not difficult to show that for the n-fold nested commutator we have:

$$[...[[a_p, \widehat{K}], \widehat{K}], ...] = \sum_q [\hat{K}^n]_{pq} a_q. \quad (3.30)$$

As a result we find:

$$\begin{aligned}\overline{a}_p = e^{-\widehat{K}} a_p e^{\widehat{K}} &= \sum_q a_q \left(\delta_{pq} + \kappa_{pq} + \frac{1}{2!}[\hat{K}^2]_{pq} + \frac{1}{3!}[\hat{K}^3]_{pq} + \cdots + \frac{1}{n!}[\hat{K}^n]_{pq} + \cdots\right) \\ &= \sum_q a_q \left(\sum_{n\geq 0} \frac{1}{n!}[\hat{K}^n]_{pq}\right) = \sum_q a_q \big[e^{\hat{K}}\big]_{pq},\end{aligned}$$

corresponding precisely to the original expression of $\widetilde{a}_p$. Finally, we conclude that $\hat{H}_1$ may be diagonalized by the exponential of a one-body operator as follows:

$$\begin{aligned}\hat{H}_1 = \sum_r \eta_r \widetilde{a}_r^\dagger \widetilde{a}_r = \sum_r \eta_r \left(e^{-\widehat{K}} a_r e^{\widehat{K}}\right)^\dagger \left(e^{-\widehat{K}} a_r e^{\widehat{K}}\right) &= \sum_r \eta_r \left(e^{\widehat{K}^\dagger} a_r^\dagger e^{\left(-\widehat{K}\right)^\dagger}\right)\left(e^{-\widehat{K}} a_r e^{\widehat{K}}\right) \\ &= \sum_r \eta_r \left(e^{-\widehat{K}} a_r^\dagger e^{\widehat{K}}\right)\left(e^{-\widehat{K}} a_r e^{\widehat{K}}\right) \\ &= \sum_r \eta_r e^{-\widehat{K}} a_r^\dagger a_r e^{\widehat{K}} = e^{-\widehat{K}} \left(\sum_r \eta_r a_r^\dagger a_r\right) e^{\widehat{K}}.\end{aligned}$$

For future reference, we now rewrite the operator $\hat{K}$ in a convenient way for an implementation on a quantum processing unit (QPU). Note first $\widehat{K}^\dagger = \sum_{p,r} \kappa_{rp}^* a_r^\dagger a_p$ and since $\widehat{K}^\dagger = -\widehat{K}$ it must be that $\kappa_{rp}^* = -\kappa_{pr}$. Hence, we may rewrite $\hat{K}$ as follows:

$$\begin{aligned}\widehat{K} = \sum_{p,r} \kappa_{pr} a_p^\dagger a_r &= \sum_p \kappa_{pp} a_p^\dagger a_p + \sum_{p\neq r} \kappa_{pr} a_p^\dagger a_r \\ &= \sum_p \kappa_{pp} a_p^\dagger a_p + \underbrace{\sum_{p<r} \big(\kappa_{pr} a_p^\dagger a_r + \kappa_{rp} a_r^\dagger a_p\big)}_{\text{group terms } (p,r) \text{ and } (r,p)} \\ &= \sum_p \kappa_{pp} a_p^\dagger a_p + \sum_{p<r} \big(\kappa_{pr} a_p^\dagger a_r - \kappa_{pr}^* a_r^\dagger a_p\big).\end{aligned}$$

Let us now expand the coefficients $\{\kappa\}_{pr}$ into their real and imaginary parts $\kappa_{pr} = \mathfrak{Re}(\kappa_{pr}) + i\mathfrak{Im}(\kappa_{pr})$. Recall that $\kappa_{rp}^* = -\kappa_{pr}$, so writing $\kappa_{pr} = \mathfrak{Re}(\kappa_{pr}) + i\mathfrak{Im}(\kappa_{pr})$ and $\kappa_{rp}^* = \mathfrak{Re}(\kappa_{rp}) - i\mathfrak{Im}(\kappa_{rp})$ gives us that $\mathfrak{Re}(\kappa_{pr}) = -\mathfrak{Re}(\kappa_{rp})$ and $\mathfrak{Im}(\kappa_{pr}) = \mathfrak{Im}(\kappa_{rp})$. We then have:

$$\begin{cases}\kappa_{pr} = \mathfrak{Re}(\kappa_{pr}) + i\mathfrak{Im}(\kappa_{pr}) & (1) \\ \kappa_{rp} = \mathfrak{Re}(\kappa_{rp}) + i\mathfrak{Im}(\kappa_{rp}) = -\mathfrak{Re}(\kappa_{pr}) + i\mathfrak{Im}(\kappa_{pr}) & (2)\end{cases}$$

We now see that (1) + (2) yields $\mathfrak{Im}(\kappa_{pr}) = (\kappa_{pr} + \kappa_{rp})/(2i)$ and (1) − (2) gives $\mathfrak{Re}(\kappa_{pr}) = (\kappa_{pr} - \kappa_{rp})/2$. By then noting that $\kappa_{pp} = i\mathfrak{Im}(\kappa_{pp})$ (i.e. the diagonal of $\hat{K}$ is purely imaginary), using also (1) we are now able to rewrite $\hat{K}$ in terms of the real and imaginary parts of its coefficients[19]:

$$\begin{aligned}\hat{K} &= \sum_p \kappa_{pp} a_p^\dagger a_p + \sum_{p<r} \left(\kappa_{pr} a_p^\dagger a_r - \kappa_{pr}^* a_r^\dagger a_p\right) \\ &= i\sum_p \mathfrak{Im}(\kappa_{pp}) a_p^\dagger a_p + i\sum_{p<r} \mathfrak{Im}(\kappa_{pr})(a_p^\dagger a_r + a_r^\dagger a_p) + \sum_{p<r} \mathfrak{Re}(\kappa_{pr})(a_p^\dagger a_r - a_r^\dagger a_p).\end{aligned}$$

Thus, since we are considering real orbitals, we can only retain the real part of $\hat{K}$ so that our new (real) orbital operator is:

$$\mathfrak{Re}(\hat{K}) = \sum_{p<r} \mathfrak{Re}(\kappa_{pr})(a_p^\dagger a_r - a_r^\dagger a_p). \tag{3.31}$$

Remark 3.2.1. *In the literature, it is a convention to write the orbital operator without the real part notation, especially when the context is clear. It is then set to be* $\hat{K} = \sum_{p<r} \kappa_{pr}(a_p^\dagger a_r - a_r^\dagger a_p)$.

3.2.3 Second Approach - Unitary Linear Transformations

Let us present an alternative construction for $\hat{H}_1$ (see [14, pp.3] or [20]). Recall the Summary of the Hartree-Fock (HF) method for the approximation of the ground-state wavefunction with a number of restricted Slater determinants. To perform HF, the goal was to find the Fock operator associated to an input set of atomic orbitals which was then diagonalized to obtain new orbitals (i.e. linear combinations of the old orbitals). The process was repeated until the orbitals converged (in the sense that we have defined a localized set of orbitals). Let us see how the HF procedure can be mapped onto a quantum computer. The key idea is that performing a unitary rotation of the initial (core) orbital basis $\varphi_p(r)$, one can obtain a new valid set of orbitals $\widetilde{\varphi}_p(r)$ as a linear combination of the initial ones:

$$\widetilde{\varphi}_p(r) = \sum_q \left[e^{\hat{K}}\right]_{pq} \varphi_q(r), \tag{3.32}$$

where $\widehat{K} = \sum_{p,r} \kappa_{pr} a_p^\dagger a_r$ is an anti-Hermitian matrix (i.e. $\widehat{K}^\dagger = -\widehat{K}$) and $\left[e^{\hat{K}}\right]_{pq}$ is the (p,q) element of the matrix exponential of $\widehat{K}$[16]. Now, as we have seen in Subsection (2.2.2), any particle-conserving rotation of the single-particle basis can be expressed using $\widetilde{\varphi}_p = \sum_q U_{pq}\varphi_q$, $\widetilde{a}_p := U^\dagger a_p = \sum_r U_{pr}^* a_r$ and $\widetilde{a}_p^\dagger := a_p^\dagger U = \sum_r U_{pr} a_r^\dagger$ where these new operators correspond to spin-orbitals and operators in the rotated basis generated by the $N \times N$ unitary matrix U. From the Thouless Theorem, this single-particle rotation is equivalent to applying the $2^N \times 2^N$ operator

$$\mathcal{U}(U) = \exp\left(\sum_{pq} [\log(U)]_{pq}(a_p^\dagger a_q - a_q^\dagger a_p)\right), \tag{3.33}$$

where $[\log(U)]_{pq}$ is the (p,q) element of the matrix $\log(U)$[14]. To efficiently implement $\mathcal{U}(U)$ without the overhead of Trotterization, we will decompose it using a QR decomposition of $\binom{N}{2}$ Givens rotation gates of the form

$$R_{pq}(\theta_{pq}) = \exp\big(\theta_{pq}(a_p^\dagger a_q - a_q^\dagger a_p)\big). \tag{3.34}$$

As before, we still have the mapping $\sum_r \eta_r a_r^\dagger a_r \overset{\text{qubit}}{\mapsto} \sum_r \frac{\eta_r}{2}(\mathbb{I} - Z_r)$. Now, let us look at how we can encode the matrix $e^{\widehat{K}}$ on a quantum circuit. Recall that $a_p \mapsto \left(\prod_{k=0}^{p-1} Z_k\right)\left(\frac{X_p + iY_p}{2}\right)$, $a_p^\dagger \mapsto \left(\prod_{k=0}^{p-1} Z_k\right)\left(\frac{X_p - iY_p}{2}\right)$ in the Jordan-Wigner transformation and $\widehat{K} = \sum_{p<r} \kappa_{pr}\big(a_p^\dagger a_r - a_r^\dagger a_p\big)$. Then, for $p < r$, it is clear that:

$$a_p^\dagger a_r \overset{\text{qubit}}{\mapsto} \left(\prod_{k=0}^{p-1} Z_k\right)\left(\frac{X_p - iY_p}{2}\right)\left(\prod_{k=0}^{r-1} Z_k\right)\left(\frac{X_r + iY_r}{2}\right) = \frac{1}{4}\left(\prod_{k=p+1}^{r-1} Z_k\right)\big((X_pX_r + Y_pY_r) + i(X_pY_r - Y_pX_r)\big)$$

and,

$$a_r^\dagger a_p \overset{\text{qubit}}{\mapsto} \frac{1}{4}\left(\prod_{k=p+1}^{r-1} Z_k\right)\big((X_pX_r + Y_pY_r) + i(X_rY_p - Y_rX_p)\big) = \frac{1}{4}\left(\prod_{k=p+1}^{r-1} Z_k\right)\big((X_pX_r + Y_pY_r) + i(Y_pX_r - X_pY_r)\big)$$
$$= \frac{1}{4}\left(\prod_{k=p+1}^{r-1} Z_k\right)\big((X_pX_r + Y_pY_r) - i(X_pY_r - Y_pX_r)\big)$$

where in the third equality we have used the fact that Pauli operators acting on different qubits commute, thus $X_rY_p = Y_pX_r$ and $Y_rX_p = X_pY_r$. As a result:

$$\widehat{K} = \sum_{p<r} \kappa_{pr}\big(a_p^\dagger a_r - a_r^\dagger a_p\big) \overset{\text{qubit}}{\mapsto} \frac{i}{2}\sum_{p<r}\kappa_{pr}\left(\prod_{k=p+1}^{r-1} Z_k\right)(X_pY_r - Y_pX_r).$$

Considering the exponential of this operator leads, for instance, to a Trotter decomposition in the form (up to a global phase):

$$\exp\big(\widehat{K}\big) = \exp\left(\sum_{p<r}\kappa_{pr}\big(a_p^\dagger a_r - a_r^\dagger a_p\big)\right) \overset{\text{qubit}}{\mapsto} \exp\left(\frac{i}{2}\sum_{p<r}\kappa_{pr}\left(\prod_{k=p+1}^{r-1} Z_k\right)(X_pY_r - Y_pX_r)\right) \tag{3.35}$$
$$\simeq \left(\prod_{p<r}\exp\left(\frac{i\kappa_{pr}}{2}\left(\prod_{k=p+1}^{r-1} Z_k\right)(X_pY_r - Y_pX_r)\right)\right). \tag{3.36}$$

There are multiple possible implementations of this operator, some methods being more efficient and less costly than others. For instance, as previously seen in Subsection (2.2.1), a direct implementation of the time evolutions of $X_pZ_{p+1}\cdots Z_{r-1}Y_r$ and $Y_pZ_{p+1}\cdots Z_{r-1}X_r$ separately then requires the simple implementation of a Pauli-Z string. As an alternative, we may also compute the parity of the Z-string $Z_{p+1}\cdots Z_{r-1}$ first and then apply the Givens rotation gate[18] in the Pauli basis. Let us see how can this second method be implemented on a quantum computer with and without a Trotter decomposition.

• Based on a Trotter formula, we can implement Eq. (3.36) in terms of controlled-Z gates and Givens rotations[21]. The idea is to implement each exponential in the product with the following Jordan-Wigner transformation:

$$\exp\big(\widehat{K}\big) \simeq \exp\left(i\frac{\kappa_{pr}}{2}\left(\prod_{k=p+1}^{r-1} Z_k\right)(X_pY_r - Y_pX_r)\right) \overset{\text{qubit}}{\mapsto} \left(\prod_{k=p+1}^{r-1}\widehat{CZ}_{rk}\right)\texttt{Givens}(\kappa_{pr})\left(\prod_{k=p+1}^{r-1}\widehat{CZ}_{rk}\right), \tag{3.37}$$

such that $\widehat{CZ}_{rk}$ is a controlled-Z gate with control qubit r and $\texttt{Givens}(\kappa_{pr}) := \exp(i(\kappa_{pr}/2)(X_pY_r - Y_pX_r))$ where in the computational basis we have

$$\texttt{Givens}(\kappa_{pr}) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\kappa_{pr}) & -\sin(\kappa_{pr}) & 0 \\ 0 & \sin(\kappa_{pr}) & \cos(\kappa_{pr}) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Let us briefly justify how to find the transformation (3.37) based on [22, pp.10-15] where the full details may be found. Note that the Pauli-Z string accounts for a change of sign due to the anticommutation relations of the fermionic creation and annihilation operators. As shown in [22], this parity may be implemented on a

QPU via a product of nearest neighbour fermionic-SWAP (f-SWAP) operators $\hat{\mathcal{F}}_{r\delta(r)}$ that transforms a fermion creation operator $a_r^\dagger$ in such a way that $\hat{\mathcal{F}}_{r\delta(r)} a_r^\dagger \hat{\mathcal{F}}_{r\delta(r)}^{-1} = a_{\delta(r)}^\dagger$. This nearest neighbour f-SWAP gate acts in the computational basis in the following way:

$$\begin{aligned}
\hat{\mathcal{F}}_{r\delta(r)} |0\rangle_r |0\rangle_{\delta(r)} &= |0\rangle_r |0\rangle_{\delta(r)} , \\
\hat{\mathcal{F}}_{r\delta(r)} |0\rangle_r |1\rangle_{\delta(r)} &= |1\rangle_r |0\rangle_{\delta(r)} , \\
\hat{\mathcal{F}}_{r\delta(r)} |1\rangle_r |0\rangle_{\delta(r)} &= |0\rangle_r |1\rangle_{\delta(r)} , \\
\hat{\mathcal{F}}_{r\delta(r)} |1\rangle_r |1\rangle_{\delta(r)} &= -|1\rangle_r |1\rangle_{\delta(r)} ,
\end{aligned}$$

for which a matrix representation and an equivalent circuit are given by:

$$\hat{\mathcal{F}}_{r\delta(r)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \mapsto \mathrm{SWAP}_{r\delta(r)} \widehat{CZ}_{r\delta(r)} \equiv \widehat{CZ}_{r\delta(r)} \mathrm{SWAP}_{r\delta(r)}. \tag{3.38}$$

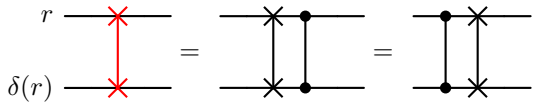


Figure 3.2: Fermionic SWAP (f-SWAP) $\hat{\mathcal{F}}_{r\delta(r)}$ gate decomposition on nearest neighbours. We denote the gate f-SWAP with a red SWAP gate. The black wire with two controls denotes a CZ gate.

One can then show that the corresponding long-range f-SWAP gate $\hat{\mathcal{F}}_{rj}$ may be implemented as a product of nearest neighbour f-SWAP operators in the following way:

$$\hat{\mathcal{F}}_{rj} \mapsto \left(\prod_{r \le k \le j} \widehat{CZ}_{jk} \right) \hat{f}_{rj} \left(\prod_{r \le k \le j} \widehat{CZ}_{jk} \right), \qquad \hat{f}_{rj} = \mathrm{SWAP}_{rj} \widehat{CZ}_{rj} = \widehat{CZ}_{rj} \mathrm{SWAP}_{rj}. \tag{3.39}$$

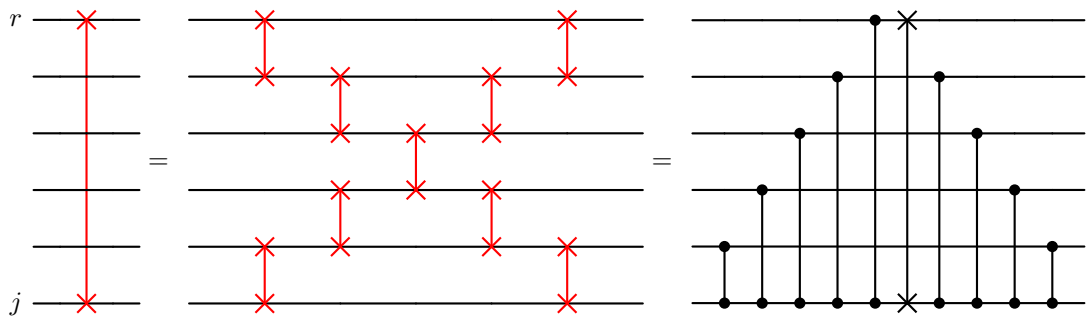


Figure 3.3: Long-range Fermionic SWAP (f-SWAP) $\hat{\mathcal{F}}_{rj}$ gate decomposition in terms of nearest-neighbour f-SWAP gates. We denote the gate f-SWAP with a red SWAP gate. The black wire with two controls denotes a CZ gate. There are $|r-j|$ CZ gates at the left of the SWAP gate and $|r-j|-1$ at the right. See a proof for this chain of equalities in [22, pp.12].

Now, for simplicity, denote by $\hat{Z}_{\mathrm{JW},rj} = \prod_{r<k<j} Z_k$ the product of the Z-string. Then, consider the time-evolution operator of this product against another Hermitian operator $\hat{h}_{rj}$ satisfying $\left[\hat{h}_{rj}, \hat{Z}_{\mathrm{JW},rj}\right] = 0$. Note that in the Jordan-Wigner encoding, $\hat{h}_{rj}\hat{Z}_{\mathrm{JW},rj}$ is Hermitian and quadratic in terms of fermion operators, satisfying $\hat{\mathcal{F}}_{kj}\hat{h}_{rj}\hat{Z}_{\mathrm{JW},rj}\hat{\mathcal{F}}_{kj} = \hat{h}_{rk}\hat{Z}_{\mathrm{JW},rk}$. On the one hand, knowing that

$$\hat{\mathcal{F}}_{kj} \exp\left(-i\hat{h}_{rj}\hat{Z}_{\mathrm{JW},rj}\right)\hat{\mathcal{F}}_{kj} = \exp\left(-i\hat{h}_{rk}\hat{Z}_{\mathrm{JW},rk}\right)$$

and $\hat{Z}_{JW,r\delta(r)} = \mathbb{I}$, implies that by using two long-range f-SWAP gates $\hat{\mathcal{F}}_{\delta(r)j}$ we can remove the Jordan-Wigner string $\hat{Z}_{JW,rj}$ from the exponent thus giving us:

$$\exp\left(-i\hat{h}_{rj}\hat{Z}_{\mathrm{JW},rj}\right) = \hat{\mathcal{F}}_{\delta(r)j}\exp\left(-i\hat{h}_{r\delta(r)}\right)\hat{\mathcal{F}}_{\delta(r)j}, \qquad \text{for } r < \delta(r) < j. \tag{3.40}$$

Finally, by representing the long-range f-SWAP gates as a product of $\widehat{CZ}$ and SWAP gates, by also using $\widehat{CZ}_{kl}^2 = \mathbb{I}$ we can cancel the redundant $\widehat{CZ}_{kl}$ gates in order to find:

$$\exp\left(-i\hat{h}_{rj}\hat{Z}_{\mathrm{JW},rj}\right) = \left(\prod_{r<k<j}\widehat{CZ}_{jk}\right)\exp\left(-i\hat{h}_{rj}\right)\left(\prod_{r<k<j}\widehat{CZ}_{jk}\right). \tag{3.41}$$

As result, letting $\hat{h}_{rj} := -(\kappa_{rj}/2)(X_rY_j - Y_rX_j)$ in Eq. (3.41) yields the transformation from Eq. (3.37).

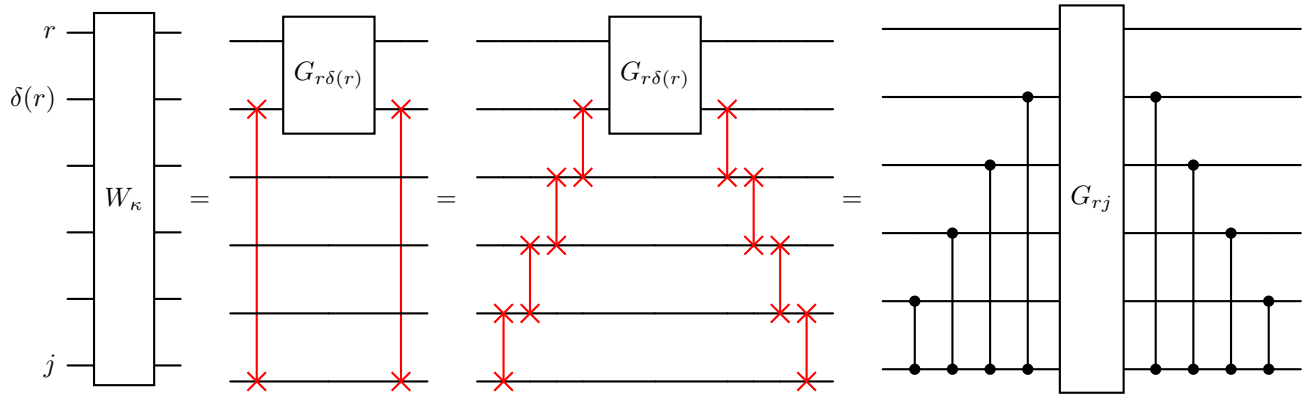


Figure 3.4: Quantum circuit for the operator $\exp\left(\hat{K}\right) \mapsto W_\kappa$, where $G_{pq} := \texttt{givens}(\kappa_{pq})$. We denote the gate f-SWAP with a red SWAP gate. The black wire with two controls denotes a CZ gate. There are $|r-j|-1$ f-SWAP gates both at the left and right of the Givens rotation gate $G_{r\delta(r)}$. There are $|r-j|-1$ CZ gates both at the left and right of the Givens rotation gate G_{rj}. See a proof for this chain of equalities in [22, pp.12].

We also remark that we can further decompose $\texttt{Givens}(\kappa_{pr})$ in terms of the XY gate to match the transformation V_{XX+YY} in [13, pp.8]:

$$\texttt{Givens}(\kappa_{pr}) = \left(\mathbb{I}\otimes R_Z\left(\frac{\pi}{2}\right)\right)\exp\left(i\frac{\kappa_{pr}}{2}(X_pX_r + Y_pY_r)\right)\left(\mathbb{I}\otimes R_Z\left(-\frac{\pi}{2}\right)\right) := V_{XX+YY}\left(-\kappa_{pr}, \frac{\pi}{2}\right). \tag{3.42}$$

• Let us finally present a method to implement $\exp\left(\hat{K}\right)$ based on a QR decomposition of Givens rotations, and thus without the use of Trotterisation. Notice that $\mathcal{U}\left(\texttt{Givens}_{pq}(\theta_{pq})\right) = R_{pq}(\theta_{pq})$ which holds by the definition of the matrix logarithm. Also note that the map $\mathcal{U}$ is a homomorphism under matrix multiplication, i.e. $\mathcal{U}(U_a)\cdot\mathcal{U}(U_b) = \mathcal{U}(U_a\cdot U_b)$ (see [14] for a proof). From this observation and Eq. (3.19) we conclude that

$$R_{pq}(\theta_{pq})\mathcal{U}(U) = \mathcal{U}\left(\texttt{Givens}_{pq}(\theta_{pq})\cdot U\right). \tag{3.43}$$

In light of this fact, the QR decomposition presented in the work of Kivlichan et al.[14] that avoids the use of non-local Jordan-Wigner strings, i.e. a QR decomposition on adjacent columns only (see Subsection (2.1.2)), is set to be:

$$R = U\cdot\prod_{j,\langle pq\rangle}\texttt{Givens}_{pq}^{(j)}(\theta_{pq}) \qquad \text{for } R = \sum_{k=1}^{N} e^{i\phi_k}\,|k\rangle\langle k|. \tag{3.44}$$

Here, the superscript (j) over the Givens rotation gate corresponds to the order of the sequence in which the rotations (eliminations) are carried out and R is a diagonal matrix containing unit phases with $\{|k\rangle\}_{k=1}^N$ being unit vectors (i.e. basis functions). Therefore, we understand that in order to implement U on a quantum

computer we must invert the Givens rotations by using the fact that $\big(\texttt{Givens}^{(j)}_{pq}(\theta_{pq})\big)^{-1} = \texttt{Givens}^{(j)}_{pq}(-\theta_{pq})$. Also, since we must eliminate the lower diagonal part of the matrix U, we will have $\binom{N}{2}$ Givens gates to implement for a total number of $(2N-3)$ layers. As a result, using the homomorphism property of $\mathcal{U}$, we have that:

$$\mathcal{U}(U) = \mathcal{U}(R) \cdot \prod_{j,\langle pq\rangle} \mathcal{U}\big(\texttt{Givens}^{(j)}_{pq}(-\theta_{pq})\big). \tag{3.45}$$

Now it is not difficult to see that $R = \text{diag}\big(e^{i\phi_1}, e^{i\phi_2}, \cdots, e^{i\phi_N}\big)$ and so, by definition of $\mathcal{U}$ in Eq. (3.33) we have:

$$\mathcal{U}(R) = \mathcal{U}\left(\sum_{k=1}^{N} e^{i\phi_k} |k\rangle \langle k|\right) = \exp\left(i \sum_{k=1}^{N} \phi_k a_k^\dagger a_k\right) = \prod_{k=1}^{N} e^{i\phi_k n_k}, \tag{3.46}$$

where $n_k := a_k^\dagger a_k$ is the so-called fermionic number operator. Moreover using the notation $R^{(j)}_{pq}(-\theta_{pq})$, where the superscript (j) over $R_{pq}(-\theta_{pq})$ (see Eq. (3.34)) corresponds to the order of the sequence in which the rotations (eliminations) are carried out, it is not difficult to see that:

$$\mathcal{U}\big(\texttt{Givens}^{(j)}_{pq}(-\theta_{pq})\big) = R^{(j)}_{pq}(-\theta_{pq}). \tag{3.47}$$

Overall, we conclude that $\mathcal{U}(U)$ may be written as:

$$\mathcal{U}(U) = \left(\prod_{k=1}^{N} e^{i\phi_k n_k}\right)\left(\prod_{j,\langle pq\rangle} R^{(j)}_{pq}(-\theta_{pq})\right), \tag{3.48}$$

where the product of the R_{pq} is over all relevant couples of (p,q). The final idea is that, to prevent gates in Eq. (3.48) to act on distant qubits, we will implement the Givens gates in parallel and also note that typically the diagonal matrix has all elements equal to 1 except the last bottom right-element (due to phase rotations and unitary U)[15]. As a result, we can rewrite Eq. (3.48) on adjacent qubits in the following way:

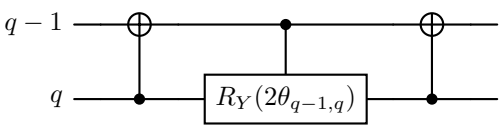
$$\mathcal{U}(U) = \prod_{j}\left(\prod_{q\in \text{Step}_j} R^{(j)}_{q-1,q}(-\theta_{q-1,q})\right) \cdot P_N(-\phi_N), \tag{3.49}$$

where $P_N(-\phi_N) := \exp(i\phi_N n_N)$. Here, Step_j denotes the row indices of the elements eliminated in parallel at the j-th step. Now as usual, we use the Jordan-Wigner encoding to produce a qubit representation of each operator where we have that[15]:

$$P_p(-\phi_p) \overset{\text{qubit}}{\mapsto} \mathbb{I}_1 \otimes \cdots \otimes \mathbb{I}_{p-1} \otimes \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi_p} \end{pmatrix} \otimes \mathbb{I}_{p+1} \otimes \cdots \otimes \mathbb{I}_N. \tag{3.50}$$

$$R_{q-1,q}(-\theta_{q-1,q}) \overset{\text{qubit}}{\mapsto} \mathbb{I}_1 \otimes \cdots \otimes \mathbb{I}_{q-2} \otimes \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta_{q-1,q}) & \sin(\theta_{q-1,q}) & 0 \\ 0 & -\sin(\theta_{q-1,q}) & \cos(\theta_{q-1,q}) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \otimes \mathbb{I}_{q+1} \cdots \otimes \mathbb{I}_N \tag{3.51}$$

$=$



, (3.52)

where we recognise the Givens rotation $\texttt{Givens}(-\theta_{q-1,q})$ in Eq. (3.51). As a result, the quantum circuit for $\mathcal{U}(U)$ representing the operator $\exp\big(\hat{K}\big)$ is given by:

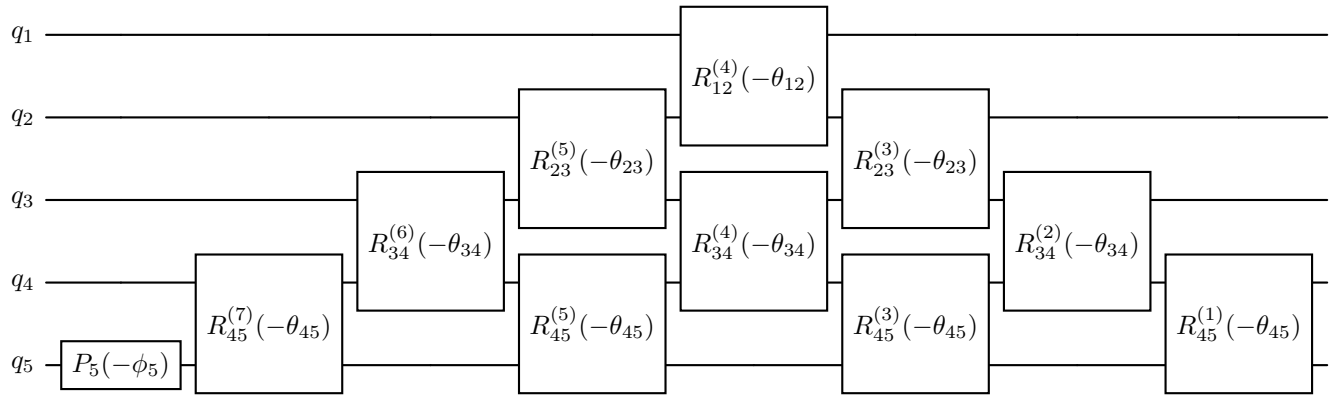


Figure 3.5: Quantum circuit for the operator $\exp\left(\hat{K}\right) \mapsto W_\kappa$ based on a QR decomposition of Givens rotations, in the case of the 5×5 matrix presented in Eq. (3.24).

Remark 3.2.2. *We note that Jiang et al.*[23] *have proposed some improvements in order to reduce the circuit complexity by eliminating a number of Givens rotations (a brief summary of the method is provided in [17, pp.3]).*

Finally, given that $\hat{H}_1 \mapsto W_\kappa^\dagger\left(\sum_r \frac{\eta_r}{2}(\mathbb{I} - Z_r)\right)W_\kappa$ it is not difficult to see that $\exp\left(-it\hat{H}_1\right) \mapsto W_\kappa^\dagger U_1(t\boldsymbol{\eta})W_\kappa$, where $U_1(t\boldsymbol{\eta}) = \bigotimes_p R_{Z_p}(-t\eta_p)$. Consequently, the circuit corresponding to the diagonalisation of the time evolution of $\hat{H}_1$ is given by:

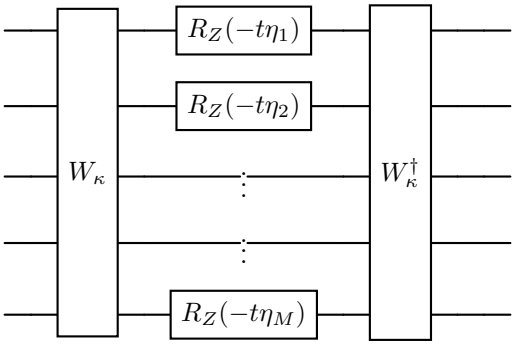


Figure 3.6: Quantum circuit for the time evolution $\exp\left(-it\hat{H}_1\right)$ where $\exp\left(\hat{K}\right) \mapsto W_\kappa$.

3.3 Qubit Mapping of the Two-electron Hamiltonian

After applying the Jordan-Wigner transformation, the second term of the Hamiltonian satisfies[24]:

$$h_{pqrs}a_p^\dagger a_q^\dagger a_r a_s + h_{spqr}a_s^\dagger a_r^\dagger a_p a_q \overset{\text{qubit}}{\mapsto} \left(\bigotimes_{k=s+1}^{r-1} Z_k\right)\left(\bigotimes_{k=q+1}^{p-1} Z_k\right) \times$$
$$\left(\frac{\Re(h_{pqrs})}{8}\left(X_sX_rX_qX_p - X_sX_rY_qY_p + X_sY_rX_qY_p + Y_sX_rX_qY_p + Y_sX_rY_qX_p - Y_sY_rX_qX_p + X_sY_rY_qX_p + Y_sY_rY_qY_p\right)\right) \times$$
$$\left(\frac{\Im(h_{pqrs})}{8}\left(Y_sX_rX_qX_p + X_sY_rX_qX_p - X_sX_rY_qX_p - X_sY_rY_qY_p - Y_sX_rY_qY_p + Y_sY_rX_qY_p + Y_sY_rX_qY_p + Y_sY_rY_qX_p\right)\right),$$

where an operator A can always be decomposed into two self-adjoint components $A = A_1 + iA_2$ such that $A_1 = (A + A^\dagger)/2 =: \mathfrak{R}(A)$ and $A_2 = -i(A - A^\dagger)/2 =: \mathfrak{I}(A)$[25]. Also note that this two-electron integral has an eight-fold symmetry satisfying $h_{pqrs} = h_{srqp} = h_{pqsr} = h_{qprs} = h_{qpsr} = h_{rsqp} = h_{rspq} = h_{srpq}$. This mapping procedure can be done directly by a software for quantum computation that will take care of this transformation. Note however that, although the calculations of these coefficients is straightforward, their storage can pose some difficulties since for a system comprising of N orbitals we have N^4 two-electron terms (compared to N^2 terms for one-electron coefficients). For an efficient implementation, one can decompose the electron repulsion integral using a Cholesky decomposition

$$\frac{h_{pqrs}}{2} \simeq \sum_{\gamma=1}^{N_\gamma} L_{pr}^\gamma L_{qs}^\gamma \; ; \quad \text{where } N_\gamma = \mathcal{O}(M), \tag{3.53}$$

for M the number of occupied spatial orbitals in the Fock space of electrons. This decomposition is possible since the real-valued matrix is symmetric due to the 8-fold symmetry of the electronic repulsion integral and thus is positive-definite[13].

3.4 Qubit Mapping of the Full Chemistry Hamiltonian

Let us now end this chapter on the full qubit representation of the Chemistry Hamiltonian using the three previous sections. Let us define an operator $\hat{L}_\gamma$ as

$$\hat{L}_\gamma := \sum_{p,r} L_{pr}^\gamma a_p^\dagger a_r. \tag{3.54}$$

Now, recall that the fermionic operators satisfy an anti-commutation relation $\{a_r, a_q^\dagger\} = \delta_{rq}$. As a result, $a_r a_q^\dagger = \delta_{rq} - a_q^\dagger a_r$ and finally we have that $a_p^\dagger a_r a_q^\dagger a_s = \delta_{rq} a_p^\dagger a_s - a_p^\dagger a_q^\dagger a_r a_s$. We now expand $\hat{L}_\gamma^2$ as follows:

$$\hat{L}_\gamma^2 = \left(\sum_{p,r} L_{pr}^\gamma a_p^\dagger a_r\right)^2 = \sum_{p,r,q,s} L_{pr}^\gamma L_{qs}^\gamma a_p^\dagger a_r a_q^\dagger a_s = \sum_{p,r,s} L_{pr}^\gamma L_{rs}^\gamma a_p^\dagger a_s - \sum_{p,r,q,s} L_{pr}^\gamma L_{qs}^\gamma a_p^\dagger a_q^\dagger a_r a_s. \tag{3.55}$$

As a result, since we have considered the Cholesky decomposition (3.53), we can match the structure of the electronic Hamiltonian (2.32) in terms of this decomposition using (3.55):

$$\begin{aligned}\hat{H} = \sum_{p,q} h_{pq} a_p^\dagger a_q + \frac{1}{2}\sum_{p,q,r,s} h_{pqrs} a_p^\dagger a_q^\dagger a_r a_s &\overset{(3.53)}{\simeq} \sum_{p,q} h_{pq} a_p^\dagger a_q + \sum_{p,r,q,s}\sum_{\gamma=1}^{N_\gamma} L_{pr}^\gamma L_{qs}^\gamma a_p^\dagger a_q^\dagger a_r a_s \\ &\overset{(3.55)}{=} \sum_{p,r}\left(h_{pr} + \sum_q \sum_{\gamma=1}^{N_\gamma} L_{pq}^\gamma L_{qr}^\gamma\right) a_p^\dagger a_r - \sum_{\gamma=1}^{N_\gamma} \hat{L}_\gamma^2 =: \hat{J} - \sum_{\gamma=1}^{N_\gamma} \hat{L}_\gamma^2.\end{aligned}$$

Thus, we can similarly as before diagonalize all the matrices forming the Hamiltonian in order to find a qubit representation of $\hat{H}$. Recalling the mapping $\sum_r \xi_r a_r^\dagger a_r \overset{\text{qubit}}{\mapsto} \sum_r \frac{\xi_r}{2}(\mathbb{I} - Z_r)$, denoting by $\{\xi_p\}_p$ the eigenvalues of $\hat{J}$ and $\{\lambda_p^\gamma\}_p$ the eigenvalues of $\hat{L}_\gamma$ we obtain:

$$\hat{H} = W_{\kappa_0}^\dagger \left(\sum_p \xi_p a_p^\dagger a_p\right) W_{\kappa_0} - \sum_{\gamma=1}^{N_\gamma} W_{\kappa_\gamma}^\dagger \left(\sum_{p,q} \lambda_p^\gamma \lambda_q^\gamma a_p^\dagger a_p a_q^\dagger a_q\right) W_{\kappa_\gamma} \tag{3.56}$$

$$\overset{\text{qubit}}{\mapsto} W_{\kappa_0}^\dagger \left(\sum_p \frac{\xi_p}{2}(\mathbb{I} - Z_p)\right) W_{\kappa_0} - \sum_{\gamma=1}^{N_\gamma} W_{\kappa_\gamma}^\dagger \left(\sum_{p,q} \frac{\lambda_p^\gamma \lambda_q^\gamma}{4}(\mathbb{I} - Z_p)(\mathbb{I} - Z_q)\right) W_{\kappa_\gamma} =: \sum_{\gamma=0}^{N_\gamma} \hat{H}_{\kappa_\gamma}, \tag{3.57}$$

such that the matrices $\{W_{\kappa_\gamma}\}_{\gamma=0}^{N_\gamma}$ are the matrices of eigenvectors diagonalizing $\hat{J}$ and $\{\hat{L}_\gamma\}_\gamma$. For convenience, we have compressed the Hamiltonian $\hat{H}$ into single Hamiltonians $\{\hat{H}_{\kappa_\gamma}\}_{\gamma=0}^{N_\gamma}$ that we will now study indepen-

dently. Since we will be interested in the time evolution operator of the Hamiltonian $\hat{H}$, we will make use of a Trotter decomposition in order to find a corresponding circuit representation:

$$e^{-i\Delta t\hat{H}} \simeq \prod_{\gamma=0}^{N_\gamma} e^{-i\Delta t\hat{H}_{\kappa_\gamma}} + \mathcal{O}(\Delta t^2). \tag{3.58}$$

Note that, as before, the time evolution operator of $\hat{H}_{\kappa_0}$ may be implemented via the qubit transformation $\exp\big(-it\hat{H}_{\kappa_0}\big) \mapsto W_{\kappa_0}^\dagger U_1(t\boldsymbol{\xi})W_{\kappa_0}$ where $U_1(t\boldsymbol{\xi}) = \bigotimes_p R_{Z_p}(-t\xi_p)$. For the second product over the sub-Hamiltonians $\{\hat{H}_{\kappa_\gamma}\}_{\gamma=1}^{N_\gamma}$, it is not difficult to see that their time-evolutions can be implemented via a product of 2-qubit controlled-phase gates $\mathrm{cP}\big(t\lambda_p^\gamma\lambda_q^\gamma\big)$ for all (p,q) (see (3.18)). Equivalently, we can first make some simplifications. Using the Trotter decomposition and recalling that $Z_p := \mathrm{diag}(1,-1)_p$, we have:

$$\begin{aligned}
\exp\left(i\Delta t\sum_{p,q}\frac{\lambda_p^\gamma\lambda_q^\gamma}{4}(\mathbb{I}-Z_p)(\mathbb{I}-Z_q)\right) &\simeq \prod_{p,q}\exp\left(i\Delta t\frac{\lambda_p^\gamma\lambda_q^\gamma}{4}(\mathbb{I}-Z_p)(\mathbb{I}-Z_q)\right) + \mathcal{O}(\Delta t^2) \\
&= \left(\prod_p \exp\left(i\Delta t\frac{\big(\lambda_p^\gamma\big)^2}{4}(\mathbb{I}-Z_p)^2\right)\right)\left(\prod_{p\neq q}\exp\left(i\Delta t\frac{\lambda_p^\gamma\lambda_q^\gamma}{4}(\mathbb{I}-Z_p)(\mathbb{I}-Z_q)\right)\right) + \mathcal{O}(\Delta t^2) \\
&\equiv \underbrace{\left(\prod_p \exp\left(i\Delta t\frac{\big(\lambda_p^\gamma\big)^2}{2}(\mathbb{I}-Z_p)\right)\right)}_{(*)}\underbrace{\left(\prod_{p<q}\exp\left(i\Delta t\frac{\lambda_p^\gamma\lambda_q^\gamma}{2}(\mathbb{I}-Z_p)(\mathbb{I}-Z_q)\right)\right)}_{(**)} + \mathcal{O}(\Delta t^2).
\end{aligned}$$

Now, as usual, the term $(*)$ may be implemented (up to a global phase) by a series of R_Z rotation gates represented by the operator $U_1(-t(\boldsymbol{\lambda}^\gamma)^2) = \bigotimes_p R_{Z_p}\big(t\big(\lambda_p^\gamma\big)^2\big)$. The term $(**)$ on the other hand can still be implemented with $M(M-1)$ controlled-phase gates $\mathrm{cP}\big(-2t\lambda_p^\gamma\lambda_q^\gamma\big)$ for all $p<q$. Thus, for all $\gamma\in\{1,\dots,N_\gamma\}$, the time-evolution operator of $\hat{H}_{\kappa_\gamma}$ may be implemented via the qubit transformation $\exp\big(-it\hat{H}_{\kappa_\gamma}\big) \mapsto W_{\kappa_\gamma}^\dagger U_2(t\boldsymbol{\lambda}^\gamma)W_{\kappa_\gamma}$ such that:

$$U_2(t\boldsymbol{\lambda}^\gamma) = \left(\prod_{p<q}\big[\mathrm{cP}\big(-2t\lambda_p^\gamma\lambda_q^\gamma\big)\big]_{p,q}\right)U_1(-t(\boldsymbol{\lambda}^\gamma)^2), \tag{3.59}$$

where $[\mathrm{cP}]_{p,q}$ denotes the application of cP over qubits (p,q). Note that we assumed **all-to-all qubit connectivity**[26]. If we considered only a linear qubit connectivity, some extra SWAP gates would be required so as to account for all the couples $\{(p,q)\}_{p<q}$ in order to implement the cP gates. Overall we conclude that the time-evolution operator of the full chemistry Hamiltonian $\hat{H}$ may be decomposed term-by-term via a Trotter decomposition (3.58):

$$e^{-i\Delta t\hat{H}} \simeq \prod_{\gamma=0}^{N_\gamma} e^{-i\Delta t\hat{H}_{\kappa_\gamma}} + \mathcal{O}(\Delta t^2) \overset{\text{qubit}}{\mapsto} \prod_{\gamma=1}^{N_\gamma}\Big(W_{\kappa_\gamma}^\dagger U_2(\Delta t\boldsymbol{\lambda}^\gamma)W_{\kappa_\gamma}\Big)W_{\kappa_0}^\dagger U_1(\Delta t\boldsymbol{\xi})W_{\kappa_0} + \mathcal{O}(\Delta t^2). \tag{3.60}$$

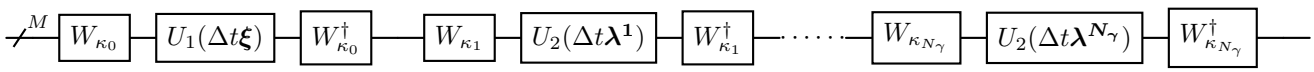


Figure 3.7: Quantum circuit for the time evolution of the chemistry Hamiltonian $\exp\big(-i\Delta t\hat{H}\big)$.

4 | Quantum Subspace Methods

In this chapter, we aim to present the general ideas behind subspace methods for finding approximate values of the energies of the system. This will be done by approximating the eigenvalues of the electronic Hamiltonian via the resolution of the Schrödinger equation in smaller subspaces of the Hilbert space, relying on the projection of quantum states in the Krylov basis. Moreover, we will observe the power and accuracy of these methods on small benchmark molecules and rely on molecular fragmentation techniques whenever necessary. In this chapter, we consider a fermionic framework by defining the spin Hamiltonian

$$\hat{H} = \sum_{\substack{p,q \\ \sigma}} h_{pq} a^\dagger_{p\sigma} a_{q\sigma} + \frac{1}{2} \sum_{\substack{p,q,r,s \\ \sigma,\tau}} h_{pqrs} a^\dagger_{p\sigma} a^\dagger_{q\tau} a_{r\tau} a_{s\sigma}, \tag{4.1}$$

such that $\sigma, \tau \in \{\uparrow, \downarrow\}$ and setting the following notation for qubit indices:

$$p\sigma = \begin{cases} p, & \sigma = \uparrow \\ p + M, & \sigma = \downarrow \end{cases} . \tag{4.2}$$

4.1 Krylov Subspace Method Construction

The following introduction is based on the review of Mario Motta et al.[13] on the general description of subspace methods for electronic structure simulations on quantum computers. Quantum subspace methods construct a variational subspace spanned by a set of states $\{|v_\alpha\rangle\}_{\alpha=0}^{n-1}$ where eigenstates of the Hamiltonian are approximated by a linear combination of basis states, $|\tilde{\Psi}^{(n)}_\mu\rangle = \sum_\alpha C_{\alpha\mu} |v_\alpha\rangle$. Then in order to obtain the expansion coefficients $\{C_{\alpha\mu}\}_\alpha$, we solve the generalized eigenvalue problem $\tilde{H}C = \tilde{E}^{(n)}SC$ on a classical computer where the matrix elements of the matrices S and $\tilde{H}$ given by $S_{\alpha\beta} = \langle v_\alpha | v_\beta \rangle$ and $\tilde{H}_{\alpha\beta} = \langle v_\alpha | \hat{H} | v_\beta \rangle$ are computed on a quantum computer. Upon solving this generalized eigenvalue problem, we obtain some approximate eigenpairs $\{\tilde{E}^{(n)}_\mu, |\tilde{\Psi}^{(n)}_\mu\rangle\}_\mu$ for the energies of the system and their corresponding eigenstates. Building on this general construction, one can consider the instance where the set of states is built by powers of a Hamiltonian $\hat{H}$ applied to some trial state $|v_0\rangle$ so that $|v_\alpha\rangle = \hat{H}^\alpha |v_0\rangle$, $\alpha \in \{0, \ldots, n-1\}$. Then considering the subspace spanned by the powers of $\hat{H}$ applied to v_0, $K_n\big(\hat{H}, |v_0\rangle\big) := \text{span}\big\{ |v_0\rangle, \hat{H}|v_0\rangle, \cdots, \hat{H}^{n-1}|v_0\rangle \big\}$, one can construct the overlap and Hamiltonian matrices S and $\tilde{H}$ for which their matrix elements are given by $S_{\alpha\beta} = \langle v_0 | \hat{H}^{\alpha+\beta} | v_0 \rangle$ and $\tilde{H}_{\alpha\beta} = \langle v_0 | \hat{H}^{\alpha+\beta+1} | v_0 \rangle$. The idea is then to solve, as before, the generalized eigenvalue problem to obtain the approximate eigenpairs of the system.

• An intuitive justification for Krylov subspace methods is as follows[13]: consider a Hamiltonian $\hat{H}$ evolving in a Hilbert space $\mathcal{H}$ with a non-degenerate ground-state $|\psi_0\rangle$ such that $\hat{H}$ solves the usual Schrödinger equation $\hat{H}|\psi_\mu\rangle = E_\mu |\psi_\mu\rangle$ for $\mu \in \{0, \cdots, \dim(\mathcal{H}) - 1\}$ where $\{E_\mu\}_\mu$ are the energies (i.e. eigenvalues) and $\{|\psi_\mu\rangle\}_\mu$ are the corresponding eigenstates of the system. Then, it is known that any state $|v_0\rangle$ non-orthogonal to $|\psi_0\rangle$ is mapped onto the ground state by an imaginary-time evolution for a sufficiently long time $\tau > 0$,

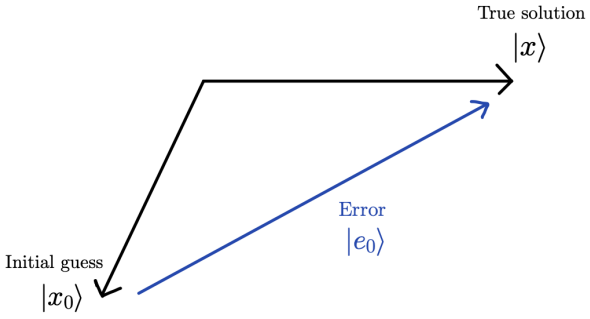
$$\lim_{\tau\to\infty} |\Phi_\tau\rangle := \lim_{\tau\to\infty} \frac{e^{-\tau\hat{H}}|v_0\rangle}{\|e^{-\tau\hat{H}}|v_0\rangle\|} = |\psi_0\rangle . \tag{4.3}$$

Thus, since $e^{-\tau\hat{H}}$ can be approximated via a truncated Taylor series, $e^{-\tau\hat{H}} \simeq \sum_{\alpha=0}^{n-1} \frac{(-\tau)^\alpha}{\alpha!}\hat{H}^\alpha$, the state $|\Phi_\tau\rangle$ can be approximated by a linear combination of vectors $\hat{H}^\alpha |v_0\rangle$, i.e. by vectors in the Hamiltonian Krylov space $K_n\big(\hat{H}, |v_0\rangle\big)$. The success and usefulness of these subspace methods depend on (i) the nature of the basis vectors $\{|v_\beta\rangle\}_\beta$, (ii) the dimension n of the subspace, and (iii) the cost and associated error of computing the matrices H, S.

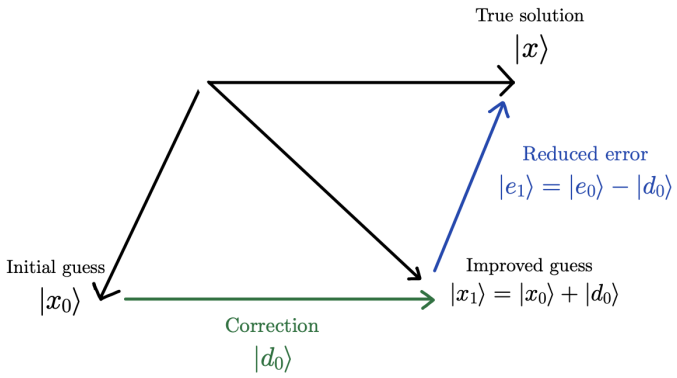
• Let us present, for completeness, a geometrical justification to why we should consider a Krylov subspace to approximate matrix eigenvalues via iterative refinement of eigenvector approximations. This will be done by considering the steepest descent of our space, and we will argue that after an initial guess for the ground state, the space of successive corrections to the initial guess that yields the fastest convergence is a Krylov subspace. First, suppose we have a matrix $\mathcal{M}$ which is, for simplicity, symmetric and positive definite. We do not impose any assumptions on the sparsity of the matrix nor that $\mathcal{M}$ is Hermitian (which it will have to be in the case where $\mathcal{M}$ is a Hamiltonian). We will consider problems of the form

$$\mathcal{M}|\mathbf{x}\rangle = |\mathbf{b}\rangle\,, \tag{4.4}$$

where notably this form encapsulates eigenvalue problems with $|\mathbf{b}\rangle = c\,|\mathbf{x}\rangle$ for some constant c. Consider a vector $|\mathbf{x_0}\rangle$ to be the initial guess for an approximation of the solution of Eq. (4.4). The **error** corresponding to the initial guess with respect to the true solution is $|\mathbf{e_0}\rangle := |\mathbf{x}\rangle - |\mathbf{x_0}\rangle$ with a corresponding **residual** given by $|R_0\rangle := \mathcal{M}|\mathbf{e_0}\rangle = |\mathbf{b}\rangle - \mathcal{M}|\mathbf{x_0}\rangle$. The situation can be sketched as follows:



Let us then look for a next step by considering a correction to the initial guess of the form $|\mathbf{x_1}\rangle = |\mathbf{x_0}\rangle + |\mathbf{d_0}\rangle$, for some yet to be determined vector $|\mathbf{d_0}\rangle$, in the hope of improving the approximation. The corresponding error is $|\mathbf{e_1}\rangle = |\mathbf{x}\rangle - |\mathbf{x_1}\rangle = |\mathbf{e_0}\rangle - |\mathbf{d_0}\rangle$, and the situation becomes:



Now, we would like to understand how the error behaves against the matrix $\mathcal{M}$. For instance, we may consider the energy norm[27] of the error:

$$\||\mathbf{e_1}\rangle\|_{\mathcal{M}}^2 = \langle\mathbf{e_1}|\,\mathcal{M}\,|\mathbf{e_1}\rangle = \langle\mathbf{e_0}|\,\mathcal{M}\,|\mathbf{e_0}\rangle - 2\,\langle\mathbf{e_0}|\,\mathcal{M}\,|\mathbf{d_0}\rangle + \langle\mathbf{d_0}|\,\mathcal{M}\,|\mathbf{d_0}\rangle = \langle\mathbf{e_0}|\,\mathcal{M}\,|\mathbf{e_0}\rangle - 2\,\langle R_0|\mathbf{d_0}\rangle + \langle\mathbf{d_0}|\,\mathcal{M}\,|\mathbf{d_0}\rangle\,, \tag{4.5}$$

where we have used the fact that $\mathcal{M}$ is symmetric. We now want to determine the form of the vector $|\mathbf{d_0}\rangle$, thus we can view Eq. (4.5) as a function $f(|\mathbf{d_0}\rangle)$ of $|\mathbf{d_0}\rangle$ and study its variations. Since f represents the error $|\mathbf{x}\rangle$, we want to minimise the function f. Using the symmetry of $\mathcal{M}$, the gradient yields $\nabla f(|\mathbf{d_0}\rangle) = 2(\mathcal{M}\,|\mathbf{d_0}\rangle - |R_0\rangle)$. Recall that the gradient points in the direction of steepest ascent, meaning that the opposite direction tells us where the function decreases the most. Considering the steepest descent and setting the gradient to zero, we see that the function f decreases the most in the direction of the residual $|R_0\rangle$. As a result, a good choice for the initial guess is the addition of a vector $|\mathbf{d_0}\rangle = \alpha_0\,|R_0\rangle$ for some constant α_0. In the next step, we can similarly choose a vector $|\mathbf{d_1}\rangle$ for the current approximation and use the same argument as above by choosing $|\mathbf{d_1}\rangle = \alpha_1\,|R_1\rangle$. By iterating this same argument, the approximation at the kth iteration, for $k \geq 1$, yields:

$$|\mathbf{x_k}\rangle = |\mathbf{x_0}\rangle + \sum_{j=0}^{k-1} \alpha_j\,|R_j\rangle\,. \tag{4.6}$$

Equivalently, this means that $|\mathbf{x_k}\rangle \in |\mathbf{x_0}\rangle + \text{span}\{|R_0\rangle\,, \ldots, |R_{k-1}\rangle\}$. Moreover, remark the following:

$$|R_k\rangle = |\mathbf{b}\rangle - \mathcal{M}\,|\mathbf{x_k}\rangle = |\mathbf{b}\rangle - \mathcal{M}\big(\,|\mathbf{x_{k-1}}\rangle + \alpha_{k-1}\,|R_{k-1}\rangle\,\big) = |R_{k-1}\rangle - \alpha_{k-1}\mathcal{M}\,|R_{k-1}\rangle = \big(\mathbb{I} - \alpha_{k-1}\mathcal{M}\big)\,|R_{k-1}\rangle\,. \tag{4.7}$$

Iterating this process, we conclude:

$$|R_k\rangle = \left[\prod_{j=0}^{k-1}\big(\mathbb{I} - \alpha_j\mathcal{M}\big)\right]|R_0\rangle = \left[\sum_{j=0}^{k}(-1)^j\sigma_j(\alpha_0, \alpha_1, \ldots, \alpha_{k-1})\mathcal{M}^j\right]|R_0\rangle\,, \tag{4.8}$$

where the coefficients $\{\sigma_j\}_j$ are *elementary symmetric polynomials* defined by:

$$\sigma_j(\alpha_0, \alpha_1, \ldots, \alpha_{k-1}) = \sum_{0 \leq i_1 < i_2 < \ldots < i_j \leq k-1} \alpha_{i_1}\alpha_{i_2}\ldots\alpha_{i_j}\,. \tag{4.9}$$

As a result, we conclude that

$$\text{span}\big\{\,|R_0\rangle\,, |R_1\rangle\,, \ldots, |R_k\rangle\,\big\} \equiv \text{span}\big\{\,|R_0\rangle\,, \mathcal{M}\,|R_0\rangle\,, \ldots, \mathcal{M}^k\,|R_0\rangle\,\big\}. \tag{4.10}$$

In other words, the space that most efficiently approximates the true solution $|\mathbf{x}\rangle$ is exactly given by the *Krylov subspace*. This subspace corresponds to the space spanned by the vectors of successive directions of steepest descent.

4.2 Krylov Subspace Method on a QPU & Performance

As discussed in the previous section, Krylov subspace methods can become handy for the case of diagonalising a Hamiltonian in order to obtain its eigenvalues and eigenvectors. However, as we will shortly see, the benefits of these methods depend on the problem at hand. An efficient way to observe a potential advantage of Krylov methods is the time to accomplish the diagonalisation of a matrix with respect to its dimension. We first plot, in Figure (4.1), the time taken for a classical computer to diagonalise a matrix, where its dimension ranges on $\{0, 1, \ldots, 200\}$. We observe that the required time grows significantly as the random matrices get larger. We now provide a second simulation, in Figure (4.2), where now we compare the performance of brute force diagonalisation against a Krylov method. Note that some occasional sharp *spikes* appear, possibly due to the random nature of the matrices or perhaps the implementation on the classical processor used. For an improved readability, we smooth the curves with a Gaussian filter which replaces each data with a weighted average of some neighbour points, for some Gaussian distributed weights dependent on a variance parameter σ.

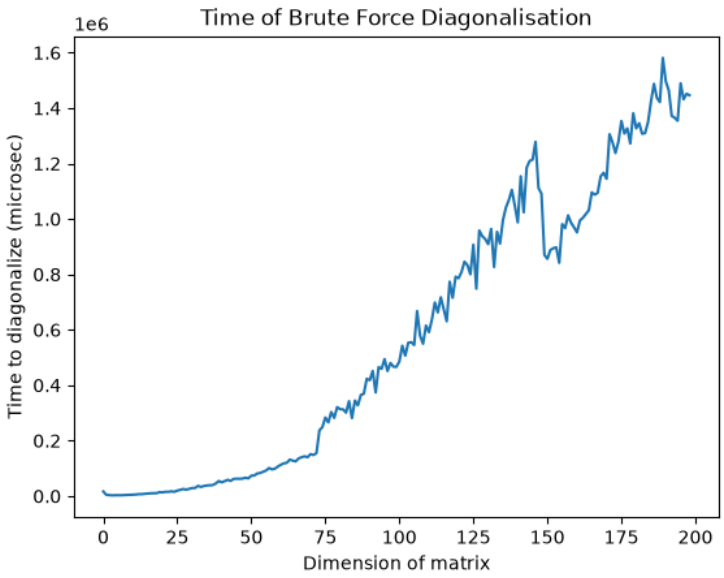


Figure 4.1: Time (in microsecond) taken by a classical computer to perform brute force diagonalisation on a random matrix, where the dimensions are in the range $\{0, 1, \ldots, 200\}$.

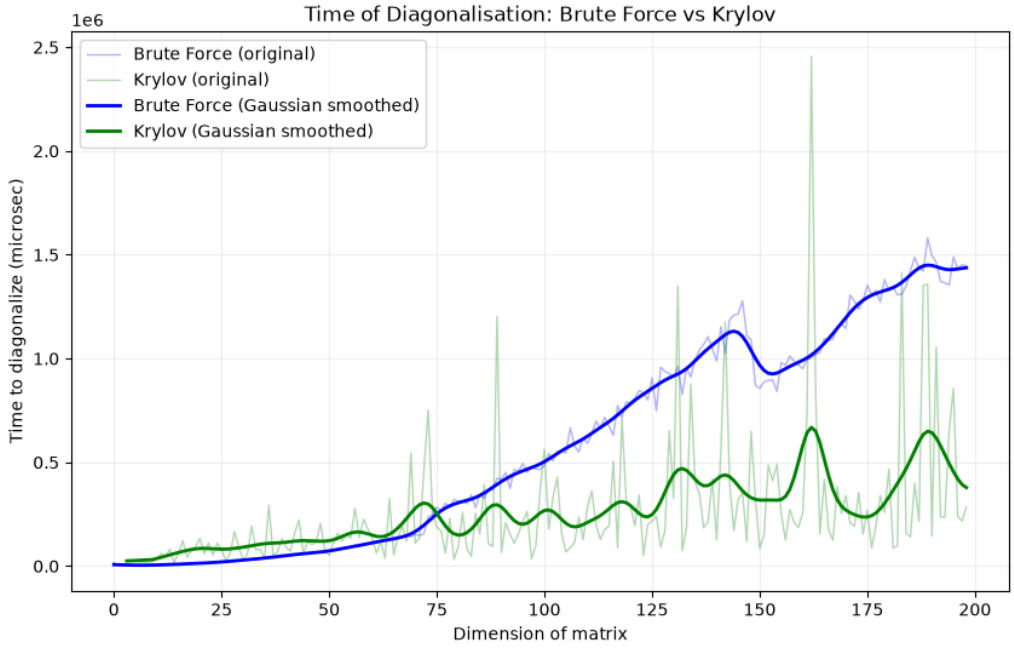


Figure 4.2: Comparison of the time (in microsecond) taken by a classical computer to perform brute force diagonalisation versus Krylov diagonalisation on a random matrix, for dimensions in the range $\{0, 1, \ldots, 200\}$.

We remark that a brute force diagonalisation remains preferable over a Krylov method for matrices no larger than 75 in dimension. Beyond this point, it is highly likely that a Krylov subspace method is preferable for very large matrices and within a required tolerance of error. However, for such very large matrices, the most time-consuming task is the generation of the so-called Krylov subspace, and as a result this is the critical point where a quantum computer might be desirable.

Now, we would like to study the dynamics of a quantum state, for which the operator of interest is the real-time evolution $U = e^{-itH}$ given a Hamiltonian H. We saw that the *power Krylov subspace* $K_n(H, |\psi_0\rangle)$ required

the evaluation of multiples powers of the Hamiltonian, which is in general a costly operation. Moreover, the Hamiltonian H is not necessarily unitary and thus it cannot be generated directly on a quantum computer. In place of this space, we can instead consider the *unitary Krylov subspace* $K_n^U(H, |\psi_0\rangle)$ which replaces the usual Hamiltonian with the time-evolution operator. The advantage is that instead of consider powers of an operator, we are only multiplying the Hamiltonian by a scalar in the exponent of the time-evolution. This new Krylov space yields:

$$K_n^U(H, |\psi_0\rangle) := \text{span}\{ |\psi_0\rangle, e^{-i\tau H} |\psi_0\rangle, e^{-2i\tau H} |\psi_0\rangle, \ldots, e^{-(n-1)i\tau H} |\psi_0\rangle \}, \tag{4.11}$$

for some initial state $|\psi_0\rangle$ and time-step τ. Denote by $|\psi_n\rangle$ the state of the subspace being time-evolved for a total time of $(n\tau)$. In this operation, we thus project H into the basis of the unitary Krylov subspace and we denote by $\tilde{H}$ the projected Hamiltonian. Now, the matrix elements of $\tilde{H}$ are given by the expectation values $\tilde{H}_{mn} = \langle \Psi_m | H | \psi_n \rangle$ which can be evaluated by the so-called *Hadamard test*. The aim of this procedure is to be able to calculate each matrix element allowing to obtain an overlap between quantum states. Given that $\dim(K^U) = r$, the Hamiltonian projected into the subspace will have dimension $r \times r$. With moderate values of r, where usually $r \ll 100$ is sufficient for good convergence results, we can then diagonalise $\tilde{H}$ classically. However, since the Krylov space vectors are not necessarily orthogonal, we must measure their overlap by constructing a Gram matrix S whose elements are $S_{mn} = \langle \psi_m | \psi_n \rangle$. The corresponding eigenstate problem is then given by $\tilde{H}\mathbf{c} = ES\mathbf{c}$ for some eigenvalues E to be determined.

4.3 Sample-Based Quantum Diagonalisation (SQD)

We now introduce a second subspace method to solve the electronic structure problem using the *Sample-based Quantum Diagonalization (SQD)* procedure, which is an iterative version of a general quantum subspace method. SQD bases its idea on the fact that sampling a quantum state Ψ in the computational basis yields measurement outcomes in the form of bitstrings $\mathbf{x} \in \{0,1\}^{2N_{\text{MO}}}$ (i.e. Slater determinants), where N_{MO} is the number of molecular orbitals[28]. The method iterates this idea by using a quantum circuit $|\Phi_{\text{qc}}\rangle$ to sample a set of computational basis states $\chi = \{\mathbf{x}_1, \ldots, \mathbf{x}_d\}$ (d is the dimension of a subspace) from the probability distribution $\mathbb{P}(\mathbf{x}) = |\langle \mathbf{x} | \Phi_{\text{qc}} \rangle|^2$ and a classical computer to solve the Schrödinger equation in the subspace spanned by such computational basis states[29]. Of course, on a quantum computer, we sample bitstrings in the presence of noise and so the set of measurement outcomes $\tilde{\chi} = \{\mathbf{x} | \mathbf{x} \sim \tilde{\mathbb{P}}_\Psi(\mathbf{x})\}$ is in fact dictated by the probability density $\tilde{\mathbb{P}}_\Psi(\mathbf{x}) = \langle \mathbf{x} | \tilde{\rho} | \mathbf{x} \rangle$ of sampling a specific bitstring $\mathbf{x}$ in the presence of noise, where $\tilde{\rho}$ is the density matrix representing the noisy wavefunction. For instance, Javier Robledo-Moreno et al.[30] use a depolarizing noise $\tilde{\mathbb{P}}_\Psi(\mathbf{x}) = \alpha \mathbb{P}_\Psi(\mathbf{x}) + (1-\alpha) 2^{-N_{\text{MO}}}$ for their simulations, with a parameter $\alpha \in [0,1]$ controlling the amount of noiseless quantum signal. Now, to handle the cost of the classical post-processing steps, we subsample the measurement samples $\tilde{\chi}$ into K batches of configurations $\{\mathcal{S}^{(b)}\}_{b=1}^K$ following the same probability density $\tilde{\mathbb{P}}_\Psi$ (i.e. the empirical frequency of each $\mathbf{x}$ in $\tilde{\chi}$) of the noisy samples with a fixed batch size d. The many-body Hamiltonian is then projected into each subspace using a projector $\hat{P}$:

$$\hat{H}_{\mathcal{S}^{(b)}} = \hat{P}_{\mathcal{S}^{(b)}} \hat{H} \hat{P}_{\mathcal{S}^{(b)}}, \qquad \text{where } \hat{P}_{\mathcal{S}^{(b)}} = \sum_{\mathbf{x} \in \mathcal{S}^{(b)}} |\mathbf{x}\rangle \langle \mathbf{x}|, \tag{4.12}$$

such that we then diagonalize the projected Hamiltonian by solving the following eigenvalue equation with a spin-constrained solver:

$$\left(\hat{H}_{\mathcal{S}^{(b)}} + \lambda \left[\hat{S}^2_{\mathcal{S}^{(b)}} - s(s+1) \right]^2 \right) |\psi^{(b)}\rangle = E^{(b)} |\psi^{(b)}\rangle, \tag{4.13}$$

for a Lagrange multiplier λ set to the default value $\lambda = 0.2$ and where the total spin operator $\hat{S}^2$ (having $\{s(s+1)\}_s$ as its eigenvalues) satisfies

$$\hat{S}^2_{\mathcal{S}^{(b)}} = \hat{P}_{\mathcal{S}^{(b)}} \hat{S}^2 \hat{P}_{\mathcal{S}^{(b)}}. \tag{4.14}$$

We have introduced a constraint on the spins, through the implementation of a quadratic penalty method (4.13), since two scenarios can happen on a noisy device: (i) particle number and total spin-z may not be conserved (i.e. the sampled computational basis states may correspond to Slater determinants with an incorrect number of spin-up and/or spin-down electrons) and (ii) the subspace spanned by the sampled computational basis states may not allow to produce eigenstates of $\hat{S}^2$ the total spin operator[29]. In order to account for these scenarios, the authors in [30] introduce an iterative self-consistent configuration recovery routine accounting for the following steps: the routine finds a configuration $\mathbf{x}$ in $\tilde{\chi}$ having the wrong particle number $N_{\mathbf{x}} \neq N$ because of error accumulation in the circuit. If $N_{\mathbf{x}} > N$ (resp. $N_{\mathbf{x}} < N$), $|N_{\mathbf{x}} - N|$ bits are further sampled to be flipped from the set of occupied (resp. empty) spin orbitals according to a selection probability density proportional to $|x_{p\sigma} - n_{p\sigma}|$, the distance between the bit value $x_{p\sigma}$ and the current average orbital occupancy obtained from the previous round for each spin-orbital $p\sigma$ (see [30, pp. S7-S9] for the details). Upon iteration of this procedure, we obtain a new set of recovered configurations χ_R. Finally, we can compute the ground-state wavefunctions $\left\{ |\psi_0^{(b)}\rangle \right\}_{b=1}^{K}$ and energies $\left\{E_0^{(b)}\right\}_{b=1}^{K}$ in every batch and choose the lowest energy, $\min_b E_0^{(b)}$, as the best approximation to the ground-state energy of the full system. Note that, in the configuration recovery routine, we update the occupation number distribution

$$n_{p\sigma} = \frac{1}{K} \sum_{b=1}^{K} \langle \psi_0^{(b)} | \, a_{p\sigma}^{\dagger} a_{p\sigma} \, | \psi_0^{(b)} \rangle \,, \tag{4.15}$$

for the number operator $\hat{n}_{p\sigma} = a_{p\sigma}^{\dagger} a_{p\sigma}$, by using the wavefunctions $\left\{ |\psi_0^{(b)}\rangle \right\}_{b=1}^{K}$ as inputs in the next rounds of the iteration procedure. We provide the full SQD algorithm below.

Algorithm 1: Sample-based Quantum Diagonalization (SQD) - [28]

Data: Bitstrings $\mathbf{x}$, Hamiltonian $\hat{H}$, number of batches K, samples per batch d, number of iterations N_{iter}, spin and particle number targets.
Result: Approximate eigenvalue $E_0^{(b)}$ and eigenstate $|\psi_0^{(b)}\rangle$.
for iteration $\in \{1, \ldots, N_{\text{iter}}\}$ **do**
  if iteration $= 1$ **then**
    Discard bitstrings with incorrect particle number.
  else
    Modify bitstrings via bit-flip correction using previous $n_{p\sigma}$ to match target symmetry.
  end
  Group valid bitstrings into batches $\{\mathcal{S}^{(b)}\}_{b=1}^{K}$ of d samples.
  for each batch $\mathcal{S}^{(b)}$ **do**
    Project Hamiltonian by using $\hat{H}_{\mathcal{S}^{(b)}} = \hat{P}_{\mathcal{S}^{(b)}} \hat{H} \hat{P}_{\mathcal{S}^{(b)}}$.
    Solve eigenvalue equation with a spin penalty to obtain eigenpairs $\{E^{(b)}, |\psi^{(b)}\rangle\}_{b=1}^{K}$:

$$\left(\hat{H}_{\mathcal{S}^{(b)}} + \lambda \left[\hat{S}^2_{\mathcal{S}^{(b)}} - s(s+1) \right]^2 \right) |\psi^{(b)}\rangle = E^{(b)} \, |\psi^{(b)}\rangle \,.$$

    Compute and update the spin-resolved orbital occupancy:

$$n_{p\sigma} = \frac{1}{K} \sum_{b=1}^{K} \langle \psi_0^{(b)} | \, a_{p\sigma}^{\dagger} a_{p\sigma} \, | \psi_0^{(b)} \rangle \,.$$

  end
  Choose the lowest-energy eigenstate, $\min_b E_0^{(b)}$, from all the batches.
end
return $E_0^{(b)}$, $|\psi_0^{(b)}\rangle$.

5 | Machine Learning for Many-Body Quantum Systems

The wavefunction Ψ is a fundamental object in Quantum Physics and possibly the hardest to grasp in a classical world. It is a mathematical quantity that contains all the information on a quantum state, whether it is a single particle or a complex molecule. In principle, an exponential amount of information is needed to fully encode a generic many-body quantum state. However, Nature often proves herself benevolent, and a wavefunction representing a physical many-body system can be typically characterized by an amount of information much smaller than the maximum capacity of the corresponding Hilbert space[31]. Numerous numerical approaches have been designed to rely on the wavefunction for a sampling of relevant configurations in a Hilbert space, or also for a *compression* of the quantum state to counter the exponential scaling of the configuration space. Several stochastic methods have been used to efficiently sample physically relevant configurations, such as Quantum Monte Carlo (QMC) methods[32,33,34]. On the other hand, methods aiming to compress the wavefunction have also been widely studied, especially methods revolving around matrix product states (MPS)[35,36,37], or generalising this notion to more general tensor network structures[38,39] such as projected entangled pair states (PEPS). These methods do not come without drawbacks, especially with Quantum Monte Carlo methods which suffer from the so-called *sign problem*. On the other hand, for tensor networks, MPS is considered to be the best representation of one-dimensional systems, however they do not generalise well to high dimensions. PEPS are considered for two- (three-) dimensional systems, but there is still a relatively large cost in the simulation of very high dimensional quantum states with tensor networks. In this chapter, we present a sign-problem-free method based on variational Monte Carlo that compresses the wavefunction, where the aim is to solve the many-body quantum problem with neural networks. Such quantum states, with a given neural network architecture, are named *Neural Quantum States* (NQS)[31]. We first present the sign-problem in general quantum Monte Carlo. We then provide the foundations of variational Monte Carlo methods (static[40] / time-dependent[41,42]) and neural networks, before presenting the general structure of neural quantum states. We supplement the theory with several numerical implementations of physical systems. Most of the theory on classical Monte Carlo methods and convergence diagnostics are from lecture notes in *Stochastic Simulation* by Prof. Fabio Nobile (EPFL)[43].

5.1 The Sign-Problem in Quantum Monte Carlo

The issue for quantum Monte Carlo estimation is the following: in classical Monte Carlo methods, if one wants to evaluate the expectation value of a quantity A, $\langle A\rangle = Z^{-1}\sum_{\omega\in\Omega} A(\omega)W(\omega)$ where $Z = \sum_{\omega\in\Omega} W(\omega)$ for some weights W and over a high-dimensional space Ω of configurations $\{\omega\}$, one can consider the Monte Carlo average $\langle A\rangle \simeq M^{-1}\sum_{j=1}^{M} A(\omega_j)$ with M i.i.d. sample configurations $\{\omega_j\}_j$ generated from the probability distribution $p(\omega) = W(\omega)/Z$ as an unbiased estimator for the true mean. The deviation of this estimator from the true mean (i.e. the statistical error) is bounded by a quantity proportional to $1/\sqrt{M}$, a relatively slow rate of convergence leading to a costly sampling of the space Ω for an accurate estimation. In classical statistical physics, the distribution is chosen as the Gibbs-Boltzmann distribution $p(\omega) = Z^{-1}\exp(-\beta E(\omega))$ where $\beta = 1/(k_B T)$ is the inverse temperature and $E(\omega)$ is the energy of the configuration ω. In a quantum system with a Hamiltonian

operator H, the average of an observable A has the operator expression $\langle A\rangle = Z^{-1}\mathrm{Tr}[A\exp(-\beta H)]$, where $Z = \mathrm{Tr}[\exp(-\beta H)]$. Monte Carlo methods can again be applied to reduce the exponential scaling of the problem, but only after mapping the quantum model to a classical one[44]. This means that we can always find some weights W such that we can write $\langle A\rangle = Z^{-1}\mathrm{Tr}[A\exp(-\beta H)] = Z^{-1}\sum_{\omega\in\Omega} A(\omega)W(\omega)$. If all the weights W are positive, then we can apply the standard Monte Carlo method to estimate the average of an observable A, as it is the case for frustrated quantum magnets and bosonic systems. In fermionic systems however, negative weights $W < 0$ can arise due to the Pauli exclusion principle, since exchanging two fermions in the wavefunction Ψ produces a negative sign, i.e. $\Psi(\mathbf{x}_1,\ldots,\mathbf{x}_i,\ldots,\mathbf{x}_j,\ldots,\mathbf{x}_N) = -\Psi(\mathbf{x}_1,\ldots,\mathbf{x}_j,\ldots,\mathbf{x}_i,\ldots,\mathbf{x}_N)$ for an N-fermionic system. To illustrate this fact, consider a stochastic series expansion (SSE) of the partition function Z (i.e. a Taylor expansion of Z):

$$Z := \mathrm{Tr}[\exp(-\beta H)] = \sum_{n=0}^{\infty}\frac{(-\beta)^n}{n!}\mathrm{Tr}(H^n) = \sum_{n=0}^{\infty}\sum_{i_1,\ldots,i_n}\frac{(-\beta)^n}{n!}\langle i_1|H|i_2\rangle\langle i_2|H|i_3\rangle\ldots\langle i_n|H|i_1\rangle \equiv \sum_{\omega\in\Omega} W(\omega), \tag{5.1}$$

where here, a configuration ω is a sequence $(i_1,\ldots,i_n)$ of n basis states such that the weight $W(\omega)$ is the corresponding product of matrix elements of H multiplied by the factor $(-\beta)^n/n!$. Now, from the expansion in Eq. (5.1), we see that the final sign of the weight will be determined by the number of negative matrix elements multiplied. For fermionic bipartite lattices (square or honeycomb lattices) with periodic boundary conditions, it was shown that this product was always of positive sign, however for non-bipartite lattices (such as triangle or kagome lattices) the number of negative signs is always odd and thus the SSE simulation will have the sign problem[45]. Now, one important observation is that if the sign problem occurs it does not mean that we cannot perform a Monte Carlo simulation, however one needs to rescale the averages to be sign-problem-free:

$$\langle A\rangle = \frac{\sum_{\omega\in\Omega} A(\omega)W(\omega)}{\sum_{\omega\in\Omega} W(\omega)} = \frac{\sum_{\omega\in\Omega} A(\omega)s(\omega)|W(\omega)|/\sum_{\omega\in\Omega}|W(\omega)|}{\sum_{\omega\in\Omega} s(\omega)|W(\omega)|/\sum_{\omega\in\Omega}|W(\omega)|} \equiv \frac{\langle As\rangle_{P_W}}{\langle s\rangle_{P_W}}, \tag{5.2}$$

where $s(\omega) := W(\omega)/|W(\omega)|$ and $\langle B\rangle_{P_W} := \sum_{\omega\in\Omega} B(\omega)P_W(\omega)$ is an average of the observable B with respect to a *new* probability distribution given by $P_W(\omega) := |W(\omega)|/\sum_{\omega\in\Omega}|W(\omega)|$. To estimate $\langle A\rangle$, we can now perform usual Monte Carlo with respect to the (positive) distribution P_W. However, one major caveat to this story arises when looking at the order of magnitude of the average sign $\langle s\rangle_{P_W}$. First, notice that the average sign is given as:

$$\langle s\rangle_{P_W} = \sum_{\omega\in\Omega} s(\omega)P_W(\omega) = \sum_{\omega\in\Omega}\frac{W(\omega)}{|W(\omega)|}\frac{|W(\omega)|}{\sum_{\omega'\in\Omega}|W(\omega')|} =: \frac{Z}{Z'}, \tag{5.3}$$

where $Z = \sum_{\omega\in\Omega} W(\omega)$ and $Z' = \sum_{\omega\in\Omega}|W(\omega)|$. Now for a general macroscopic system with a Helmholtz free energy F, we have that $Z/Z' = \exp(-\beta N\Delta F)$ where ΔF is a difference in free energy. Thus the relative standard error $\Delta s/\langle s\rangle$, which assesses the precision of an estimate in a standardized way, grows exponentially with an increasing particle number at inverse temperature β:

$$\frac{\Delta s}{\langle s\rangle} := \frac{\sigma(s)}{\langle s\rangle} = \frac{\sqrt{(\langle s^2\rangle - \langle s\rangle^2)/M}}{\langle s\rangle} = \frac{\sqrt{1-\langle s\rangle^2}}{\langle s\rangle\sqrt{M}} \propto \frac{e^{\beta N\Delta F}}{\sqrt{M}} + \mathcal{O}(\langle s\rangle). \tag{5.4}$$

In most cases, ΔF is a quantity that barely changes in simulations, and so when there is a sign problem the QMC simulation usually takes an exponential time to have a reasonable estimation of the errorbars for physical observables[45]. In fact, the sign-problem is known to be an **NP**-hard problem and even **NP**-complete in some instances (i.e. **NP** + **NP**-hard)[44].

5.2 Variational Monte Carlo & Stochastic Sampling

Consider a system of N particles for which we consider many-body amplitudes $\psi_M(\boldsymbol{\sigma})$ over a discrete set of configurations $\{\boldsymbol{\sigma}\}$, for which a corresponding quantum state is set to be $|\Psi\rangle = \sum_{\boldsymbol{\sigma}}\psi_M(\boldsymbol{\sigma})|\boldsymbol{\sigma}\rangle$ with $\psi_M(\boldsymbol{\sigma}) =$

$\langle \boldsymbol{\sigma}|\Psi\rangle$. The standard formalism for the search of the ground state energy E_0 is the minimization of the energy expectation value over all configurations given by the Rayleigh quotient:

$$E_0 := \min_{\Psi} \langle \hat{H} \rangle_\Psi = \min_{\Psi} \frac{\langle \Psi | \, H_q \, | \Psi \rangle}{\langle \Psi | \Psi \rangle} \equiv \min_{\Psi} \frac{\sum_{\boldsymbol{\sigma},\boldsymbol{\sigma'}} \langle \Psi | \boldsymbol{\sigma} \rangle \langle \boldsymbol{\sigma} | \, H_q \, | \boldsymbol{\sigma'} \rangle \, \langle \boldsymbol{\sigma'} | \Psi \rangle}{\sum_{\boldsymbol{\sigma}} \langle \Psi | \boldsymbol{\sigma} \rangle \langle \boldsymbol{\sigma} | \Psi \rangle}, \tag{5.5}$$

where H_q corresponds to the fermionic Hamiltonian mapped to a qubit encoding such as the Jordan-Wigner transformation, the Bravyi-Kitaev encoding or the parity encoding (to name a few), and where we have used the completeness relation $\mathbb{I} = \sum_{\boldsymbol{\sigma}} |\boldsymbol{\sigma}\rangle \langle \boldsymbol{\sigma}|$. For instance, under the Jordan-Wigner transformation, H_q will be a linear combination of N-fold Pauli operators (i.e. elements of the Pauli group $\mathcal{P}_N$ on N qubits), $H_q = \sum_j \alpha_j \hat{P}_j$ with $\hat{P}_j \in \mathcal{P}_N$ for all $j \in \mathbb{N}$ and some coefficients $\{\alpha_j\}_j$. Now, this expectation value may be written in the form $\langle \hat{H} \rangle_\Psi = \sum_{\boldsymbol{\sigma}} \mathbb{P}(\boldsymbol{\sigma}) E_{\text{loc}}(\boldsymbol{\sigma})$ for some local energies E_{loc}. Noting that the Born distribution of a configuration $\boldsymbol{\sigma}$ is $\mathbb{P}(\boldsymbol{\sigma}) = |\psi_M(\boldsymbol{\sigma})|^2$, we may evaluate $\langle \hat{H} \rangle_\Psi$ using Monte Carlo sampling from the fact that the numerator in (5.5) may be re-written as follows:

$$\begin{aligned}
\langle \Psi | \, H_q \, | \Psi \rangle &= \left(\sum_{\boldsymbol{\sigma'}} \psi_M^*(\boldsymbol{\sigma'}) \langle \boldsymbol{\sigma'} | \right) H_q \left(\sum_{\boldsymbol{\sigma}} \psi_M(\boldsymbol{\sigma}) |\boldsymbol{\sigma}\rangle \right) \\
&= \sum_{\boldsymbol{\sigma},\boldsymbol{\sigma'}} \psi_M^*(\boldsymbol{\sigma'}) \psi_M(\boldsymbol{\sigma}) \langle \boldsymbol{\sigma'} | \, H_q \, | \boldsymbol{\sigma} \rangle \\
&= \sum_{\boldsymbol{\sigma}} |\psi_M(\boldsymbol{\sigma})|^2 \left(\sum_{\boldsymbol{\sigma'}} \frac{\psi_M(\boldsymbol{\sigma'})}{\psi_M(\boldsymbol{\sigma})} \langle \boldsymbol{\sigma} | \, H_q \, | \boldsymbol{\sigma'} \rangle \right) \equiv \sum_{\boldsymbol{\sigma}} |\psi_M(\boldsymbol{\sigma})|^2 E_{\text{loc}}(\boldsymbol{\sigma}).
\end{aligned}$$

Then, given that the normalization constant is $\langle \Psi | \Psi \rangle = \sum_{\boldsymbol{\sigma}} |\psi_M(\boldsymbol{\sigma})|^2$, the average energy is evaluated via Monte Carlo by considering

$$\langle \hat{H} \rangle_\Psi = \frac{\sum_{\boldsymbol{\sigma}} |\psi_M(\boldsymbol{\sigma})|^2 E_{\text{loc}}(\boldsymbol{\sigma})}{\sum_{\boldsymbol{\sigma}} |\psi_M(\boldsymbol{\sigma})|^2} = \mathbb{E}_{\boldsymbol{\sigma} \sim \mathbb{P}(\boldsymbol{\sigma})} \big[E_{\text{loc}}(\boldsymbol{\sigma}) \big], \tag{5.6}$$

where we have defined the local energies E_{loc} and the probability distribution $\mathbb{P}(\boldsymbol{\sigma})$ as:

$$E_{\text{loc}}(\boldsymbol{\sigma}) = \sum_{\boldsymbol{\sigma'}} \frac{\psi_M(\boldsymbol{\sigma'})}{\psi_M(\boldsymbol{\sigma})} \langle \boldsymbol{\sigma} | \, H_q \, | \boldsymbol{\sigma'} \rangle \qquad \text{and} \qquad \mathbb{P}(\boldsymbol{\sigma}) = \frac{|\psi_M(\boldsymbol{\sigma})|^2}{\sum_{\boldsymbol{\sigma}} |\psi_M(\boldsymbol{\sigma})|^2}. \tag{5.7}$$

Note that given some samples $\mathcal{M}$ drawn from the distribution $\mathbb{P}(\boldsymbol{\sigma})$, the average over the samples $\hat{E}_{\mathcal{M}} = \langle E_{\text{loc}}(\boldsymbol{\sigma}) \rangle_{\mathcal{M}}$ produces an unbiased estimator of the energy as a result of an ergodicity condition. The above expectation value is exact in the limit of infinite sample size $|\mathcal{M}| \to \infty$, where in addition to the computations one usually computes the Monte Carlo error $|\hat{E}_{\mathcal{M}} - \hat{E}_{\text{exact}}| \leq c_{1-\alpha/2} \sqrt{\text{Var}(E_{\text{loc}})} / \sqrt{|\mathcal{M}|}$. One can also compute an asymptotic $(1-\alpha)$ confidence interval $I_{\alpha,\mathcal{M}} = \big[\hat{E}_{\mathcal{M}} \pm c_{1-\alpha/2} \sqrt{\text{Var}(E_{\text{loc}})} / \sqrt{|\mathcal{M}|} \big]$. Here, c_α denotes the α-quantile of the Normal distribution satisfying $\Phi(c_\alpha) = \alpha$ and Φ the cumulative distribution function of a standard Normal random variable. Note also in practice that this error and the corresponding confidence interval are not practical to work with since the variance is usually unknown. However, it can be replaced by the sample variance estimator computed using the same samples $\mathcal{M}$ such that $\widehat{\text{Var}}_{\mathcal{M}} = (|\mathcal{M}| - 1)^{-1} \sum_{j=1}^{|\mathcal{M}|} (\boldsymbol{\sigma}_j - \hat{E}_{\mathcal{M}})^2$, where $\{\boldsymbol{\sigma}_j\}_j$ are the individual samples of $\mathcal{M}$. Moreover, in generic Quantum Chemistry problems, the cost of evaluating the local energies scales in the worst case as $\mathcal{O}(N^4)$ compared to the linear scaling in systems with nearest neighbour interactions[46]. The idea of Monte Carlo sampling is to select the most important configurations in order to estimate the energy on this smaller set of configurations. Now, to sample from the probability distribution $\mathbb{P}$ we often resolve to the use of the Markov Chain Monte Carlo (MCMC) algorithm, where in our case we consider the Metropolis Hastings (MH) algorithm[47]. The goal is to construct a Markov chain on the space of configurations that has $\mathbb{P}$ as a stationary distribution of the process, i.e. given the stochastic matrix Γ for the transition density of the chain, one has that $\mathbb{P}$ is an invariant distribution for Γ if $\mathbb{P}\Gamma = \mathbb{P}$. For the remainder of this section, we define the invariant distribution as π. Now, suppose we want to sample N configurations $\{\boldsymbol{\sigma}^{(0)}, \boldsymbol{\sigma}^{(1)}, \ldots, \boldsymbol{\sigma}^{(N-1)}\}$. It turns out that a simpler method to verify that π is invariant for Γ in the case of a

reversible Markov chain $\big($i.e. if $\{\boldsymbol{\Sigma}_n\}_{n\geq 0} \sim \text{Markov}(\pi, \Gamma)$ then, for all $N \geq 1$, $\{\boldsymbol{\Sigma}_{N-n}\}_{n\geq 0} \sim \text{Markov}(\pi, \Gamma)\big)$ is to verify a **detailed balance** condition, for all $(j,k) \in \{0,\dots,N-1\} \times \{0,\dots,N-1\}$, given by:

$$\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(j)}\big)\Gamma\big(\boldsymbol{\Sigma}_{n+1} = \boldsymbol{\sigma}^{(k)} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(j)}\big) = \mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big)\Gamma\big(\boldsymbol{\Sigma}_{n+1} = \boldsymbol{\sigma}^{(j)} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big). \tag{5.8}$$

As a result of this condition, we know that:

$$\frac{\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(j)}\big)}{\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big)} = \frac{\Gamma\big(\boldsymbol{\Sigma}_{n+1} = \boldsymbol{\sigma}^{(j)} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big)}{\Gamma\big(\boldsymbol{\Sigma}_{n+1} = \boldsymbol{\sigma}^{(k)} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(j)}\big)}. \tag{5.9}$$

The interesting point of the MH algorithm is that we know how to compute the ratio on the left of Eq. (5.9) since we got rid of the normalization constant being computationally intractable. However, we do not know the general form of the transition density matrix $\Gamma = \{\Gamma_{jk}\}_{(j,k)\in[N]\times[N]} \equiv \big\{\Gamma\big(\boldsymbol{\Sigma}_{n+1} = \boldsymbol{\sigma}^{(k)} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(j)}\big)\big\}_{(j,k)\in[N]\times[N]}$ for $[N] := \{0,\dots,N-1\}$. As a result, we are required to find an ansatz which in this context is chosen as the product of the probability to propose the configuration $\boldsymbol{\sigma}^{(k)}$ given $\boldsymbol{\sigma}^{(j)}$, by the probability to accept the configuration $\boldsymbol{\sigma}^{(k)}$ given $\boldsymbol{\sigma}^{(j)}$:

$$\Gamma_{jk} = \Gamma^{\text{prop}}_{jk}\Gamma^{\text{acc}}_{jk}. \tag{5.10}$$

This is an intuitive motivation since the algorithm that we are designing will have an acceptance/rejection step after the proposal of a new candidate state. Note that in general, the proposal density will not have $\mathbb{P}_\pi$ as invariant distribution and thus it must be corrected. As a result, under this ansatz, Eq. (5.9) may be re-written as follows:

$$\frac{\Gamma^{\text{acc}}_{kj}}{\Gamma^{\text{acc}}_{jk}} = \frac{\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(j)}\big)}{\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big)}\frac{\Gamma^{\text{prop}}_{jk}}{\Gamma^{\text{prop}}_{kj}}. \tag{5.11}$$

Finally, we need to find an acceptance probability that still respects the detailed balance condition. Although this choice is not unique, W.K. Hastings[47] proposed that it could take the form

$$\Gamma^{\text{acc}}_{kj} = \min\left\{1,\ \frac{\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(j)}\big)}{\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big)}\frac{\Gamma^{\text{prop}}_{jk}}{\Gamma^{\text{prop}}_{kj}}\right\} \text{ if } \Gamma^{\text{prop}}_{kj} \neq 0, \text{ and } \Gamma^{\text{acc}}_{kj} = 0 \text{ if } \Gamma^{\text{prop}}_{kj} = 0.$$

Finally, we have the following algorithm:

Algorithm 2: Metropolis Hastings Algorithm

Data: Initial distribution λ, proposal Γ^{prop}, target distribution $\mathbb{P}$.
Generate $\boldsymbol{\sigma}^{(0)} \sim \lambda$.
for $n = 0, 1, \dots$ **do**
  Generate a new candidate configuration $\tilde{\boldsymbol{\sigma}}^{(n+1)} \sim \Gamma^{\text{prop}}_{\boldsymbol{\sigma}^{(n)},\cdot}$
  Generate $U \sim \text{Unif}([0,1])$
  if $U \leq \Gamma^{\text{acc}}_{\boldsymbol{\sigma}^{(n)},\tilde{\boldsymbol{\sigma}}^{(n+1)}}$ **then**
    set $\boldsymbol{\sigma}^{(n+1)} = \tilde{\boldsymbol{\sigma}}^{(n+1)}$. `/* ` $\tilde{\boldsymbol{\sigma}}^{(n+1)}$ `accepted with probability` $\Gamma^{\text{acc}}_{\boldsymbol{\sigma}^{(n)},\tilde{\boldsymbol{\sigma}}^{(n+1)}}$ `*/`
  else
    set $\boldsymbol{\sigma}^{(n+1)} = \boldsymbol{\sigma}^{(n)}$. `/* ` $\tilde{\boldsymbol{\sigma}}^{(n+1)}$ `rejected with probability` $1 - \Gamma^{\text{acc}}_{\boldsymbol{\sigma}^{(n)},\tilde{\boldsymbol{\sigma}}^{(n+1)}}$ `*/`
  end
end

Note that if the proposal Γ^{prop} is symmetric, then the ratio in the acceptance probability reduces to a simpler form in terms of the invariant distribution: $\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(j)}\big)/\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big)$. In this case, we see that $\tilde{\boldsymbol{\sigma}}^{(n+1)}$ will get accepted whenever the probability of the new configuration is larger than the probability of the current one, i.e. $\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \tilde{\boldsymbol{\sigma}}^{(n+1)}\big) \geq \mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(n)}\big)$. In the case where $\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \tilde{\boldsymbol{\sigma}}^{(n+1)}\big) < \mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(n)}\big)$, the new configuration $\tilde{\boldsymbol{\sigma}}^{(n+1)}$ will be accepted only with a probability of $\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \tilde{\boldsymbol{\sigma}}^{(n+1)}\big)/\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(n)}\big)$. Hence the new configuration has a high probability of being rejected in the case where $\mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \tilde{\boldsymbol{\sigma}}^{(n+1)}\big) \ll \mathbb{P}_\pi\big(\boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(n)}\big)$. We now close the subject of MCMC by two concluding results on the transition density of the chain produced

by the MH algorithm, and also a short proof to show that it produces the correct chain.

Proposition 5.2.1. *Denote by P_{kj} the transition density of the chain produced by the MH algorithm. Then:*

$$P_{kj} = \Gamma^{\text{acc}}_{kj}\Gamma^{\text{prop}}_{kj} + \left(1 - \sum_j \Gamma^{\text{acc}}_{kj}\Gamma^{\text{prop}}_{kj}\right)\delta_{kj}. \tag{5.12}$$

Proof. First, consider $k \neq j$:

$$\begin{aligned} P_{kj} &= \mathbb{P}\big(\boldsymbol{\Sigma}_{n+1} = \boldsymbol{\sigma}^{(j)} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big) \\ &= \mathbb{P}\big(\tilde{\boldsymbol{\Sigma}}_{n+1} = \boldsymbol{\sigma}^{(j)}, \boldsymbol{\Sigma}_{n+1} = \tilde{\boldsymbol{\Sigma}}_{n+1} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big) \\ &= \mathbb{P}\big(\boldsymbol{\Sigma}_{n+1} = \tilde{\boldsymbol{\Sigma}}_{n+1} | \tilde{\boldsymbol{\Sigma}}_{n+1} = \boldsymbol{\sigma}^{(j)}, \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big)\mathbb{P}\big(\tilde{\boldsymbol{\Sigma}}_{n+1} = \boldsymbol{\sigma}^{(j)}, \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big) = \Gamma^{\text{acc}}_{kj}\Gamma^{\text{prop}}_{kj}. \end{aligned}$$

We now consider the case $k = j$:

$$\begin{aligned} P_{kk} &= \mathbb{P}\big(\boldsymbol{\Sigma}_{n+1} = \boldsymbol{\sigma}^{(k)} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big) \\ &= \mathbb{P}\big(\tilde{\boldsymbol{\Sigma}}_{n+1} = \boldsymbol{\sigma}^{(k)}, \boldsymbol{\Sigma}_{n+1} = \tilde{\boldsymbol{\Sigma}}_{n+1} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big) + \mathbb{P}\big(\boldsymbol{\Sigma}_{n+1} \neq \tilde{\boldsymbol{\Sigma}}_{n+1} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big) \\ &= \Gamma^{\text{acc}}_{kk}\Gamma^{\text{prop}}_{kk} + \sum_j \mathbb{P}\big(\tilde{\boldsymbol{\Sigma}}_{n+1} = \boldsymbol{\sigma}^{(j)}, \boldsymbol{\Sigma}_{n+1} \neq \tilde{\boldsymbol{\Sigma}}_{n+1} | \boldsymbol{\Sigma}_n = \boldsymbol{\sigma}^{(k)}\big) \\ &= \Gamma^{\text{acc}}_{kk}\Gamma^{\text{prop}}_{kk} + \sum_j \big(1 - \Gamma^{\text{acc}}_{kj}\big)\Gamma^{\text{prop}}_{kj} \\ &= \Gamma^{\text{acc}}_{kk}\Gamma^{\text{prop}}_{kk} + \left(1 - \sum_j \Gamma^{\text{acc}}_{kj}\Gamma^{\text{prop}}_{kj}\right). \end{aligned}$$

□

Note that $\sum_j \Gamma^{\text{acc}}_{kj}\Gamma^{\text{prop}}_{kj}$ is the overall probability of accepting a new state when being in the configuration $\boldsymbol{\sigma}^{(k)}$. If such acceptance probability is very close to zero, with high probability the chain will not move, hence the random configurations $\{\boldsymbol{\Sigma}_n\}_n$ will be highly correlated which is not a desirable scenario for constructing an ergodic estimator. Remark also that a very high acceptance probability might not be desirable either. We thus have two strategies: either the chain jumps only to neighbouring states with a high acceptance rate, or it jumps to far away states but with lower acceptance rate. It is not clear which strategy to adopt if we want a decorrelated chain (as a rule of thumb, we expect an average in acceptance rate of 0.2 for efficient mixing). Let us now end by showing that MH produces the correct chain, i.e. a chain that has invariant distribution π. As we will see, this holds since the transition matrix P is in detailed balance with π.

Proposition 5.2.2. *The transition matrix P is in detailed balance with π. As a result, the chain produced by the MH algorithm is reversible and has π as invariant distribution.*

Proof. We are required to show that $\pi_k P_{kj} = \pi_j P_{jk}$ for all $(j,k) \in \{0,\dots,N-1\} \times \{0,\dots,N-1\}$. Of course the statement is true for $k = j$. Suppose then $k \neq j$. Now, if $\pi_k P_{kj} = 0$ then $P_{kj} = 0$ which implies that $\Gamma^{\text{prop}}_{kj} = \Gamma^{\text{prop}}_{jk} = 0$, thus $P_{jk} = 0$ and $\pi_k P_{kj} = \pi_j P_{jk}$. On the other hand, if $\pi_k P_{kj} \neq 0$ then $P_{kj} \neq 0$ so that $\Gamma^{\text{prop}}_{kj}, \Gamma^{\text{prop}}_{jk} \neq 0$ and:

$$\pi_k P_{kj} = \pi_k \Gamma^{\text{acc}}_{kj}\Gamma^{\text{prop}}_{kj} = \pi_k \Gamma^{\text{prop}}_{kj} \min\left\{1,\ \frac{\pi_j \Gamma^{\text{prop}}_{jk}}{\pi_k \Gamma^{\text{prop}}_{kj}}\right\} = \min\left\{\pi_k \Gamma^{\text{prop}}_{kj}, \pi_j \Gamma^{\text{prop}}_{jk}\right\}$$

$$= \min\left\{\frac{\pi_k \Gamma^{\text{prop}}_{kj}}{\pi_j \Gamma^{\text{prop}}_{jk}},\ 1\right\} \pi_j \Gamma^{\text{prop}}_{jk} = \pi_j \Gamma^{\text{acc}}_{jk} \Gamma^{\text{prop}}_{jk} = \pi_j P_{jk}.$$

□

A final task is to construct convergence diagnostics of the MCMC chain. Let us first recap what we have been doing so far. We have an f-irreducible Metropolis Markov chain for which the Ergodic Theorem holds, i.e. $\lim_{K\to\infty} K^{-1}\sum_{j=1}^{K}\varphi(\boldsymbol{\sigma}^{(j)}) = \mathbb{E}_f(\varphi)$, which holds for any function φ such that $\mathbb{E}_f(\varphi) < \infty$. As a consequence, we motivated the fact that we could compute this expectation value via Monte Carlo by considering the estimator $\hat{\mu}_N^{\text{MCMC}} = N^{-1}\sum_{j=1}^{N}\varphi(\boldsymbol{\sigma}^{(j)})$. Thus the question still unanswered is to know how to properly monitor the convergence of $\hat{\mu}^{\text{MCMC}}$ to μ and how to choose the number of samples N for a proper estimation procedure. We know that the Monte Carlo error is sublinear, and so the estimator $\hat{\mu}^{\text{MCMC}}$ is biased. We note that this bias can be reduced by introducing a burn-in / warm-up period for the chain by discarding a number β of samples, i.e. considering the estimator $\hat{\mu}_{N,\beta}^{\text{MCMC}} = N^{-1}\sum_{j=\beta+1}^{N+\beta}\varphi(\boldsymbol{\sigma}^{(j)})$. Let us now focus on the variance of the MCMC estimator, where we provide the following proposition:

Proposition 5.2.3. *Consider a sufficient burn-in period so that the chain is (almost) at stationarity with $\{\boldsymbol{\sigma}^{(n)}\} \sim$ Markov(π, P). Denote by $c(k) := \text{Cov}_\pi(\varphi(\boldsymbol{\sigma}^{(0)}), \varphi(\boldsymbol{\sigma}^{(k)})) = \text{Cov}_\pi(\varphi(\boldsymbol{\sigma}^{(j)}), \varphi(\boldsymbol{\sigma}^{(j+k)}))$ (by stationarity). Then:*

$$\text{Var}_\pi\big[\hat{\mu}_N^{\text{MCMC}}\big] = \frac{\sigma^2_{\text{MCMC},N}}{N}, \quad \text{with } \sigma^2_{\text{MCMC},N} = c(0) + 2\sum_{j=1}^{N-1}\left(1 - \frac{j}{N}\right)c(j). \tag{5.13}$$

Moreover, if $\sum_{k=0}^{\infty}|c(k)| < \infty$, then:

$$\lim_{N\to\infty} N \cdot \text{Var}_\pi\big[\hat{\mu}_N^{\text{MCMC}}\big] = \sigma^2_{\text{MCMC}}, \quad \text{with } \sigma^2_{\text{MCMC}} = c(0) + 2\sum_{k=1}^{\infty} c(k). \tag{5.14}$$

Now, if $\{\boldsymbol{\sigma}^{(n)}\}_{n=1}^{N}$ were i.i.d. from the distribution π, then the variance of the crude Monte Carlo estimator $\hat{\mu}_N^{\text{MC}}$ is given by $\text{Var}\big[\hat{\mu}_N^{\text{MC}}\big] = c(0)/N$. As a result:

$$\lim_{N\to\infty} \frac{\text{Var}\big[\hat{\mu}_N^{\text{MCMC}}\big]}{\text{Var}\big[\hat{\mu}_N^{\text{MC}}\big]} = \frac{\sigma^2_{\text{MCMC}}}{c(0)} = 1 + 2\sum_{k=1}^{\infty}\frac{c(k)}{c(0)}. \tag{5.15}$$

Hence, $\hat{\mu}_N^{\text{MCMC}}$ is generally less effective than a pure i.i.d. sampling from π due to the correlation in the chain. To quantify this, we introduce the **Effective Sample Size (ESS)**:

$$\text{ESS} := N\frac{c(0)}{\sigma^2_{\text{MCMC}}} = \frac{N}{1 + 2\sum_{k=1}^{\infty}\rho_k}, \quad \text{where } \rho_k = \frac{c(k)}{c(0)} \text{ is the autocorrelation at lag } (k). \tag{5.16}$$

The ESS represents the size of an equivalent set of independent samples to the one of a set of MCMC samples that would lead to the same variance of the estimator. For instance, say the ESS is equal to 200 for an MCMC chain of 1000 correlated samples, then these correlated draws roughly carry the same amount of information as 200 samples. We then understand that, higher the ESS is, the better. This has an interesting connection to the autocorrelations in the chain, a measure that quantifies how strongly a sample is correlated to earlier ones. Thus if autocorrelation is high, the chain moves slowly and samples are expected to be redundant; however if it is low then the chain explores the distribution more efficiently. We now present a second proposition providing a Central Limit Theorem (CLT) for the Metropolis-Hastings Markov chain:

Proposition 5.2.4. *Let $\{\boldsymbol{\sigma}^{(n)}\}$ be an f-irreducible, aperiodic Metropolis-Hastings chain with invariant distribution π, density f, and $\varphi : \mathcal{X} \to \mathbb{R}$ ($\mathcal{X}$ is the state space), such that:*

$$\sigma^2_{\mathrm{MCMC}} := \mathrm{Var}_\pi(\varphi(\boldsymbol{\sigma}^{(0)})) + 2\sum_{j=1}^{\infty} \mathrm{Cov}_\pi(\varphi(\boldsymbol{\sigma}^{(0)}), \varphi(\boldsymbol{\sigma}^{(j)})) < \infty, \tag{5.17}$$

then, as $N \to \infty$, one has:

$$\sqrt{N}\big(\hat{\mu}_N^{\mathrm{MCMC}} - \mu\big) \xrightarrow{d} \mathcal{N}\big(0, \sigma^2_{\mathrm{MCMC}}\big). \tag{5.18}$$

The interesting point of this proposition is that we can provide asymptotic confidence intervals. One caveat to this is that we need to provide an estimator for σ^2_{MCMC}. One relatively simple way to find such an estimator is to recall the expression of σ^2_{MCMC} in Eq. (5.14) and use *covariance methods*. Then, if we impose a sufficient burn-in β in the chain $\{\boldsymbol{\sigma}^{(n)}\}_{n=\beta+1}^{N+\beta}$ so that it is (almost) at stationarity, a sample estimator for $c(k)$ is given by:

$$\hat{c}(k) = \frac{1}{N-k-1} \sum_{j=\beta+1}^{N+\beta-k} \big(\varphi(\boldsymbol{\sigma}^{(j)}) - \hat{\mu}_{N,\beta}^{\mathrm{MCMC}}\big)\big(\varphi(\boldsymbol{\sigma}^{(j+k)}) - \hat{\mu}_{N,\beta}^{\mathrm{MCMC}}\big), \tag{5.19}$$

ans so we can obtain plug-in *naive* estimators for the MCMC variance:

$$\hat{\sigma}^2_{\mathrm{MCMC}} = \hat{c}(0) + 2\sum_{k=1}^{N-1} \hat{c}(k). \tag{5.20}$$

Note in practice that this is **not** a *good* estimator as it is not consistent. Its variance is $\mathcal{O}(1)$, however for a *good* estimator we expect $\mathcal{O}(1/N)$. Thus in practice, one should truncate the sum in Eq. (5.20) to a level $M < N-1$. For this, C.J. Geyer[48] showed that by considering strictly positive, decreasing and convex sequences of the form $\Gamma_k := c(2k) + c(2k+1)$ for reversible Markov chains, a suitable choice for M in order to obtain *good* estimators is given by $M := 2\min\{k : \hat{c}(2k) + \hat{c}(2k+1) < 0\}$. Note that we could have also used other methods for the task of estimating $\hat{\sigma}^2_{\mathrm{MCMC}}$ such as *batch means methods*, however we will not delve into them in this manuscript.
For future reference, we also briefly present the **Gelman-Rubin statistic**[49] $\hat{R}$ to monitor the convergence of an MCMC chain. This approach is all about what we might expect when a chain has converged. If we were to start multiple chains in many different starting configurations, then in theory they should all eventually converge to the stationary distribution. Thus at some point in time, it should be impossible to distinguish the multiple chains. An effective way to assess this is to compare the variation between chains to the variation within the chains. If all the chains are, after some time, close to the stationary distribution, then the between chain variation should be close to zero.
One can see that as $N \to \infty$ and as $\hat{B} \to 0$, $\hat{R}$ approaches the value of 1. One can then reason that we should run our chains until the value of $\hat{R}$ is close to 1. Now, while the Gelman-Rubin statistic is intuitively appealing and is practically useful, in particular it is unit free since it is a ratio and hence is a good candidate to monitor convergence, it can also demonstrate some undesirable properties. We will not expose them here, however one can see, for instance, [50] for more details.

We end this section by briefly presenting another possibility for a sampling procedure with the so-called **Metropolis Adjusted Langevin Algorithm (MALA)**. It relies on the following observation: consider the Langevin dynamics $dX_t = \nabla \log f(X_t) + \sqrt{2}dW_t$ where $t > 0$, $X_t \in \mathbb{R}^d$, W_t is a standard Brownian motion and f is the target density. We consider the probability density function of X_t to be $\rho(x,t)$, i.e. $\int_A \rho(x,t)dx = \mathbb{P}(X_t \in A)$, for which it is well-known that it is solution to the Fokker-Planck equation $\partial_t \rho + \mathrm{div}(\rho \nabla \log(f)) - \Delta\rho = 0$, where for a vector $\mathbf{A} = (A_{x_1}, \ldots, A_{x_d})$ we have $\mathrm{div}(\mathbf{A}) := \nabla \cdot \mathbf{A} = \sum_{j=1}^d \partial_{x_j} A_{x_j}$.

Algorithm 3: Construction of the Gelman-Rubin Statistic

Data: Burn-in value β, number of remaining samples N, number of chains J.
for $j = 1, \dots, J$ **do**

Sample N configurations $\{\boldsymbol{\sigma_j}^{(k)}\}_{k=\beta}^{\beta+N-1}$ from the j-th Markov chain with a burn-in of β samples.
Calculate the mean $\hat{\mu}_j$ and the *within* variance $\hat{s}_j^2$ of the j-th chain, given respectively by:

$$\hat{\mu}_j = \frac{1}{N}\sum_{w=1}^{N} \boldsymbol{\sigma_j}^{(w)} \quad \text{and,} \quad \hat{s}_j^2 = \frac{1}{N-1}\sum_{w=1}^{N} \big(\boldsymbol{\sigma_j}^{(w)} - \hat{\mu}_j\big)^2. \tag{5.21}$$

end
Calculate the grand-mean $\hat{\mu}$ and the *within* chain variance $\hat{B}$, given respectively by:

$$\hat{\mu} = \frac{1}{J}\sum_{j=1}^{J} \hat{\mu}_j \quad \text{and,} \quad \hat{B} = \frac{N}{J-1}\sum_{j=1}^{J} \big(\hat{\mu}_j - \hat{\mu}\big)^2. \tag{5.22}$$

Set $\hat{W} = J^{-1}\sum_{j=1}^{J} s_j^2$ and calculate the Gelman-Rubin statistic:

$$\hat{R} := \frac{\frac{N-1}{N}\hat{W} + \frac{1}{N}\hat{B}}{\hat{W}}. \tag{5.23}$$

It is not difficult to see that $\overline{\rho}(x,t) := f(x)$ is a stationary solution of this equation since

$$\partial_t \overline{\rho} + \operatorname{div}(\overline{\rho}\nabla \log(f)) - \Delta\overline{\rho} = \sum_{j=1}^{d} \partial_{x_j}\left(\overline{\rho}\frac{\partial_{x_j} f}{f}\right) - \Delta\overline{\rho} = 0. \tag{5.24}$$

Under some mild assumptions on f, we can impose that this stationary solution is unique and $\lim_{t\to\infty}\rho(\cdot,t) = f$. Hence the Langevin dynamics have f as invariant distribution, and thus we can propose a sampling procedure based on the dynamics. However, the exact solution of the Langevin dynamics is not know in general and as a result we must rely on an Euler-Maruyama scheme as a discretisation procedure with a given time-step τ:

$$X_{n+1} = X_n + \tau\nabla \log(f(X_n)) + \sqrt{2\tau}\xi_n, \quad \text{for } \xi_n \sim \mathcal{N}(0, \mathbb{I}). \tag{5.25}$$

In other words, (X_{n+1}) has a Gaussian distribution:

$$X_{n+1} \sim \mathcal{N}\big(X_n + \tau\nabla \log(f(X_n)), 2\tau\mathbb{I}\big). \tag{5.26}$$

Note however that after discretisation, the scheme does not have f as invariant distribution anymore. Nonetheless, we can use Eq. (5.26) as a proposal density $\tilde{\boldsymbol{\sigma}} \sim \mathcal{N}\big(\boldsymbol{\sigma}^{(n)} + \tau\nabla \log\big(f\big(\boldsymbol{\sigma}^{(n)}\big)\big), 2\tau\mathbb{I}\big)$ in a Metropolis-Hastings algorithm, where the acceptance probability $\alpha(\boldsymbol{\sigma}^{(n)}, \tilde{\boldsymbol{\sigma}})$ is given by:

$$\alpha(\boldsymbol{\sigma}^{(n)}, \tilde{\boldsymbol{\sigma}}) := \min\left\{1\ ,\ \frac{f(\tilde{\boldsymbol{\sigma}})}{f(\boldsymbol{\sigma}^{(n)})}\frac{\exp\big(-\|\boldsymbol{\sigma}^{(n)} - \tilde{\boldsymbol{\sigma}} - \tau\nabla\log(f(\tilde{\boldsymbol{\sigma}}))\|^2/(2\tau)\big)}{\exp\big(-\|\tilde{\boldsymbol{\sigma}} - \boldsymbol{\sigma}^{(n)} - \tau\nabla\log\big(f(\boldsymbol{\sigma}^{(n)})\big)\|^2/(2\tau)\big)}\right\}. \tag{5.27}$$

5.2.1 Implementation - 2D Ising Model with the Metropolis-Hastings Algorithm

The goal of this subsection is to demonstrate an implementation of the Metropolis-Hastings Algorithm in order to study the energy of a classical Ising Model. We consider a lattice of $m \times m$ spins so that the number of possible states is 2^{m^2}. A specific spin σ_{ij} in position (i,j) can take the values $\{-1,+1\}$, and a system configuration S is described by the matrix $S = (\sigma_{ij}) \in \{-1,1\}^{m\times m}$. Given a configuration S, the Hamiltonian of the system is

given by:

$$H(S) = -B\sum_{i,j=1}^{m}\sigma_{ij} - \frac{1}{2}J\sum_{i,j=1}^{m}\sigma_{ij}\big(\sigma_{i-1,j}+\sigma_{i+1,j}+\sigma_{i,j-1}+\sigma_{i,j+1}\big) \tag{5.28}$$

$$= -B\sum_{i,j=1}^{m}\sigma_{ij} - J\sum_{i,j=1}^{m-1}\sigma_{ij}\big(\sigma_{i+1,j}+\sigma_{i,j+1}\big), \tag{5.29}$$

where $J > 0$ is a magnetic coupling constant (model of ferromagnetism), B is a constant for the external magnetic field, and we assume the boundary conditions $\sigma_{0,j} = \sigma_{j,0} = \sigma_{m+1,j} = \sigma_{j,m+1} = 0$. The probability of obtaining a specific system state is dictated by the Boltzmann distribution with probability mass function

$$\mathbb{P}_\beta(S) = \frac{e^{-\beta H(S)}}{\sum_S e^{-\beta H(S)}}, \tag{5.30}$$

where $\beta := 1/(k_B T)$ denotes the inverse temperature with k_B the Boltzmann constant and T the absolute temperature. We define $M(S) := m^{-2}\sum_{i,j=1}^{m}\sigma_{ij}$ the average magnetic moment of the system for a given configuration S. The computation of the *partition function* $Z_\beta := \sum_S e^{-\beta H(S)}$ being computationally expensive, we rely on the Metropolis-Hastings algorithm to generate some sample configurations. In particular, we consider a uniform proposal density $\Gamma_{ij}^{\text{prop}} = 1/m^2$ where at each step a candidate configuration is proposed by randomly choosing an atom with uniform probability, and flipping its spin. Notice that the Hamiltonian requires $\mathcal{O}(m^2)$ operations for the evaluation of the energy, which can be computationally expensive. However remark that the MH algorithm only requires to evaluate differences in the energy of two different states. In particular, denote by S the current configuration and S^{prop} the candidate state. Then, suppose that the difference between the two configurations occurs in position (r,k) so that $S = (\sigma_{ij})$, $S^{\text{prop}} = (\sigma_{ij}^{\text{prop}})$ with $\sigma_{ij}^{\text{prop}} = \sigma_{ij}\big(\mathbb{1}_{\{i\neq r, j\neq k\}} - \mathbb{1}_{\{i=r,j=k\}}\big)$. Consequently, the difference in energy is given by:

$$\begin{aligned}\Delta H(S, S^{\text{prop}}) = H(S^{\text{prop}}) - H(S) &= -J(\sigma_{rk}^{\text{prop}} - \sigma_{rk})(\sigma_{r-1,k}+\sigma_{r+1,k}+\sigma_{r,k-1}+\sigma_{r,k+1}) - B(\sigma_{rk}^{\text{prop}} - \sigma_{rk}) \\ &= 2J\sigma_{rk}(\sigma_{r-1,k}+\sigma_{r+1,k}+\sigma_{r,k-1}+\sigma_{r,k+1}) + 2B\sigma_{rk}.\end{aligned}$$

Hence, since the proposal density is symmetric, the MH acceptance rate becomes:

$$\alpha(S, S^{\text{prop}}) = \min\left\{1,\ \frac{\mathbb{P}_\beta(S^{\text{prop}})}{\mathbb{P}_\beta(S)}\right\} = \min\big\{1,\ \exp[-\beta\Delta H(S, S^{\text{prop}})]\big\}. \tag{5.31}$$

We perform a `Python` implementation of this system for $\beta = \{0.3, 0.5\}$, also with $J = 1$, $B = 0.1$, $N = 5\times 10^6$ and $m = 50$ for an Ising Model on a 50×50 lattice. For $\beta = 0.3$ in Figure (5.1), we observe that the final state still has some groups of spin-up and spin-down. However for $\beta = 0.5$ in Figure (5.2), we observe a phase transition such that (almost) all the states eventually align in the same direction.

5.3 Artificial Neural Networks

Artificial Neural Networks (ANNs), or Neural Networks (NNs) for short, are a large class of models used to process data in Machine Learning (ML) tasks. They are parametrized functions being themselves composed of many simple functions. As the name suggests, ANNs were originally constructed taking inspiration from networks of neurons that constitutes the brain. The following standard terminology is defined in [51]. ANNs are typically composed of interconnected *layers* that sequentially process information. Each layer contains multiple nodes/units, also called *artificial neurons* or *perceptrons*[52]. Each node j takes as input a vector $\mathbf{x} = (x_1, x_2, \cdots, x_m) \in \mathbb{R}^m$ corresponding to the activation of all nodes in the previous layer. It outputs a scalar

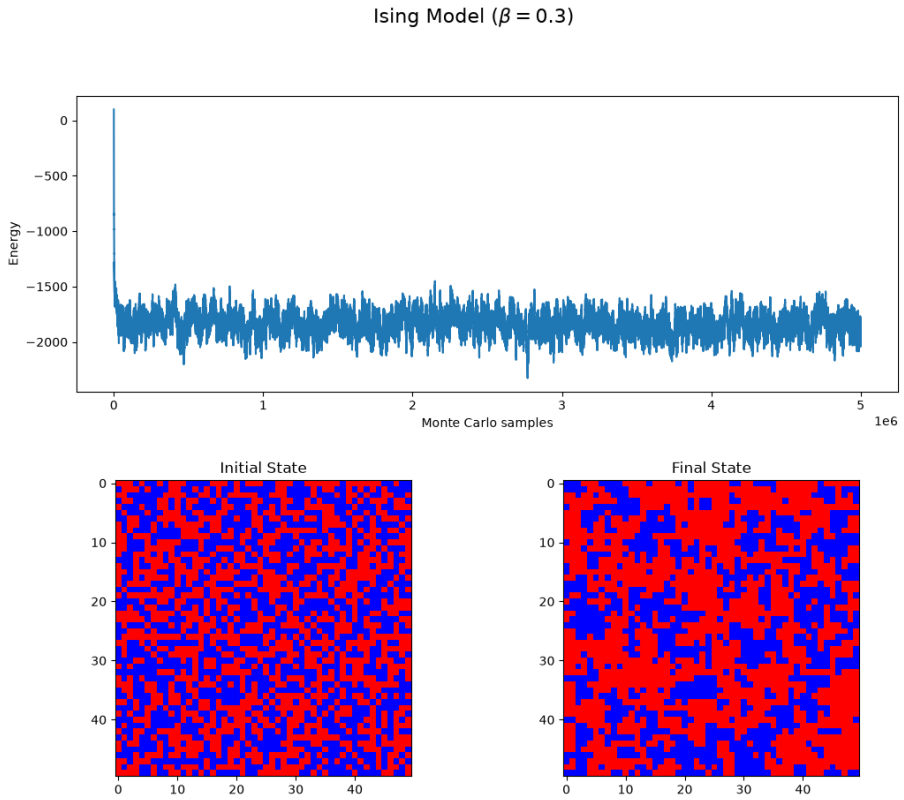


Figure 5.1: Simulation of a 2D Ising Model on a 50×50 lattice with $\beta = 0.3$.

value $a_j \in \mathbb{R}$ (its *activation*) that is computed recursively for every layer as follows:

$$a_j^{(1)} = \phi\left(\sum_k w_{kj}^{(1)} x_k + b_j^{(1)}\right), \quad a_j^{(2)} = \phi\left(\sum_k w_{kj}^{(1)} a_k^{(1)} + b_j^{(1)}\right), \quad \dots \quad a_j^{(D)} = \phi\left(\sum_k w_{kj}^{(D)} a_k^{(D-1)} + b_j^{(D)}\right),$$

where the matrix $w_{kj}^{(1)}$ and the bias vector $b_j^{(1)}$ are arbitrary parameters that are fed to the subsequent layers. The function ϕ is a non-linear function named *activation function*. Common choices are the following:

1. *Rectified Linear Unit* (ReLU):

$$\phi(z) = \max(0, z). \tag{5.32}$$

2. *Sigmoid function*:

$$\phi(z) = \frac{1}{1 + \exp(-z)}. \tag{5.33}$$

3. *Tanh function*:

$$\phi(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}. \tag{5.34}$$

Thus, from the full construction of a layer, a feed-forward neural network F can be seen as a nested composition of non-linear vector functions:

$$F(x_1, x_2, \dots, x_m) = \phi^{(D)} \circ W^{(D)} \circ \dots \circ \phi^{(2)} \circ W^{(2)} \circ \phi^{(1)} \circ W^{(1)}\mathbf{x}, \tag{5.35}$$

where the size of the matrices at each layer are referred to as the *width* of the neural network, whereas D is referred to as the *depth*. The first layer is the *input layer*, the last one is the *output layer*, and all intermediate layers are *hidden layers*. NNs where each node is by default connected to all nodes in the subsequent layer

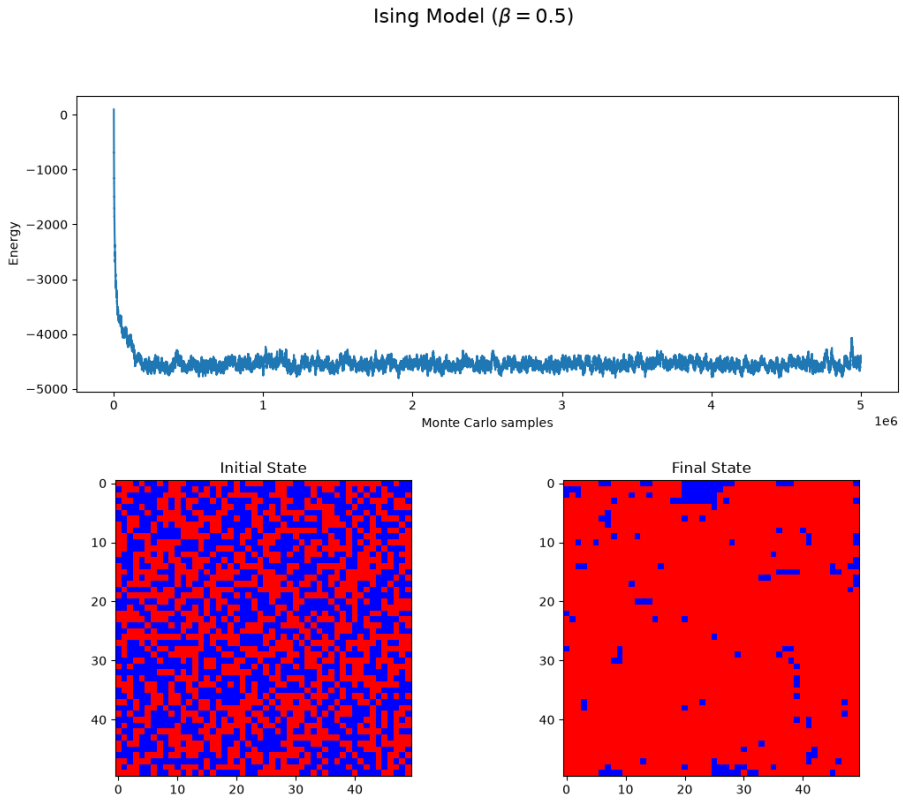


Figure 5.2: Simulation of a 2D Ising Model on a 50×50 lattice with $\beta = 0.5$.

is said to be *fully connected*. The number of layers, nodes, and their connections is known as the *architecture* of the NN. NNs are considered to be *deep* if they are composed of many hidden layers, although there is no rigorous definition to a shallow versus deep architectures. The Machine Learning methods based on deep neural networks (DNNs) as models fall under the name of *Deep Learning*. The central question for neural networks is to know what types of functions they can represent. The answer is provided by different authors, however we mention two important results: the Universal Approximation Theorem (UATs) (see for instance[53, 54, 55, 56, 57]) and the Kolmogorov-Arnold Representation Theorem. The former was originally stated for a shallow network comprising of a single hidden layer and arbitrary width.

Theorem 5.3.1 (UAT for sigmoidal activation functions[55])**.** *Let G be an n-dimensional function of real variables. Consider $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{y}_j \in \mathbb{R}^n$ and $\alpha_j, \theta_j \in \mathbb{R}$. Equivalently, set $\mathbf{x} \in \mathbb{R}^n$, $\boldsymbol{\alpha} \in \mathbb{R}^N$, $\mathbf{Y} \in \mathbb{R}^{n \times N}$ and $\boldsymbol{\theta} \in \mathbb{R}^N$. Let ϕ be any sigmoidal activation function. Then, finite sums of the form*

$$G(\mathbf{x}) = \boldsymbol{\alpha} \cdot \big(\phi \circ (\mathbf{Y} \cdot \mathbf{x} + \boldsymbol{\theta})\big) = \sum_{j=1}^{N} \alpha_j \phi(\mathbf{y}_j^T \mathbf{x} + \theta_j) \tag{5.36}$$

are dense in $C(I_n)$, the space of continuous functions on the n-dimensional unit cube $I_n = [0,1]^n$. In other words, given any $f \in C(I_n)$ and $\epsilon > 0$, there is a sum $G(\mathbf{x})$ of the above form for which

$$|G(\mathbf{x}) - f(\mathbf{x})| < \epsilon \quad \text{for all } \mathbf{x} \in I_n. \tag{5.37}$$

This statement shows that networks with a single hidden layer and an arbitrary activation function can approximate continuous functions with an arbitrary precision provided that there are no constraints on the number of neurons or the size of the weights. Alternatively, in the latter case, we can also mention the Kolmogorov-Arnold

Representation Theorem[58] stating that one can represent any function G with a polynomial number $\mathcal{O}(n^2)$ of one-dimensional nonlinear functions.

Theorem 5.3.2 (Kolmogorov-Arnold Representation Theorem)**.** *Let $G : [0,1]^n \to \mathbb{R}$ be an n-dimensional function of real variables. Consider $\mathbf{x} \in \mathbb{R}^n$. Consider some nonlinear functions ξ_i, ζ_{ij} acting on the individual components of the input $\mathbf{x} = (x_1, \ldots, x_n)$. Then, for $\zeta_{jk} : [0,1] \to \mathbb{R}$ and $\xi_j : \mathbb{R} \to \mathbb{R}$:*

$$G(\mathbf{x}) = \sum_{j=1}^{2n+1} \xi_j \left(\sum_{k=1}^{n} \zeta_{jk}(x_k) \right). \tag{5.38}$$

Note that this strongly resembles the structure of a neural network with two hidden layers. However, in practice, the nonlinear functions must be chosen carefully depending on the target functions, and also nonlinearities are typically fixed, i.e. $\xi_i = \zeta_{ij}$ for all (i, j). There are several variants of UATs for neural networks, which wont be mentioned in this manuscript. However, we still note that until now we have provided results for a fixed depth and arbitrary width, but we also have results of the converse case (fixed width and arbitrary depth) in some exotic cases; see for instance [59]. Moreover, since NNs are parametrised, weights are usually optimized via methods of Gradient Descent or ADAM[60], only to name a few.

5.4 Neural Quantum States & Stochastic Reconfiguration

In this section, we present a quantum-inspired method for the task of solving the electronic structure problem using classical neural networks. The idea is similar in nature to the variational quantum Monte Carlo method where now instead of storing the full configurations $\{\boldsymbol{\sigma}\}$ which can become very costly for large systems, we will store a vector $\boldsymbol{\theta}$ living in a variational space $\mathcal{W}$ such that $\dim(\mathcal{W}) \ll 2^N$. We thus consider a *compression* of the wavefunction which is now set to be $|\Psi_{\boldsymbol{\theta}}\rangle = \sum_{\boldsymbol{\sigma}} \psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) |\boldsymbol{\sigma}\rangle$. Of course, as before, we are still searching for the ground state of the system by minimizing the Rayleigh quotient

$$E_0 := \min_{\boldsymbol{\theta}} \langle \hat{H} \rangle_{\Psi_{\boldsymbol{\theta}}} = \min_{\boldsymbol{\theta}} \frac{\langle \Psi_{\boldsymbol{\theta}} | H | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \equiv \min_{\boldsymbol{\theta}} \frac{\sum_{\boldsymbol{\sigma},\boldsymbol{\sigma'}} \langle \Psi_{\boldsymbol{\theta}} | \boldsymbol{\sigma} \rangle \langle \boldsymbol{\sigma} | H | \boldsymbol{\sigma'} \rangle \langle \boldsymbol{\sigma'} | \Psi_{\boldsymbol{\theta}} \rangle}{\sum_{\boldsymbol{\sigma}} \langle \Psi_{\boldsymbol{\theta}} | \boldsymbol{\sigma} \rangle \langle \boldsymbol{\sigma} | \Psi_{\boldsymbol{\theta}} \rangle}. \tag{5.39}$$

In order to fully characterize the wavefunction, we must impose a specific form known as a *variational ansatz*. A first simple form is a product state $\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = \psi_{\theta_1}(\sigma_1) \cdots \psi_{\theta_N}(\sigma_N)$, however this does not encode all quantum correlations of the system and thus cannot represent a general quantum state. As an extension, one can consider a matrix product state (MPS) of the form $\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = \sum_{j_1 \cdots j_N} A^{(1)\sigma_1}_{j_1 j_2} A^{(2)\sigma_2}_{j_2 j_3} \cdots A^{(N)\sigma_N}_{j_N j_1}$ where $\{A^{(k)}\}_{k=1}^N$ are three-dimensional tensors and such that the *legs* $\{j_k\}_{k=1}^N$ encode correlations between different sites. It turns out that the MPS formalism is the most efficient structure one can use to solve the electronic structure problem for one-dimensional systems. However, in general, it is not yet clear how to deal with higher dimensional problems such as molecular systems without any clear structure. In order to overcome some of the limitations of existing variational representations, G. Carleo & M. Troyer[31] proposed an ansatz $\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = \exp(\mathrm{NN}_{\boldsymbol{\theta}}(\sigma_1, \cdots, \sigma_N))$ based on a (parametrized) Neural Network (NN). The justification to use neural networks is given by the so called Universal Approximation Theorems (UATs), proving that a neural network with a certain structure can approximate (in theory) any continuous function to any desired degree of accuracy. See Section (4.3) for more details. Note that neural network quantum states $|\Psi\rangle_{\mathrm{NN}} = \sum_{\boldsymbol{\sigma}} \psi_{\mathrm{NN}}(\boldsymbol{\sigma}) |\boldsymbol{\sigma}\rangle$ always provide valid (unnormalized) states. Furthermore, it was shown in [61] that neural quantum states can represent any MPS with polynomial resource, while they can approximate any projected entangled-pair states[62] (PEPS - generalization of MPS to higher dimensional systems) with polynomial resources.

As we mentioned previously, the idea of Monte Carlo sampling is to select the most important configurations to simplify the estimation of the ground state energy E_0. Let us now see how this can be achieved in the

context of neural quantum states. Inserting the completeness relation $\mathbb{I} = \sum_{\boldsymbol{\sigma}} |\boldsymbol{\sigma}\rangle \langle\boldsymbol{\sigma}|$ and multiplying/dividing by $\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle$ one has:

$$\langle \hat{H} \rangle_{\Psi_{\boldsymbol{\theta}}} := \frac{\langle\Psi_{\boldsymbol{\theta}}| H |\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}} \frac{\langle\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} \langle\boldsymbol{\sigma}| H |\Psi_{\boldsymbol{\theta}}\rangle = \sum_{\boldsymbol{\sigma}} \frac{|\langle\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle|^2}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} \frac{\langle\boldsymbol{\sigma}| H |\Psi_{\boldsymbol{\theta}}\rangle}{\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle} =: \mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \left[\frac{\langle\boldsymbol{\sigma}| H |\Psi_{\boldsymbol{\theta}}\rangle}{\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle} \right],$$

where $p_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) \in \mathbb{R}_+$ are the Born amplitudes and are such that $\sum_{\boldsymbol{\sigma}} p_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = 1$. Define also the local Hamiltonian as $H^{\text{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) := \langle\boldsymbol{\sigma}| H |\Psi_{\boldsymbol{\theta}}\rangle / \langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle$. Note that we are allowed to divide by $\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle$ since if we were to have $\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle = 0$ then this would also imply $\langle\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle = 0$ and so the terms would have already been cancelled in the first equality. Now, as before, given a set of samples $\mathcal{M}$ such that $|\mathcal{M}| = N_S$ we can estimate this expectation via a Monte Carlo estimator:

$$\mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \left[H^{\text{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) \right] \simeq \frac{1}{N_S} \sum_{j=1}^{N_S} H^{\text{loc}}_{\boldsymbol{\theta}}\big(\boldsymbol{\sigma}^{(j)}\big), \qquad \text{for } \{\boldsymbol{\sigma}^{(j)}\}_{j=1}^{N_S} \overset{\text{iid}}{\sim} p_{\boldsymbol{\theta}}(\boldsymbol{\sigma}). \tag{5.40}$$

In order for this procedure to be meaningful, we must ensure that $H^{\text{loc}}_{\boldsymbol{\theta}}$ can be computed in polynomial time. Now, the denominator $\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle$ satisfies this condition since it is just a query of the neural network, however the term $H|\Psi_{\boldsymbol{\theta}}\rangle$ might be trickier to evaluate since

$$H^{\text{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) := \frac{\langle\boldsymbol{\sigma}| H |\Psi_{\boldsymbol{\theta}}\rangle}{\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma'}} \langle\boldsymbol{\sigma}| H |\boldsymbol{\sigma'}\rangle \frac{\langle\boldsymbol{\sigma'}|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle},$$

and the number of matrix elements $\langle\boldsymbol{\sigma}| H |\boldsymbol{\sigma'}\rangle$ of H might be exponentially large. **Assume for now that the computation of these matrix elements is tractable** (e.g. one could assume that H is logarithmically sparse). We recall that the sampling of the configurations $\{\boldsymbol{\sigma}\}$ according to the probability distribution $p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ can be done via MCMC with a burn-in of a couple of samples, and over a sufficient number of sweeps to ensure a proper decorrelation of the configurations forming the chain.

Let us finally describe how to find the ground state from the variational principle (see Eq. (5.39)), where for this task we must compute gradients of the energy. Indeed, we seek for $E_0 = \min_{\boldsymbol{\theta}} E(\boldsymbol{\theta})$ subject to an update rule for the parameters given by gradient descent $\boldsymbol{\theta}^{(j+1)} = \boldsymbol{\theta}^{(j)} - \eta \nabla E\big(\boldsymbol{\theta}^{(j)}\big)$ with a learning rate $\eta \in \mathbb{R}_+$. Note that the wavefunction we are considering is a map $\Psi : \mathcal{W} \times \mathcal{B}(\mathcal{H}) \to \mathbb{C}$ such that $(\boldsymbol{\theta}, \boldsymbol{\sigma}) \mapsto \Psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ where $\mathcal{B}(\mathcal{H})$ is the basis of the Hilbert space $\mathcal{H}$. Assume first that the variational space $\mathcal{W}$ is real, i.e. $\mathcal{W} \subset \mathbb{R}^k$ for some $k \in \mathbb{N}$. Then, to apply the variational principle, we have to take derivatives of a complex wavefunction with respect to real parameters. However $\Psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ is in general **not** a holomorphic function. This will actually not be a problem in our case since the gradient $\nabla_{\boldsymbol{\theta}} \Psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ is always well defined (see the article on $\mathbb{CR}$-calculus by K. Kreutz-Delgado[63]on the usage of Wirtinger derivatives) and also the identity $\nabla_{\boldsymbol{\theta}} \Psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = (\nabla_{\boldsymbol{\theta}} \Psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))^*$ holds. However, if the variational space $\mathcal{W}$ is complex then the gradient alone is no longer sufficient since we have to consider derivatives with respect to $\boldsymbol{\theta}$ and also $\boldsymbol{\theta}^*$.

Remark 5.4.1. *In general Quantum Chemistry applications, whenever $\boldsymbol{\theta}$ is complex we also have that Ψ is holomorphic with respect to $\boldsymbol{\theta}$, i.e. Ψ respects the Cauchy-Riemann condition $\nabla_{\boldsymbol{\theta}^*} \Psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = 0$. In this setup, note that we would have $\langle\boldsymbol{\sigma}| \nabla_{\boldsymbol{\theta}^*} |\Psi_{\boldsymbol{\theta}}\rangle = \nabla_{\boldsymbol{\theta}^*} \Psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = 0$.*

Also, expanding to first order, note the following for the energy function $E : \mathbb{C}^k \to \mathbb{R}$:

$$\begin{aligned} |E(\boldsymbol{\theta} + \delta\boldsymbol{\theta}) - E(\boldsymbol{\theta})| = \big|\delta\boldsymbol{\theta}^T \nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta}) + \delta\boldsymbol{\theta}^{*T} \nabla_{\boldsymbol{\theta}^*} E(\boldsymbol{\theta}) + \mathcal{O}(\|\delta\boldsymbol{\theta}\|^2)\big| &= 2\big|\mathfrak{Re}\big[\delta\boldsymbol{\theta}^T \nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta})\big]\big| + \mathcal{O}(\|\delta\boldsymbol{\theta}\|^2) \\ &\leq 2\|\delta\boldsymbol{\theta}\| \cdot \|\nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta})\|, \end{aligned}$$

where we have used the fact that $\nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta}) = (\nabla_{\boldsymbol{\theta}^*} E(\boldsymbol{\theta}))^*$ since the energy is a real function, also that $|\mathfrak{Re}(z)| \leq |z|$ and the Cauchy-Schwarz inequality. Then, the $\delta\boldsymbol{\theta}$ that saturates this inequality is the one for which the

imaginary part of $\delta\boldsymbol{\theta}\nabla_{\boldsymbol{\theta}}E(\boldsymbol{\theta})$ is zero, i.e. for $\delta\boldsymbol{\theta} = -\nabla_{\boldsymbol{\theta}^*}E(\boldsymbol{\theta})$. Hence, if one wants to minimize a real valued function of complex parameters then we should follow the **conjugate** gradient rather than the usual gradient by performing a gradient descent $\boldsymbol{\theta}^{(j+1)} = \boldsymbol{\theta}^{(j)} - \eta\nabla_{\boldsymbol{\theta}^*}E\big(\boldsymbol{\theta}^{(j)}\big)$. Thus, using the quotient and product rule we obtain:

$$\nabla_{\boldsymbol{\theta}^*}E(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}^*}\frac{\langle\Psi_{\boldsymbol{\theta}}|\,H\,|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \frac{\nabla_{\boldsymbol{\theta}^*}(\langle\Psi_{\boldsymbol{\theta}}|\,H\,|\Psi_{\boldsymbol{\theta}}\rangle)}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} - \frac{\langle\Psi_{\boldsymbol{\theta}}|\,H\,|\Psi_{\boldsymbol{\theta}}\rangle\left[\nabla_{\boldsymbol{\theta}^*}(\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle)\right]}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle^2} \tag{5.41}$$

$$= \frac{\langle\nabla_{\boldsymbol{\theta}}\Psi_{\boldsymbol{\theta}}|\,H\,|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} + \frac{\langle\Psi_{\boldsymbol{\theta}}|\,H\,|\nabla_{\boldsymbol{\theta}^*}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} - E(\boldsymbol{\theta})\frac{\nabla_{\boldsymbol{\theta}^*}(\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle)}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} \tag{5.42}$$

$$= \frac{\langle\nabla_{\boldsymbol{\theta}}\Psi_{\boldsymbol{\theta}}|\,H\,|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} + \frac{\langle\Psi_{\boldsymbol{\theta}}|\,H\,|\nabla_{\boldsymbol{\theta}^*}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} - E(\boldsymbol{\theta})\frac{\langle\nabla_{\boldsymbol{\theta}}\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle + \langle\Psi_{\boldsymbol{\theta}}|\nabla_{\boldsymbol{\theta}^*}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle}. \tag{5.43}$$

Let us now compute all four terms in the previous expansion. For the very first term, we start by importing the completeness relation $\mathbb{I} = \sum_{\boldsymbol{\sigma}}|\boldsymbol{\sigma}\rangle\langle\boldsymbol{\sigma}|$ and then multiply/divide by $\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle$ in order to find:

$$\frac{\langle\nabla_{\boldsymbol{\theta}}\Psi_{\boldsymbol{\theta}}|\,H\,|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}}\frac{\langle\nabla_{\boldsymbol{\theta}}\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle\langle\boldsymbol{\sigma}|\,H\,|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}}\frac{|\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle|^2}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle}\frac{\langle\boldsymbol{\sigma}|\,H\,|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle}\frac{\langle\nabla_{\boldsymbol{\theta}}\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle} \tag{5.44}$$

$$= \sum_{\boldsymbol{\sigma}}p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\frac{\nabla_{\boldsymbol{\theta}^*}\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}{\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \tag{5.45}$$

$$= \sum_{\boldsymbol{\sigma}}p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\nabla_{\boldsymbol{\theta}^*}\log(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma})) \tag{5.46}$$

$$=: \mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left[H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\nabla_{\boldsymbol{\theta}^*}\log(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right]. \tag{5.47}$$

Remark 5.4.2. ***Warning !*** *Dividing the first term by $\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle$ is formally* ***not*** *allowed as it can yield the value zero. While this effort is to make the probability distribution $p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ appear, we still need to bear in mind that this inner product may cause some problems for specific configurations. In particular, issues typically arise when the wavefunction $\Psi_{\boldsymbol{\theta}}$ has a configuration that will not yield zero when looking at the respective derivative.*

For the second term, we perform similar steps in order to find:

$$\frac{\langle\Psi_{\boldsymbol{\theta}}|\,H\,|\nabla_{\boldsymbol{\theta}^*}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}}\frac{\langle\Psi_{\boldsymbol{\theta}}|\,H\,|\boldsymbol{\sigma}\rangle\langle\boldsymbol{\sigma}|\nabla_{\boldsymbol{\theta}^*}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}}\frac{|\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle|^2}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle}\frac{\langle\Psi_{\boldsymbol{\theta}}|\,H\,|\boldsymbol{\sigma}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle}\frac{\langle\boldsymbol{\sigma}|\nabla_{\boldsymbol{\theta}^*}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle} \tag{5.48}$$

$$= \sum_{\boldsymbol{\sigma}}p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\big[H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\big]^*\frac{\nabla_{\boldsymbol{\theta}^*}\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}{\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \tag{5.49}$$

$$= \sum_{\boldsymbol{\sigma}}p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\big[H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\big]^*\nabla_{\boldsymbol{\theta}^*}\log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})) \tag{5.50}$$

$$=: \mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left[\big[H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\big]^*\nabla_{\boldsymbol{\theta}^*}\log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right]. \tag{5.51}$$

For the third term we find:

$$\frac{\langle\nabla_{\boldsymbol{\theta}}\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}}\frac{\langle\nabla_{\boldsymbol{\theta}}\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}}\frac{|\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle|^2}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle}\frac{\langle\nabla_{\boldsymbol{\theta}}\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle} = \sum_{\boldsymbol{\sigma}}p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\frac{\nabla_{\boldsymbol{\theta}^*}\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}{\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \tag{5.52}$$

$$= \sum_{\boldsymbol{\sigma}}p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\nabla_{\boldsymbol{\theta}^*}\log(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma})) \tag{5.53}$$

$$=: \mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\big[\nabla_{\boldsymbol{\theta}^*}\log(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\big]. \tag{5.54}$$

Finally for the fourth term we obtain:

$$\frac{\langle\Psi_{\boldsymbol{\theta}}|\nabla_{\boldsymbol{\theta}^*}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}} \frac{\langle\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle\langle\boldsymbol{\sigma}|\nabla_{\boldsymbol{\theta}^*}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}} \frac{|\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle|^2}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} \frac{\langle\boldsymbol{\sigma}|\nabla_{\boldsymbol{\theta}^*}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle} = \sum_{\boldsymbol{\sigma}} p_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) \frac{\nabla_{\boldsymbol{\theta}^*}\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}{\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \tag{5.55}$$

$$= \sum_{\boldsymbol{\sigma}} p_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) \nabla_{\boldsymbol{\theta}^*} \log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})) \tag{5.56}$$

$$=: \mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left[\nabla_{\boldsymbol{\theta}^*} \log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right]. \tag{5.57}$$

Combining these four terms together yields a sum of covariances for the conjugate gradient of the energy, for the case where Ψ is not necessarily holomorphic and is defined on complex parameters:

$$\nabla_{\boldsymbol{\theta}^*} E(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left[\left(H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) - E(\boldsymbol{\theta})\right)\nabla_{\boldsymbol{\theta}^*}\log(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right] + \mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left[\left(\left[H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\right]^* - E(\boldsymbol{\theta})\right)\nabla_{\boldsymbol{\theta}^*}\log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right] \tag{5.58}$$

$$\equiv \mathrm{Cov}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left(H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}),\ \nabla_{\boldsymbol{\theta}^*}\log(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right) + \mathrm{Cov}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left(\left[H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\right]^*,\ \nabla_{\boldsymbol{\theta}^*}\log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right), \tag{5.59}$$

where we have used the fact that $\mathrm{Cov}(X,Y) = \mathbb{E}[(X-\mathbb{E}(X))Y] = \mathrm{Cov}(Y,X)$. Note that for a holomorphic wavefunction, the Wirtinger's derivative of the energy reduces to:

$$\nabla_{\boldsymbol{\theta}^*} E^{\mathrm{hol}}(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left[\left(H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) - E(\boldsymbol{\theta})\right)\left(\nabla_{\boldsymbol{\theta}}\log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right)^*\right] \tag{5.60}$$

$$\equiv \mathrm{Cov}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left(H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}),\ \left(\nabla_{\boldsymbol{\theta}}\log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right)^*\right), \tag{5.61}$$

while for a wavefunction of real parameters, using the bilinearity of the covariance, the energy gradient reduces further to:

$$\nabla_{\boldsymbol{\theta}} E^{\mathrm{real}}(\boldsymbol{\theta}) = 2\,\mathfrak{Re}\left[\mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left[\left(H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) - E(\boldsymbol{\theta})\right)\left(\nabla_{\boldsymbol{\theta}}\log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right)^*\right]\right] \tag{5.62}$$

$$\equiv 2\,\mathfrak{Re}\left[\mathrm{Cov}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left(H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}),\ \left(\nabla_{\boldsymbol{\theta}}\log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right)^*\right)\right]. \tag{5.63}$$

We note an interesting property of the (conjugate) gradient of the energy: since it has the form of a covariance, the gradients are particularly stable with respect to sampling noise. Indeed, in machine learning, continuing training for too long on a fixed dataset can lead to overfitting, where performance on the training set improves but performance on unseen data deteriorates. However this situation does not happen in the context of ground state search. Whenever gradient descent makes the wavefunction close to the ground state, $H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ does not depend on $\boldsymbol{\sigma}$ anymore since in this case we have found an eigenstate of the Hamiltonian H, i.e. $H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})_0 := \langle\boldsymbol{\sigma}|\,H\,|\Psi^0_{\boldsymbol{\theta}}\rangle\,/\,\langle\boldsymbol{\sigma}|\Psi^0_{\boldsymbol{\theta}}\rangle \simeq E_0$. Thus, the first covariance vanishes so that $\nabla_{\boldsymbol{\theta}^*}E(\boldsymbol{\theta}) = 0$ (for a holomorphic Ψ) and as a result one can end the optimisation procedure. **Assume from now on that Ψ is holomorphic**.
Let us now see how we can compute this gradient in practice. One generates samples $\{\boldsymbol{\sigma}^{(j)}\}_{j=1}^{N_S} \overset{\mathrm{iid}}{\sim} p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ from the probability distribution so that the gradient is computed via Monte Carlo:

$$\nabla_{\boldsymbol{\theta}^*} E(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{\sigma}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})}\left[\left(H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) - \mathbb{E}\left[H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma})\right]\right)\nabla_{\boldsymbol{\theta}^*}\log(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))\right] \tag{5.64}$$

$$\simeq \frac{1}{N_S}\sum_{j=1}^{N_S} \underbrace{\left(H^{\mathrm{loc}}_{\boldsymbol{\theta}}\left(\boldsymbol{\sigma}^{(j)}\right) - \frac{1}{N_S}\sum_{k=1}^{N_S} H^{\mathrm{loc}}_{\boldsymbol{\theta}}\left(\boldsymbol{\sigma}^{(k)}\right)\right)}_{=:\Delta H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(j)})} \nabla_{\boldsymbol{\theta}^*}\log\left(\psi^*_{\boldsymbol{\theta}}\left(\boldsymbol{\sigma}^{(j)}\right)\right). \tag{5.65}$$

For completeness, we recall that the samples $\left\{\boldsymbol{\sigma}^{(j)}\right\}_{j=1}^{N_S}$ may be organised inside a matrix $\Sigma \in \mathbb{R}^{N_S\times N}$, the local Hamiltonians $\left\{H^{\mathrm{loc}}_{\boldsymbol{\theta}}\left(\boldsymbol{\sigma}^{(j)}\right)\right\}_{j=1}^{N_S}$ in a vector $\mathcal{V}\in\mathbb{R}^{N_S}$ and as a result the gradient $\nabla_{\boldsymbol{\theta}^*}E(\boldsymbol{\theta})$ is still a vector in the space $\mathbb{R}^{N_p}$ for N_p the number of parameters. Then the Monte Carlo estimator of the conjugate gradient of the

energy may be equivalently re-written as follows:

$$\nabla_{\boldsymbol{\theta}^*} E(\boldsymbol{\theta}) \equiv \frac{1}{N_S} \begin{pmatrix} \sum_{j=1}^{N_S} \Delta H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(j)}) \partial_{\theta_1^*} \log\left(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(j)})\right) \\ \vdots \\ \vdots \\ \sum_{j=1}^{N_S} \Delta H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(j)}) \partial_{\theta^*_{N_p}} \log\left(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(j)})\right) \end{pmatrix} \tag{5.66}$$

$$= \frac{1}{N_S} \underbrace{\begin{pmatrix} \partial_{\theta_1^*} \log\left(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(1)})\right) & \dots & \dots & \partial_{\theta_1^*} \log\left(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(N)})\right) \\ \vdots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ \partial_{\theta^*_{N_p}} \log\left(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(1)})\right) & \dots & \dots & \partial_{\theta^*_{N_p}} \log\left(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(N)})\right) \end{pmatrix}}_{=:\, J} \underbrace{\begin{pmatrix} \Delta H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(1)}) \\ \vdots \\ \vdots \\ \Delta H^{\mathrm{loc}}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}^{(N_S)}) \end{pmatrix}}_{=:\, \mathcal{E}}. \tag{5.67}$$

Remark now that the matrix J is a transposed Jacobian, and thus in practice one can compute the entries of this matrix by using automatic differentiation[64,65] (see vector Jacobian products/pullbacks). Now, a common issue in ground state energy search is that the gradient $\nabla_{\boldsymbol{\theta}}$ can get stuck in local minima. However, the energy is quadratic with respect to the wavefunction, thus if we consider a gradient $\nabla_{\Psi_{\boldsymbol{\theta}}}$ instead then we have a (theoretical) guaranteed convergence to the ground state energy. The task would be to know if we can design a method where one would follow the gradient with respect to the wavefunction and then mapping it back to the parameter space. This is the core idea of **Stochastic Reconfiguration**[66] (aka natural gradient descent). We will first express this quadratic problem in terms of real-time evolution, that is we set $|\Psi_{\boldsymbol{\theta}(\tau)}\rangle = e^{-iH\tau} |\Psi_{\boldsymbol{\theta}(0)}\rangle$ for some random initial neural quantum state $\Psi_{\boldsymbol{\theta}(0)}$. Note that in this present form, $\Psi_{\boldsymbol{\theta}(\tau)}$ is not a neural quantum state anymore and so we must find a way to project it back into the neural quantum state formalism. First, for τ small, we can linearize the time evolution to first order so that we find $|\Psi_{\boldsymbol{\theta}(\tau)}\rangle \simeq (\mathbb{I} - i\tau H) |\Psi_{\boldsymbol{\theta}(0)}\rangle + \mathcal{O}(\tau^2)$. Recall that we want to find a relationship between the dynamics from the (large) Hilbert space to some differential equation in the variational space $\partial_\tau \boldsymbol{\theta}(\tau) = g(\boldsymbol{\theta})$ for a function g to be determined. For clarity, from now on we drop the explicit time dependence in the parameters $\boldsymbol{\theta} := \boldsymbol{\theta}(\tau)$ and $\dot{\boldsymbol{\theta}} := \dot{\boldsymbol{\theta}}(\tau)$. In order to match the time evolution, consider the following state and its first order linearization:

$$|\Psi_{\boldsymbol{\theta}+\dot{\boldsymbol{\theta}}\tau}\rangle \simeq |\Psi_{\boldsymbol{\theta}}\rangle + \tau \dot{\boldsymbol{\theta}} \cdot \nabla_{\boldsymbol{\theta}} |\Psi_{\boldsymbol{\theta}}\rangle + \mathcal{O}(\tau^2) = (\mathbb{I} + \tau \dot{\boldsymbol{\theta}} \cdot \nabla_{\boldsymbol{\theta}}) |\Psi_{\boldsymbol{\theta}}\rangle + \mathcal{O}(\tau^2). \tag{5.68}$$

Denoting by $\hat{O}^*_{\boldsymbol{\theta}} := \nabla_{\boldsymbol{\theta}^*} \log(\psi^*_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))$, note that this new state will have a different norm:

$$\|\Psi_{\boldsymbol{\theta}+\dot{\boldsymbol{\theta}}\tau}\|^2 = \langle \Psi_{\boldsymbol{\theta}} | (\mathbb{I} + \tau \dot{\boldsymbol{\theta}}^* \cdot \nabla_{\boldsymbol{\theta}^*})(\mathbb{I} + \tau \dot{\boldsymbol{\theta}} \cdot \nabla_{\boldsymbol{\theta}}) |\Psi_{\boldsymbol{\theta}}\rangle + \mathcal{O}(\tau^2) \simeq \langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle \left(1 + 2\tau \mathfrak{Re}\left[\langle \hat{O}^*_{\boldsymbol{\theta}} \rangle \dot{\boldsymbol{\theta}}\right]\right) + \mathcal{O}(\tau^2), \tag{5.69}$$

where analogously to Eq. (5.52) we used

$$\langle \Psi_{\boldsymbol{\theta}} | \nabla_{\boldsymbol{\theta}^*} |\Psi_{\boldsymbol{\theta}}\rangle = \langle \nabla_{\boldsymbol{\theta}} \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle = \sum_{\boldsymbol{\sigma}} \langle \nabla_{\boldsymbol{\theta}} \Psi_{\boldsymbol{\theta}} | \boldsymbol{\sigma} \rangle \langle \boldsymbol{\sigma} | \Psi_{\boldsymbol{\theta}} \rangle = \sum_{\boldsymbol{\sigma}} p_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) \langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle \frac{\langle \nabla_{\boldsymbol{\theta}} \Psi_{\boldsymbol{\theta}} | \boldsymbol{\sigma} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \boldsymbol{\sigma} \rangle} = \langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle \langle \hat{O}^*_{\boldsymbol{\theta}} \rangle. \tag{5.70}$$

Now, if we want to match the two states $\Psi_{\boldsymbol{\theta}(\tau)}$ and $\Psi_{\boldsymbol{\theta}+\dot{\boldsymbol{\theta}}\tau}$ one needs to find the $\dot{\boldsymbol{\theta}}$ that minimizes the distance between the two, for some well-defined distance D[67]:

$$\dot{\boldsymbol{\theta}} = \arg\min_{\dot{\boldsymbol{\theta}}} \; D\left[e^{-iH\tau} |\Psi_{\boldsymbol{\theta}}\rangle, |\Psi_{\boldsymbol{\theta}+\dot{\boldsymbol{\theta}}\tau}\rangle\right]. \tag{5.71}$$

In our case, we will consider the Fubini-Study metric defined as:

$$D(\Psi, \Phi) := \arccos \sqrt{\frac{\langle \Psi | \Phi \rangle \langle \Phi | \Psi \rangle}{\langle \Psi | \Psi \rangle \langle \Phi | \Phi \rangle}}, \tag{5.72}$$

for which, by explicitly substituting an infinitesimal change of the state $|\Psi\rangle$ as $|\Phi\rangle = |\Psi\rangle + |\delta\Psi\rangle$, the infinitesimal

form of this metric[68] is

$$D(\Psi, \Psi + \delta\Psi)^2 = \frac{\langle \delta\Psi | \delta\Psi \rangle}{\langle \Psi | \Psi \rangle} - \frac{\langle \delta\Psi | \Psi \rangle \langle \Psi | \delta\Psi \rangle}{\langle \Psi | \Psi \rangle^2}. \tag{5.73}$$

We consider this distance as it is a natural metric on the space of physical quantum states. Indeed, if we consider a state $|\Phi\rangle$ that differs from another state $|\Psi\rangle$ only by a phase factor or a normalization constant, then they should be indistinguishable in the physical space. Thus, the Fubini-Study metric is the right candidate for such a task as it is a metric on the projective Hilbert space (i.e. a space of *rays*). Formally, for a Hilbert space $\mathcal{H}$, the rays form equivalence classes with the following equivalence relation: for two states $\Psi, \Phi \in \mathcal{H}$ one has $\Psi \sim \Phi$ if and only if $\Psi = \lambda\Phi$ for $\lambda \in \mathbb{C} \setminus \{0\}$. Thus, a generic ray of the Hilbert space is $[\Psi] = \{\lambda\Psi | \lambda \in \mathbb{C} \setminus \{0\}\}$. Indistinguishability is not guaranteed with the Euclidean distance as it will be non-zero for two elements in the same ray. In our context, it could be that we have two neural networks that encode two quantum states that are identical but have different parameters, or also have two neural networks that are different but encoded by parameters that are *close* together.

Remark 5.4.3. *We note that Eq. (5.73) may be written in the following way:*

$$D(\Psi, \Psi + \delta\Psi)^2 = \frac{\langle \delta\Psi | \, P_\perp \, | \delta\Psi \rangle}{\langle \Psi | \Psi \rangle}, \quad \textit{with the orthogonal projector } P_\perp := \mathbb{I} - \frac{|\Psi\rangle \langle \Psi|}{\langle \Psi | \Psi \rangle}. \tag{5.74}$$

Equivalently, as a minimization problem:

$$\dot{\boldsymbol{\theta}} = \arg\min_{\dot{\boldsymbol{\theta}}} \| P_\perp \, |\delta\Psi\rangle \|^2. \tag{5.75}$$

We now remark a little caveat upon linearization of the two states. The linearization of the first time-evolution can be physically motivated by taking a small time step τ, however the second linearization on the neural network $\Psi_{\boldsymbol{\theta}+\dot{\boldsymbol{\theta}}\tau}$ is less reasonable since neural networks are famously nonlinear and thus they can compress the wavefunction efficiently. Nonetheless, the idea will then be to choose some time steps small enough so that the neural network in parameter space behaves (almost) linearly, otherwise the procedure will break down. Remark that we could have **not** performed this second linearization in τ to respect nonlinearity, however Eq. (5.71) would become an optimization problem that will lack a closed form solution. Now, expanding the minimisation problem we find:

$$\dot{\boldsymbol{\theta}} \simeq \arg\min_{\dot{\boldsymbol{\theta}}} \left\| P_\perp \left[(\mathbb{I} - i\tau H) \, |\Psi_{\boldsymbol{\theta}}\rangle - (\mathbb{I} + \tau \dot{\boldsymbol{\theta}} \cdot \nabla_{\boldsymbol{\theta}}) \, |\Psi_{\boldsymbol{\theta}}\rangle \right] \right\|^2 \tag{5.76}$$

$$= \arg\min_{\dot{\boldsymbol{\theta}}} \left\| P_\perp \big(iH + \dot{\boldsymbol{\theta}} \cdot \nabla_{\boldsymbol{\theta}} \big) \, |\Psi_{\boldsymbol{\theta}}\rangle \right\|^2 \tag{5.77}$$

$$= \arg\min_{\dot{\boldsymbol{\theta}}} \, \langle \Psi_{\boldsymbol{\theta}} | \left(-iH + \dot{\boldsymbol{\theta}}^* \cdot \nabla_{\boldsymbol{\theta}^*} \right) P_\perp \big(iH + \dot{\boldsymbol{\theta}} \cdot \nabla_{\boldsymbol{\theta}} \big) \, |\Psi_{\boldsymbol{\theta}}\rangle \tag{5.78}$$

$$\equiv \arg\min_{\dot{\boldsymbol{\theta}}} \Bigg[\langle \Psi_{\boldsymbol{\theta}} | \, H^2 \, |\Psi_{\boldsymbol{\theta}}\rangle + \dot{\boldsymbol{\theta}}_j^* \dot{\boldsymbol{\theta}}_k \langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle + i \dot{\boldsymbol{\theta}}_j^* \langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \, H \, |\Psi_{\boldsymbol{\theta}}\rangle - i \dot{\boldsymbol{\theta}}_k \langle \Psi_{\boldsymbol{\theta}} | \, H \, | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle \tag{5.79}$$

$$- \frac{\langle \Psi_{\boldsymbol{\theta}} | \, H \, |\Psi_{\boldsymbol{\theta}}\rangle^2}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} - i \dot{\boldsymbol{\theta}}_j^* \frac{\langle \Psi_{\boldsymbol{\theta}} | \, H \, |\Psi_{\boldsymbol{\theta}}\rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle + i \dot{\boldsymbol{\theta}}_k \frac{\langle \Psi_{\boldsymbol{\theta}} | \, H \, |\Psi_{\boldsymbol{\theta}}\rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \langle \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle - \dot{\boldsymbol{\theta}}_j^* \dot{\boldsymbol{\theta}}_k \frac{\langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle \, \langle \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \Bigg]$$

$$\equiv \arg\min_{\dot{\boldsymbol{\theta}}} \, \dot{\boldsymbol{\theta}}_k \Bigg[\dot{\boldsymbol{\theta}}_j^* \langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle - i \langle \Psi_{\boldsymbol{\theta}} | \, H \, | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle + i \frac{\langle \Psi_{\boldsymbol{\theta}} | \, H \, |\Psi_{\boldsymbol{\theta}}\rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \langle \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle - \dot{\boldsymbol{\theta}}_j^* \frac{\langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle \, \langle \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \Bigg] \tag{5.80}$$

$$=: \arg\min_{\dot{\boldsymbol{\theta}}} \, \mathcal{L}(\dot{\boldsymbol{\theta}}). \tag{5.81}$$

The timestep τ disappeared from Eq. (5.77) as it does not affect the minimisation. Moreover, the first, third, fifth and sixth term in Eq. (5.79) also disappeared in Eq. (5.80) as they are independent of $\dot{\boldsymbol{\theta}}$. Thus, we seek

for $\dot{\boldsymbol{\theta}}$ that minimizes $\mathcal{L}(\dot{\boldsymbol{\theta}})$, i.e. such that $\nabla_{\dot{\boldsymbol{\theta}}}\mathcal{L}(\dot{\boldsymbol{\theta}}) = 0$. For this condition to hold, we require:

$$\dot{\boldsymbol{\theta}}_j^* \left[\langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle - \frac{\langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle \langle \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \right] = i \left[\langle \Psi_{\boldsymbol{\theta}} | H | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle - \frac{\langle \Psi_{\boldsymbol{\theta}} | H | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \langle \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle \right]. \tag{5.82}$$

Equivalently, this yields an equation of motion for $\boldsymbol{\theta}(\tau)$:

$$\dot{\boldsymbol{\theta}}_j(\tau) = -i \sum_k (S^+)_{jk} F_k \tag{5.83}$$

where S^+ is the pseudo-inverse of S (since S could be singular). In Eq. (5.83), we have defined the variational forces F_k and the Quantum Geometric Tensor S_{jk}[69,68,70], which by convention are set to be:

$$F_k := \frac{\langle \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} | H | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} - \frac{\langle \Psi_{\boldsymbol{\theta}} | H | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \frac{\langle \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle}, \tag{5.84}$$

$$S_{jk} := \frac{\langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} - \frac{\langle \partial_{\boldsymbol{\theta}_j} \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle} \frac{\langle \Psi_{\boldsymbol{\theta}} | \partial_{\boldsymbol{\theta}_k} \Psi_{\boldsymbol{\theta}} \rangle}{\langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle}. \tag{5.85}$$

We note that the variational forces and the Quantum Geometric Tensor may be efficiently estimated via Monte Carlo sampling from the Born distribution $p_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = |\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})|^2 / \langle \Psi_{\boldsymbol{\theta}} | \Psi_{\boldsymbol{\theta}} \rangle$. In a similar manner as in the derivations of the components in Eq. (5.43), namely inserting the completeness relation $\mathbb{I} = \sum_{\boldsymbol{\sigma}} |\boldsymbol{\sigma}\rangle \langle \boldsymbol{\sigma}|$, the resulting stochastic estimators are given by:

$$F_k^{\mathrm{MC}} = \mathbb{E}_{\boldsymbol{\sigma} \sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \big[O_k^*(\boldsymbol{\sigma}) \big(H_{\boldsymbol{\theta}}^{\mathrm{loc}}(\boldsymbol{\sigma}) - \mathbb{E}_{\boldsymbol{\sigma} \sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} [H_{\boldsymbol{\theta}}^{\mathrm{loc}}(\boldsymbol{\sigma})] \big) \big] = \mathrm{Cov}_{\boldsymbol{\sigma} \sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \big[O_k^*(\boldsymbol{\sigma}),\ H_{\boldsymbol{\theta}}^{\mathrm{loc}}(\boldsymbol{\sigma}) \big], \tag{5.86}$$

$$S_{jk}^{\mathrm{MC}} = \mathbb{E}_{\boldsymbol{\sigma} \sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \big[O_j^*(\boldsymbol{\sigma}) \big(O_k(\boldsymbol{\sigma}) - \mathbb{E}_{\boldsymbol{\sigma} \sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} [O_k(\boldsymbol{\sigma})] \big) \big] = \mathrm{Cov}_{\boldsymbol{\sigma} \sim p_{\boldsymbol{\theta}}(\boldsymbol{\sigma})} \big[O_j^*(\boldsymbol{\sigma}),\ O_k(\boldsymbol{\sigma}) \big], \tag{5.87}$$

where $H_{\boldsymbol{\theta}}^{\mathrm{loc}}(\boldsymbol{\sigma})$ are the local Hamiltonians and $O_k(\boldsymbol{\sigma}) = \partial_{\boldsymbol{\theta}_k} \log(\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}))$ are the log-derivatives of the variational state[71]. In practice, one finds an advantage to use Jacobian-vector products or vector-Jacobian products to compute these quantities (see [72, Appendix B] for the details). We finally mention that we have an interesting equivalence (up to a constant factor) of the Quantum Geometric Tensor with the Fisher Information metric, where one can see [68] or [73, Appendix C] for more details.

Now, we ended up with an evolution equation for $\dot{\boldsymbol{\theta}}$ given by $\dot{\boldsymbol{\theta}} = -iS^+F =: g(\boldsymbol{\theta})$. Remark that the vector of forces F corresponds precisely to the gradient of the (local) energies with respect to the variational parameters (indeed, Eq. (5.64) and (5.86) are identical). Hence, in Stochastic Reconfiguration, the variational parameters are updated following a *modified* gradient descent update rule given by:

$$\boldsymbol{\theta}^{(j+1)} = \boldsymbol{\theta}^{(j)} - \eta^{(j)} S^+ \big(\boldsymbol{\theta}^{(j)} \big) \nabla_{\boldsymbol{\theta}^*} E \big(\boldsymbol{\theta}^{(j)} \big). \tag{5.88}$$

Another caveat to this story is that inverting the matrix S might be a tough problem for a computer. Instead, one can consider the problem $S\dot{\boldsymbol{\theta}} = -iF$ and apply an ODE iterative solver to solve the problem (e.g. Conjugate Gradient method). However, if the matrix S turns out to be singular then these solvers may brake down. In order to avoid the use of these solvers or the inversion of the matrix S, a method was developed by A. Chen & M. Heyl[74] based on the properties of the system at hand. From Eq. (5.67), up to a complex factor, we have that $F = J\mathcal{E}$ and also $S = JJ^\dagger$ is a Gram matrix where the dimensions are $J \sim (N_p, N_S)$ and $\mathcal{E} \sim (N_S)$. As a result, $\dot{\boldsymbol{\theta}} = S^+F = (JJ^\dagger)^+(J\mathcal{E})$. Now, we consider the following two propositions:

Proposition 5.4.4. *The matrices $S = JJ^\dagger \sim (N_p, N_p)$ and $K = J^\dagger J \sim (N_S, N_S)$ have the same spectrum.*

Proof. Consider the Singular Value Decomposition (SVD) of $J = U\Sigma V^\dagger$, where U, V are unitary matrices and Σ is a diagonal matrix of singular values. Then, $S = JJ^\dagger = (U\Sigma V^\dagger)(V\Sigma U^\dagger) = U\Sigma^2 U^\dagger$. On the other hand, $K = J^\dagger J = (V\Sigma U^\dagger)(U\Sigma V^\dagger) = V\Sigma^2 V^\dagger$. We thus have the same spectrum, dictated by the matrix Σ^2. □

Proposition 5.4.5. *In the context of Proposition (5.4.4), we have that* $\dot{\boldsymbol{\theta}} = S^{-1}F = JK^{-1}\mathcal{E}$.

Proof. For simplicity, we **assume** that S is non-singular so that the pseudo-inverse reduces to the inverse, i.e. $S^+ \equiv S^{-1}$ (see [74] for the full general argument). We consider the SVD of $J = U\Sigma V^\dagger$, where U, V are unitary matrices and Σ is a diagonal matrix of singular values. Recall that for the unitary matrices U and V we have that $U^\dagger U = V^\dagger V = \mathbb{I}$. Then:

$$S^{-1}F = (JJ^\dagger)^{-1}(J\mathcal{E}) = (U\Sigma^2 U^\dagger)^{-1}(U\Sigma V^\dagger \mathcal{E}) = U\Sigma^{-2}U^\dagger U\Sigma V^\dagger \mathcal{E} = U\Sigma^{-1}V^\dagger \mathcal{E},$$

and on the other hand,

$$JK^{-1}\mathcal{E} = J(J^\dagger J)^{-1}\mathcal{E} = U\Sigma V^\dagger (V\Sigma^2 V^\dagger)^{-1}\mathcal{E} = U\Sigma V^\dagger V\Sigma^{-2}V^\dagger \mathcal{E} = U\Sigma^{-1}V^\dagger \mathcal{E}.$$

□

As a result, instead of inverting the large matrix $S \sim (N_p, N_p)$ dependent on the number of network parameters, one can invert the matrix $K \sim (N_S, N_S)$ dependent on the number of samples (provided that we have a moderate N_S). This allows to consider large neural networks with many parameters at the cost of using a smaller fixed batch of samples.

We may now end this section by providing the full NQS + VMC algorithm for the task of ground state energy estimation. We have previously defined neural quantum states and showed how to improve the gradient descent procedure with stochastic reconfiguration starting from the real-time evolution. However, for the problem of ground state search, we instead want to consider the imaginary-time evolution of the system since it is known that for a sufficiently long time $\tau > 0$, we have a convergence to the ground state:

$$\lim_{\tau\to\infty} |\Psi(\tau)\rangle := \lim_{\tau\to\infty} \frac{e^{-\tau\hat{H}}|\Psi(0)\rangle}{\|e^{-\tau\hat{H}}|\Psi(0)\rangle\|} = |\psi_0\rangle\,, \tag{5.89}$$

for some trial state $\Psi(0)$ and exact ground state ψ_0. In light of this motivation, we designed an algorithm to update the parameters $\boldsymbol{\theta}$ for the stochastic reconfiguration method (see Algorithm 4 below).

Algorithm 4: Real ($\xi = i$) or imaginary ($\xi = 1$) time evolution algorithm for NQS[51]

Data: Random configuration $\boldsymbol{\theta}$ as an initialization.
for $p = 1, \ldots, N_{\text{steps}}$ **do**
  Calculate F_j and S_{jk}
  Get $\dot{\boldsymbol{\theta}}$ by inverting the equation $\sum_k S_{jk}\dot{\theta}_k = -\xi F_j$
  Update $\boldsymbol{\theta}$ using an ODE integrator
end

For a neural network $\Psi(\boldsymbol{\sigma};\boldsymbol{\theta})$, considering a configuration $\boldsymbol{\sigma}$ and vector of parameters $\boldsymbol{\theta}$, we recall the acceptance probability of the Metropolis-Hastings Algorithm:

$$\Gamma^{\text{acc}}_{kj}(\boldsymbol{\theta}) = \min\left\{1,\ \frac{|\Psi(\boldsymbol{\sigma}^{(j)};\boldsymbol{\theta})|^2}{|\Psi(\boldsymbol{\sigma}^{(k)};\boldsymbol{\theta})|^2}\frac{\Gamma^{\text{prop}}_{jk}}{\Gamma^{\text{prop}}_{kj}}\right\} \text{ if } \Gamma^{\text{prop}}_{kj}(\boldsymbol{\theta}) \neq 0, \text{ and } \Gamma^{\text{acc}}_{kj}(\boldsymbol{\theta}) = 0 \text{ if } \Gamma^{\text{prop}}_{kj}(\boldsymbol{\theta}) = 0,$$

for a proposal density Γ^{prop} to be chosen. The corresponding algorithm for energy estimation using neural quantum states and including a stochastic reconfiguration update is as follows:

Algorithm 5: Ground State Search with NQS and a Stochastic Reconfiguration Update

Data: Proposal Γ^{prop}, neural network $\Psi(\boldsymbol{\sigma};\boldsymbol{\theta})$, initial set of parameters $\boldsymbol{\theta}^{(0)}$.
Generate $\boldsymbol{\sigma}^{(0)}$ uniformly at random and evaluate the Born probability $|\Psi(\boldsymbol{\sigma}^{(0)};\boldsymbol{\theta}^{(0)})|^2$.
while $E(\boldsymbol{\theta})$ *not converged* **do**
  [Metropolis-Hastings Algorithm with neural network Ψ to generate N_S samples]
  for $n = 0, 1, \ldots$ **do**
    Generate a new candidate configuration $\tilde{\boldsymbol{\sigma}}^{(n+1)} \sim \Gamma^{\text{prop}}_{\boldsymbol{\sigma}^{(n)},\cdot}$.
    Input $\tilde{\boldsymbol{\sigma}}^{(n+1)}$ in the neural network and output $\Psi(\tilde{\boldsymbol{\sigma}}^{(n+1)};\boldsymbol{\theta}^{(m)})$.
    Generate $U \sim \text{Unif}([0,1])$.
    if $U \leq \Gamma^{\text{acc}}_{\boldsymbol{\sigma}^{(n)},\tilde{\boldsymbol{\sigma}}^{(n+1)}}(\boldsymbol{\theta}^{(m)})$ **then**
      set $\boldsymbol{\sigma}^{(n+1)} = \tilde{\boldsymbol{\sigma}}^{(n+1)}$. `/* ` $\tilde{\boldsymbol{\sigma}}^{(n+1)}$ `accepted with probability` $\Gamma^{\text{acc}}_{\boldsymbol{\sigma}^{(n)},\tilde{\boldsymbol{\sigma}}^{(n+1)}}$ `*/`
    else
      set $\boldsymbol{\sigma}^{(n+1)} = \boldsymbol{\sigma}^{(n)}$. `/* ` $\tilde{\boldsymbol{\sigma}}^{(n+1)}$ `rejected with probability` $1 - \Gamma^{\text{acc}}_{\boldsymbol{\sigma}^{(n)},\tilde{\boldsymbol{\sigma}}^{(n+1)}}$ `*/`
    end
  end
  For the N_S samples $\{\boldsymbol{\sigma}^{(j)}\}_{j=1}^{N_S}$, compute the Monte Carlo estimators of $E(\boldsymbol{\theta}^{(m)})$ and $\nabla_{\boldsymbol{\theta}^*} E(\boldsymbol{\theta}^{(m)})$:

$$E(\boldsymbol{\theta}^{(m)}) \simeq \frac{1}{N_S}\sum_{j=1}^{N_S} H^{\text{loc}}_{\boldsymbol{\theta}^{(m)}}(\boldsymbol{\sigma}^{(j)}),$$

  and

$$\nabla_{\boldsymbol{\theta}^*} E(\boldsymbol{\theta}^{(m)}) \simeq \frac{1}{N_S}\sum_{j=1}^{N_S}\left(H^{\text{loc}}_{\boldsymbol{\theta}^{(m)}}(\boldsymbol{\sigma}^{(j)}) - \frac{1}{N_S}\sum_{k=1}^{N_S} H^{\text{loc}}_{\boldsymbol{\theta}^{(m)}}(\boldsymbol{\sigma}^{(k)})\right)\nabla_{\boldsymbol{\theta}^*}\log\left(\psi^*_{\boldsymbol{\theta}^{(m)}}(\boldsymbol{\sigma}^{(j)})\right).$$

  Update the parameters via Stochastic Reconfiguration:

$$\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta S^{+}(\boldsymbol{\theta}^{(m)})\nabla_{\boldsymbol{\theta}^*} E(\boldsymbol{\theta}^{(m)}).$$

end

Remark 5.4.6. *The authors in [71] make an important remark if one is interested in the dynamics of the system for the method of time-dependent Variational Monte Carlo (tVMC). If the wavefunction and its derivatives have non-identical support, i.e. there exist configurations $\boldsymbol{\sigma}$ for which $\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ is zero but $\partial_{\boldsymbol{\theta}_k}\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ is not, then the stochastic estimates F_k^{MC} and S_{jk}^{MC} can differ from the exact quantities in Eq. (5.84) and (5.85) by bias terms b_F, b_S given by:*

$$F_k = b_F + F_k^{\text{MC}} := \sum_{\boldsymbol{\sigma}:|\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})|=0} \frac{\langle\partial_{\boldsymbol{\theta}_k}\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle\langle\boldsymbol{\sigma}|H|\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} + F_k^{\text{MC}}, \tag{5.90}$$

$$S_{jk} = b_S + S_{jk}^{\text{MC}} := \sum_{\boldsymbol{\sigma}:|\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})|=0} \frac{\langle\partial_{\boldsymbol{\theta}_j}\Psi_{\boldsymbol{\theta}}|\boldsymbol{\sigma}\rangle\langle\boldsymbol{\sigma}|\partial_{\boldsymbol{\theta}_k}\Psi_{\boldsymbol{\theta}}\rangle}{\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle} + S_{jk}^{\text{MC}}. \tag{5.91}$$

This mismatch gives a bias in the case where $\langle\boldsymbol{\sigma}|\Psi_{\boldsymbol{\theta}}\rangle$ is zero on some configurations while $\langle\boldsymbol{\sigma}|\partial_{\boldsymbol{\theta}_k}\Psi_{\boldsymbol{\theta}}\rangle$ is not. This will lead to differences between tVMC and true variational dynamics.

5.4.1 Implementation - Transverse Field Ising Model with a Jastrow Ansatz

In this subsection, we aim to estimate the ground-state of a spin-$\frac{1}{2}$ Heisenberg antiferromagnetic chain using variational Monte Carlo (VMC) and Neural Quantum States (NQS). The Hamiltonian is taken to be a two-dimensional transverse-field Ising Model (TFIM) of the form:

$$\hat{H} = -h\sum_i \sigma_i^{(X)} + J\sum_{\langle i,j\rangle} \sigma_i^{(Z)}\sigma_j^{(Z)}. \tag{5.92}$$

We would like to find the variational approximation of the ground-state of this Hamiltonian by considering an ansatz of the Jastrow-Feenberg form:

$$\langle \sigma_1\sigma_2\dots\sigma_N|\Psi_{\text{Jas}}\rangle = \exp\left[\sum_i J_i^{(1)}(\sigma_i) + \sum_{i<j} J_{ij}^{(2)}(\sigma_i,\sigma_j) + \dots + \frac{1}{p!}\sum_{i_1\neq i_2\neq\dots\neq i_p} J_{i_1\dots i_p}^{(p)}(\sigma_{i_1},\dots,\sigma_{i_p})\right], \tag{5.93}$$

for some variational parameters defined by the functions $\left\{J_{i_1,\dots,i_k}^{(k)}(\sigma_{i_1},\dots,\sigma_{i_k})\right\}_{k=1}^{p}$. Note that this ansatz generalizes the plain mean-field ansatz which does not capture any of the correlations among spins since it considers a factorized wavefunction $|\Psi_{\text{MF}}\rangle = |\Phi_1\rangle\otimes|\Phi_2\rangle\otimes\dots\otimes|\Phi_N\rangle$ with corresponding amplitudes given by $\langle\sigma_1\sigma_2\dots\sigma_N|\Psi_{\text{MF}}\rangle = \prod_{i=1}^N \Phi_i(\sigma_i)$, and where the variational parameters are the single-spin wavefunctions $\theta_{2i} = \Phi_i(\uparrow)$ and $\theta_{2i+1} = \Phi_i(\downarrow)$. It is clear that the mean-field ansatz is a specific case of the Jastrow ansatz, for $p = 1$. Indeed,

$$\langle \sigma_1\sigma_2\dots\sigma_N|\Psi_{\text{Jas}}\rangle_{p=1} = \exp\left[\sum_i J_i^{(1)}(\sigma_i)\right] = \prod_i \exp\left(J_i^{(1)}(\sigma_i)\right) \equiv \prod_i \Phi_i(\sigma_i), \quad \text{where } \Phi_i(\sigma_i) := e^{J_i^{(1)}(\sigma_i)}.$$

Furthermore, we remark that we have an exact description of the system when $p = N$. Now, although a limitation of this form is that the number of variational parameters also scales exponentially with p, in practice one still observes convergence to the ground state. It is also often the case that two-body correlations are enough to accurately describe the properties of the ground state.

Let us now perform a VMC simulation of a *simple* two-dimensional 4×4 Transverse Field Ising Model, where we will assume periodic boundary conditions, an exchange energy $J = 1$ and a transverse magnetic field $h = 1$. Simulations for this work were performed using NetKet[75,76], a software built on top of JAX[77] and Flax[78].

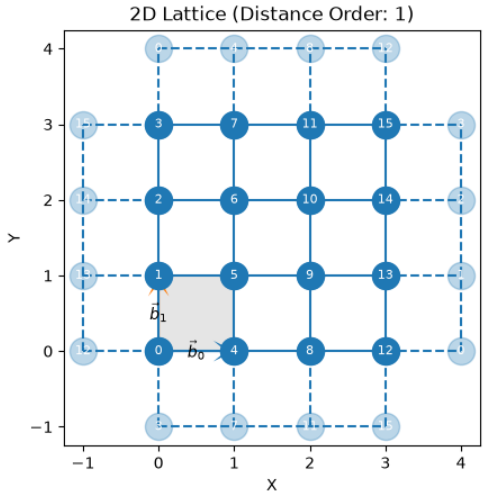


Figure 5.3: Lattice for a 2D Transverse Field Ising Model on a 4×4 lattice with periodic boundary conditions.

For our implementation, and as specified in the presentation of the Jastrow ansatz, we will only use the two-body correlations to describe the properties of the ground-state such that the ansatz is given by:

$$\langle \sigma_1 \sigma_2 \dots \sigma_N | \Psi_{\text{Jas}} \rangle = \exp\left(\sum_{i,j} \sigma_i J_{ij} \sigma_j \right). \tag{5.94}$$

We first focus on estimating the ground-state energy from variational Monte Carlo by using a Metropolis-Hastings Algorithm where the proposal density is chosen to be symmetric and consists in flipping a random spin with uniform probability in the current configuration. As we have seen before, for a specific Neural Network $\Psi(\boldsymbol{\sigma}; \boldsymbol{\theta})$, the acceptance probability reduces to:

$$\Gamma^{\text{acc}}_{kj}(\boldsymbol{\theta}) = \min\left\{ 1, \ \frac{|\Psi(\boldsymbol{\sigma}^{(j)}; \boldsymbol{\theta})|^2}{|\Psi(\boldsymbol{\sigma}^{(k)}; \boldsymbol{\theta})|^2} \right\}.$$

In NetKet, one has to define a variational function approximating the **logarithm** of the wave-function amplitudes. This variational function is usually called the *Model* and is such that $\langle \sigma_1 \sigma_2 \dots \sigma_N | \Psi \rangle = \exp(\text{Model}(\boldsymbol{\sigma}, \boldsymbol{\theta}))$. As a first approach, we do not provide a neural network and instead sample directly from the Jastrow ansatz using a probability $|\Psi(\boldsymbol{\sigma}; \boldsymbol{\theta})|^2 = |\langle \sigma_1 \sigma_2 \dots \sigma_N | \Psi_{\text{Jas}} \rangle|^2$. We then compute the energies and energy gradients via Monte Carlo estimators [see Eqs. (5.40) and (5.65)].

Remark 5.4.7. *Recall that the energies and their gradients require the computation of the local Hamiltonians:*

$$H^{loc}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) := \frac{\langle \boldsymbol{\sigma} | H | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \boldsymbol{\sigma} | \Psi_{\boldsymbol{\theta}} \rangle} = \sum_{\boldsymbol{\sigma'}} \langle \boldsymbol{\sigma} | H | \boldsymbol{\sigma'} \rangle \frac{\langle \boldsymbol{\sigma'} | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \boldsymbol{\sigma} | \Psi_{\boldsymbol{\theta}} \rangle}.$$

However, this sum is in principle defined over exponentially many entries. Instead we can remark that

$$H^{loc}_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = \sum_{\boldsymbol{\sigma'}} \langle \boldsymbol{\sigma} | H | \boldsymbol{\sigma'} \rangle \frac{\langle \boldsymbol{\sigma'} | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \boldsymbol{\sigma} | \Psi_{\boldsymbol{\theta}} \rangle} = \sum_{\boldsymbol{\sigma'} \text{ s.t. } \langle \boldsymbol{\sigma} | H | \boldsymbol{\sigma'} \rangle \neq 0} \langle \boldsymbol{\sigma} | H | \boldsymbol{\sigma'} \rangle \frac{\langle \boldsymbol{\sigma'} | \Psi_{\boldsymbol{\theta}} \rangle}{\langle \boldsymbol{\sigma} | \Psi_{\boldsymbol{\theta}} \rangle}.$$

NetKet provides an efficient tool to recover non-zero matrix elements of the Hamiltonian. Indeed, given a `Hamiltonian` H *and a configuration* `sigma` $\equiv \boldsymbol{\sigma}$*, the function* `Hamiltonian.get_conn_padded(sigma)` *returns all the configurations* $\boldsymbol{\sigma'}$ *such that* $\langle \boldsymbol{\sigma} | H | \boldsymbol{\sigma'} \rangle$ *is not zero. This function will also return the corresponding non-zero entries of the Hamiltonian. Of course, we should bear in mind that such a method is useful provided that the Hamiltonian is sufficiently sparse.*

Now, we implement Algorithm 5 with a plain Gradient Descent (GD) update, generating $N_S = 2000$ samples for Monte Carlo estimation over $m = 300$ iterations of GD with a learning rate $\eta = 5 \cdot 10^{-4}$. We plot the energy E of the system (Figure (5.4)) along with the energy errors $|E - E_{\text{GS}}|$ and relative energy errors $|E - E_{\text{GS}}|/E_{\text{GS}}$ in full- and semi-log- scale (Figure 5.5)) below. We clearly obtain some good convergence results with relative energy errors attaining values of order $\mathcal{O}(10^{-3})$, however energy errors still attain *high* values of the order of $\mathcal{O}(10^{-2})$. We also provide simulations for various number of samples, $N_S = \{5, 10, 50, 100\} \times 10^3$. For this simple 4×4 TFIM, considering more samples does not seem to improve the convergence of the energy significantly (Figure (5.6)). The only notable difference is in the energy errors (Figure (5.7)). As one should expect, the fluctuations in the Monte Carlo errors stabilize as we augment the number of samples, simply due to the Monte Carlo error rate $\epsilon_{\text{MC}} = \mathcal{O}(1/\sqrt{N_S})$ which decreases for larger sample sizes N_S.

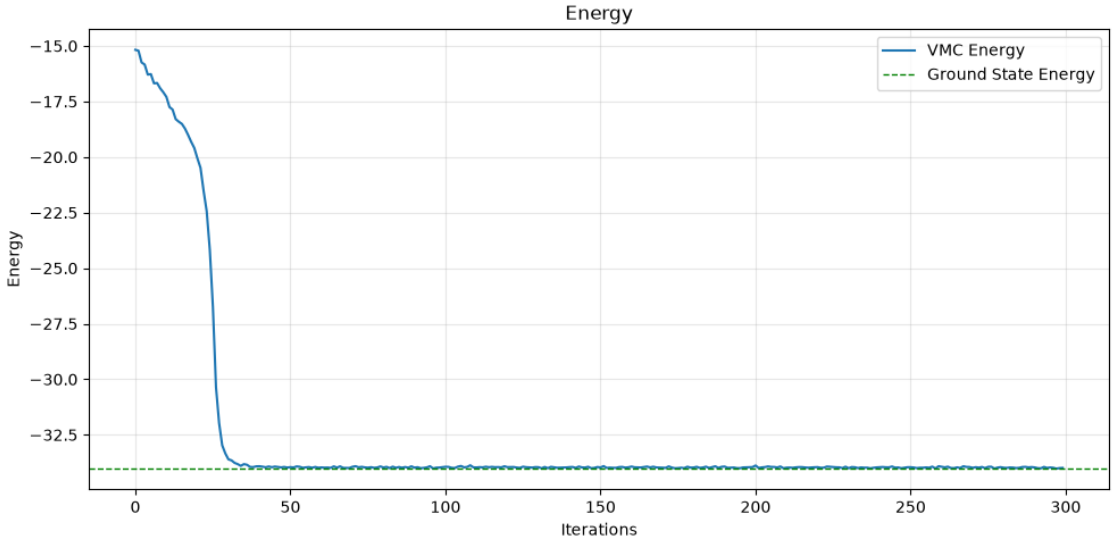


Figure 5.4: Plot of the energy using VMC via a plain Gradient Descent update with $N_S = 2000$ samples.

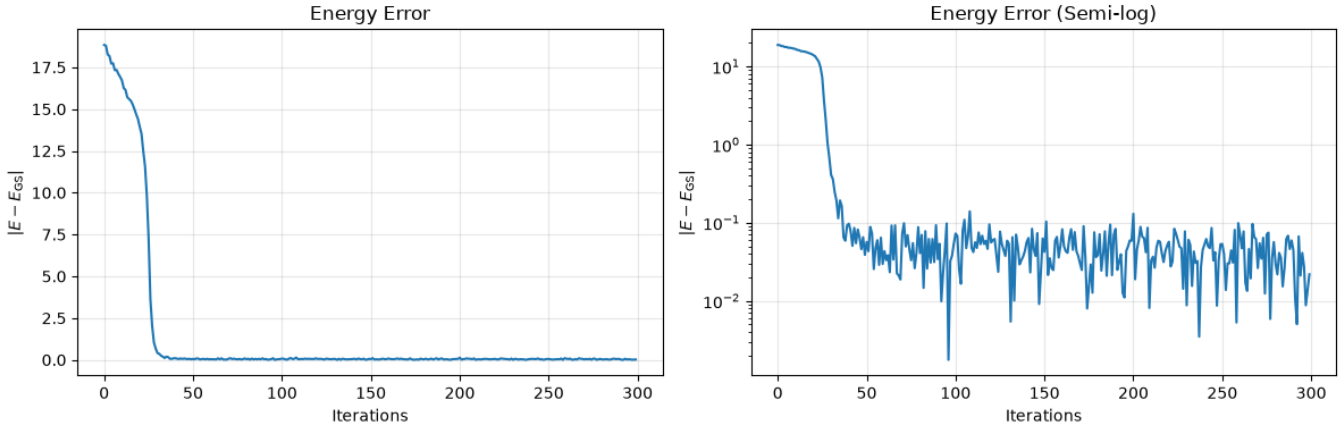


Figure 5.5: Plots of the energy errors $|E - E_{\mathrm{GS}}|$ via VMC with a plain GD update with $N_S = 2000$ samples.

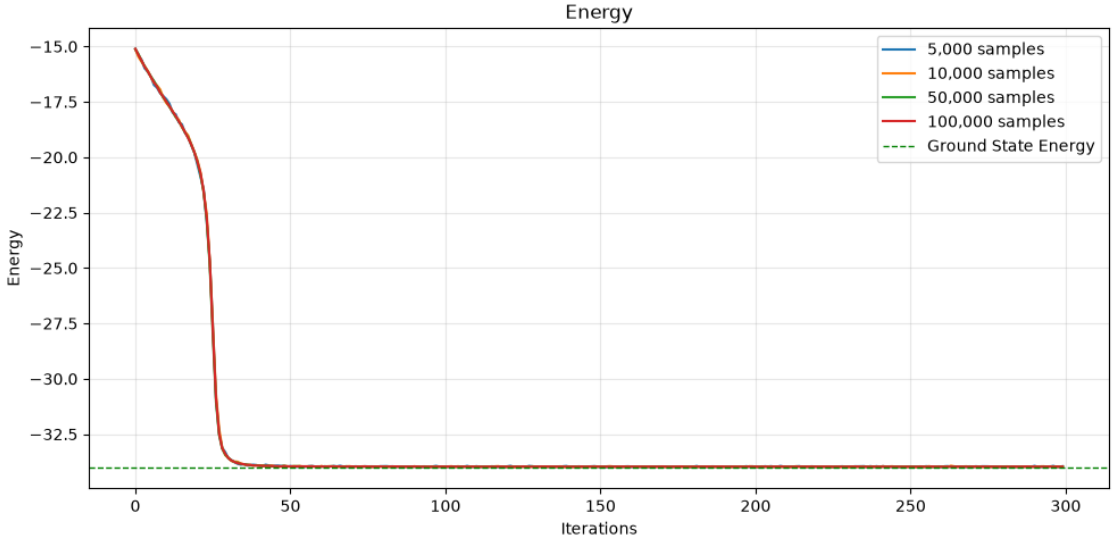


Figure 5.6: Plots of the energy E via VMC with plain GD for various Monte Carlo sample sizes.

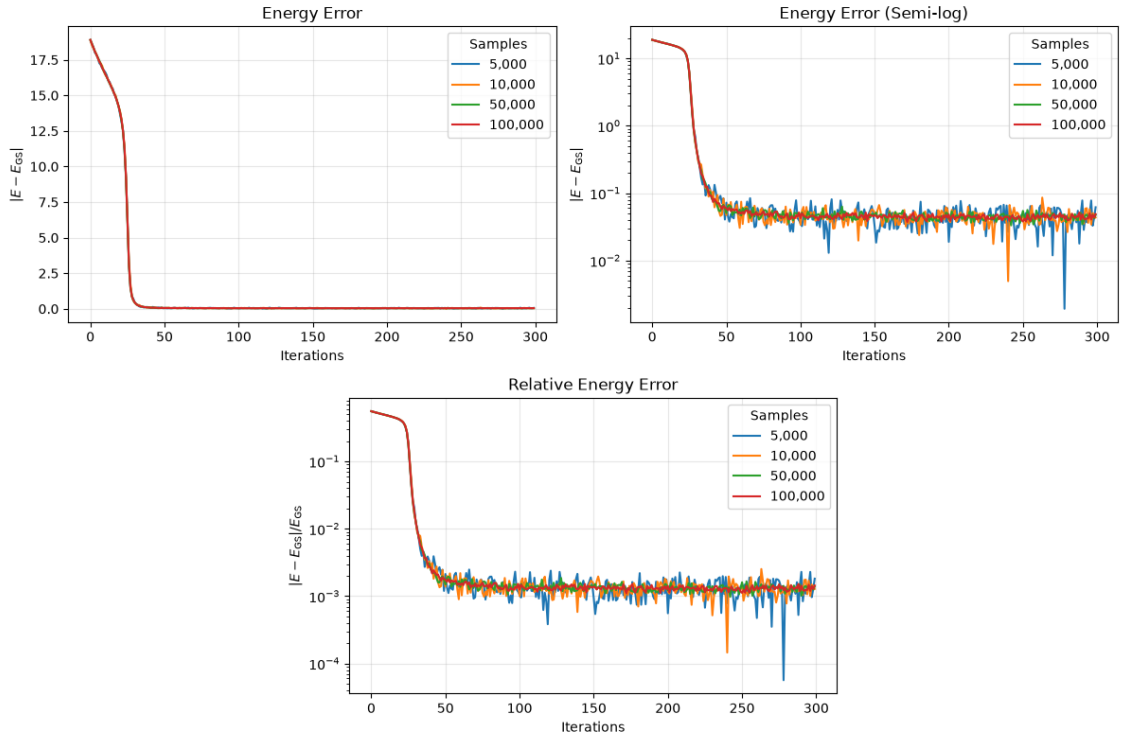


Figure 5.7: Plots of the energy errors $|E - E_{\mathrm{GS}}|$ and relative energy errors $|E - E_{\mathrm{GS}}|/E_{\mathrm{GS}}$ in full- and semi-log-scale, for various Monte Carlo sample sizes, using VMC with a plain GD update.

5.4.2 Implementation - Transverse Field Ising Model with NQS

From this point, we can take multiple directions to change the model and also tune some parameters. In this subsection, we consider the same Transverse Field Ising Model as in the previous subsection (see Figure (5.3)). First, let us create an actual NQS by defining a feedforward neural network with a ReLU activation function. As a first application, we set 2 hidden layers with 32 neurons per layer, such that the neural network provides the following mapping:

$$\mathbf{x} \mapsto \sum_{\alpha=1}^{32} W_{\alpha}^{(2)} \cdot \mathrm{ReLU}\big(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big) + \mathbf{b}^{(2)}. \tag{5.95}$$

In such a case, we are compressing the representation of a wavefunction in a Hilbert space of size $2^{16} = 65536$ with a neural network of only 1600 parameters. We implement Algorithm 5 in pure NetKet including a variational Monte Carlo simulation with a stochastic reconfiguration (SR) update. We use a learning rate of $\eta = 0.01$, a number of samples $N_S = 10000$ over a number of 20 chains and $m = 300$ iterations. We provide plots for the energy, energy errors and relative energy errors in both full- and semi-log- scale in Figure (5.8) by comparing a Mean-Field ansatz, a Jastrow ansatz (see Subsection (4.4.1)) and the feedforward neural network. The convergence of the energy in the vicinity of the ground-state energy is flagrant in all three cases, although the convergence is extremely slow for the Mean-Field ansatz. One should remark the excellent results of the feedforward network to recover the ground-state energy with errors of the order of $\mathcal{O}(10^{-4})$ and relative energy errors of order $\mathcal{O}(10^{-5})$, clearly surpassing the results for a plain Mean-Field ansatz or a Jastrow ansatz. We mention that in all of the three cases, the Gelman-Rubin statistic described in Algorithm 3 yield $\hat{R} \simeq 1$ indicating that the Metropolis Markov chains have converged to the stationary distribution. At this point, it is natural to ask whether these results can be improved even more by tuning some of the model parameters or by modifying some of the methods used in Algorithm 5. To this end, let us investigate the potential changes in the performance by modifying some characteristics and parameters.

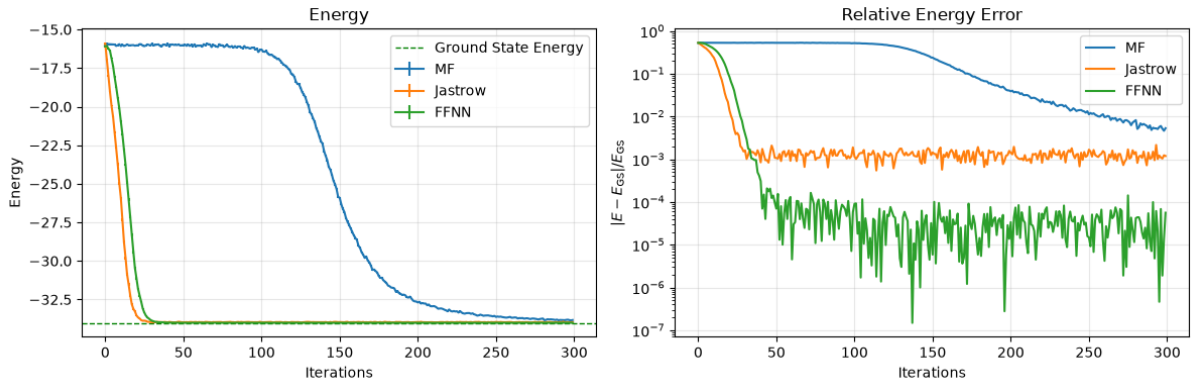


Figure 5.8: Plots of the energy E and relative energy errors (in semi-log scale) $|E - E_{\text{GS}}|/E_{\text{GS}}$ using VMC with a Stochastic Reconfiguration update.

Modifying the Optimizer: Let us first observe the impact of the update rule on the energy by modifying the optimizers. In particular, we will observe the effect of Stochastic Reconfiguration with different optimizers on the energy. In NetKet, the variational Monte Carlo update is of the form

$$\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta P_{(m)} \nabla E_{(m)}, \tag{5.96}$$

where the matrix $P_{(m)}$ is the so-called *preconditioner* at step (m). For instance, a plain stochastic gradient descent (SGD) calls for the identity $P = \mathbb{I}$, while for Stochastic Reconfiguration we set $P = S^{-1}$ to be the inverse quantum geometric tensor. We compare different optimizers with and without Stochastic Reconfiguration. The chosen optimizers for the benchmark are SGD, SGD with Momentum[79] (Algorithm 6), Root Mean Square Propagation - RMSProp (Algorithm 7), and Adam[60] (Algorithm 8). For RMSProp and Adam we have used the notation $A \odot B$ to define the Hadamard element-wise product of A and B. Adam is an efficient method for stochastic optimization that only requires first-order gradients with little memory requirement, and the authors in [60] mention that *good* defaults settings for the Adam optimizer are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. We refer the reader to the documentation for further information on the strengths and limitations of each optimizer. We plot the effect of each optimizer on the energy along with some plots of the relative errors in Figure (5.9).

Algorithm 6: SGD with Momentum Optimizer[79]

Data: Learning rate η, momentum coefficient $\mu \in [0, 1)$. Initialize the velocity $\mathcal{V}^{(0)} = 0$ and the parameters $\boldsymbol{\theta}^{(0)}$.

while $\boldsymbol{\theta}^{(m)}$ *not converged* **do**

Update velocity: $\mathcal{V}^{(m+1)} = \mu \mathcal{V}^{(m)} + \nabla E_{(m)}$.

Update parameters: $\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta \mathcal{V}^{(m+1)}$.

end

Algorithm 7: RMSProp Optimizer

Data: Learning rate η, exponential decay rate $\gamma \in [0, 1)$, small cut-off value ϵ. Initialize the weighted moving average over past squared gradients $\mathcal{S}^{(0)} = 0$ and the parameters $\boldsymbol{\theta}^{(0)}$.

while $\boldsymbol{\theta}^{(m)}$ *not converged* **do**

Update weighted moving average over past squared gradients:

$$\mathcal{S}^{(m+1)} = \gamma \mathcal{S}^{(m)} + (1 - \gamma) \nabla E_{(m)} \odot \nabla E_{(m)}.$$

Update parameters: $\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \frac{\eta}{\sqrt{\mathcal{S}^{(m+1)} + \epsilon}} \nabla E_{(m)}$.

end

Algorithm 8: Adam Optimizer[60]

Data: Learning rate η, exponential decay rates for moment estimates $\beta_1, \beta_2 \in [0, 1)$, initial parameter vector $\boldsymbol{\theta}^{(0)}$, small cut-off value ϵ. Initialize the first moment vector $\mathcal{M}^{(0)} = 0$ and the second moment vector $\mathcal{V}^{(0)} = 0$.

while $\boldsymbol{\theta}^{(m)}$ *not converged* **do**

Update biased first moment estimate: $\mathcal{M}^{(m+1)} = \beta_1 \mathcal{M}^{(m)} + (1 - \beta_1)\nabla E_{(m)}$.

Update biased second moment estimate: $\mathcal{V}^{(m+1)} = \beta_2 \mathcal{V}^{(m)} + (1 - \beta_2)\nabla E_{(m)} \odot \nabla E_{(m)}$.

Compute bias-corrected first moment estimate: $\widehat{\mathcal{M}}^{(m+1)} = \left(1 - \beta_1^{m+1}\right)^{-1} \mathcal{M}^{(m+1)}$.

Compute bias-corrected second moment estimate: $\widehat{\mathcal{V}}^{(m+1)} = \left(1 - \beta_2^{m+1}\right)^{-1} \mathcal{V}^{(m+1)}$.

Update parameters: $\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta \frac{\widehat{\mathcal{M}}^{(m+1)}}{\sqrt{\widehat{\mathcal{V}}^{(m+1)}} + \epsilon}$.

end

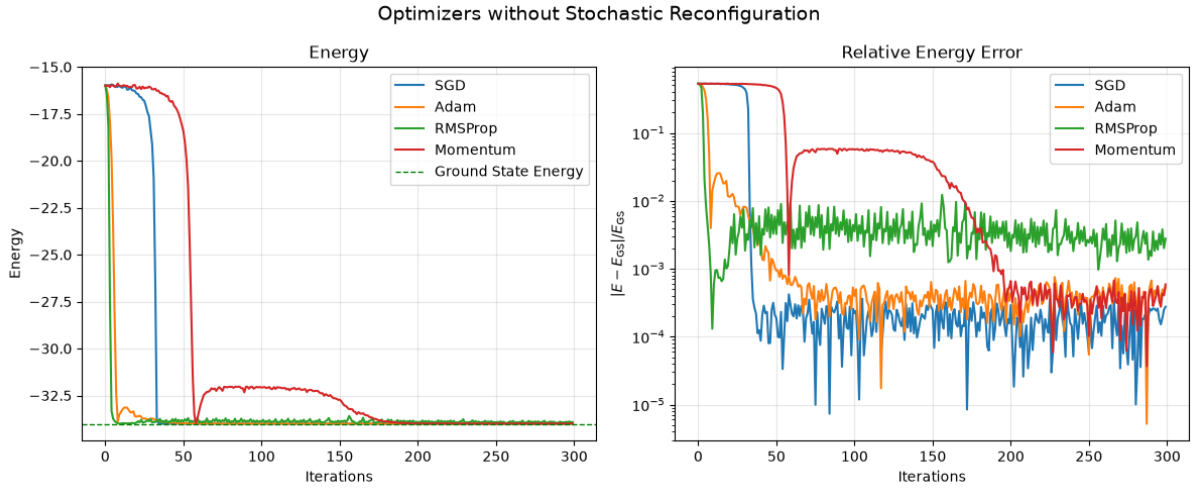


Figure 5.9: Plots of the energy E and relative energy errors (in semi-log scale) $|E - E_{\text{GS}}|/E_{\text{GS}}$ using VMC without a Stochastic Reconfiguration as preconditioner. We look at Stochastic Gradient Descent (SGD), Adam, RMSProp and SGD with Momentum.

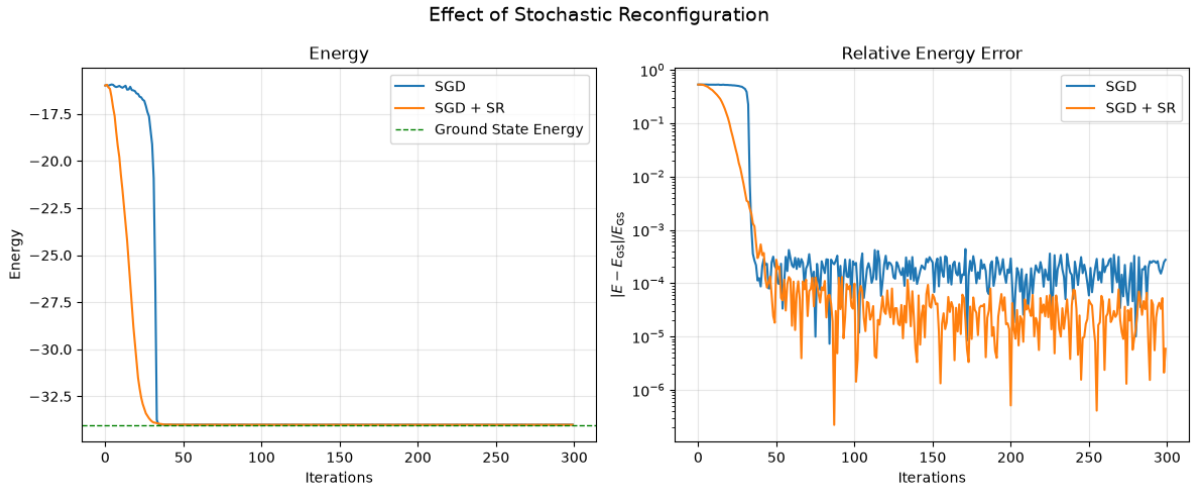


Figure 5.10: Plots of the energy E and relative energy errors (in semi-log scale) $|E - E_{\text{GS}}|/E_{\text{GS}}$ using VMC to observe the effect of Stochastic Reconfiguration as preconditioner on Stochastic Gradient Descent (SGD).

The implementation was carried out with a learning rate of $\eta = 0.01$ for all optimizers except for SGD with Momentum. Indeed with a Momentum optimizer, we have found that a learning rate of $\eta = 0.01$ or $\eta = 0.0001$ made the energy diverge from the target ground-state energy. For this optimizer, we have thus chosen a learning

rate $\eta = 0.001$. From the plots and for this *simple* TFIM, we find that SGD and Adam perform well with a fast convergence to order of relative errors of $\mathcal{O}(10^{-4})$. An SGD with Momentum also performs at a similar rate, however it takes a considerable amount of iterations before we can observe a convergence of the energy to the ground-state energy. Concerning RMSProp, we also observe a convergence of the energy, however the relative error rate is worsen at a level of $\mathcal{O}(10^{-3})$. It is still to be investigated whether a plain SGD would still converge to such low error rates for larger and more complex physical systems. However, as we can see in Figure (5.10), if we now introduce Stochastic Reconfiguration as a preconditioner to a plain SGD, we observe that the rates of relative errors are even lower. Thus, with such an NQS, Stochastic Reconfiguration leads the optimization procedure to a finer estimation of the ground-state energy. For completeness, we also provide some simulations of the energy where the optimization procedure combines the previously defined optimizers along with Stochastic Reconfiguration as a preconditioner. The idea is to build the effective direction of the optimization via $\mathcal{D}_{(m)} := S^{-1}_{(m)} \nabla E_{(m)}$ and replace all the *plain* gradients by the SR update $\mathcal{D}_{(m)}$. This will lead to new combinations of optimizers of the form {Optimizer + SR}. For example, Momentum + SR would have a velocity of the form $\widetilde{\mathcal{V}}^{(m+1)} = \mu \mathcal{V}^{(m)} + \mathcal{D}_{(m)}$ leading to a parameter update given by $\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta \widetilde{\mathcal{V}}^{(m+1)}$. As we can see in Figure (5.11), incorporating Stochastic Reconfiguration as a preconditioner improves the convergence overall to rates approching orders of $\mathcal{O}(10^{-5})$. We have already observed this fact previously for a *plain* SGD, but we can see that it is also the case for Adam and SGD with Momentum. As an added bonus, we see that the SGD with Momentum optimizer converges in fewer iterations with a Stochastic Reconfiguration as preconditioner than without. Concerning RMSProp however, even with a preconditioner, the convergence does not seem to improve significantly.

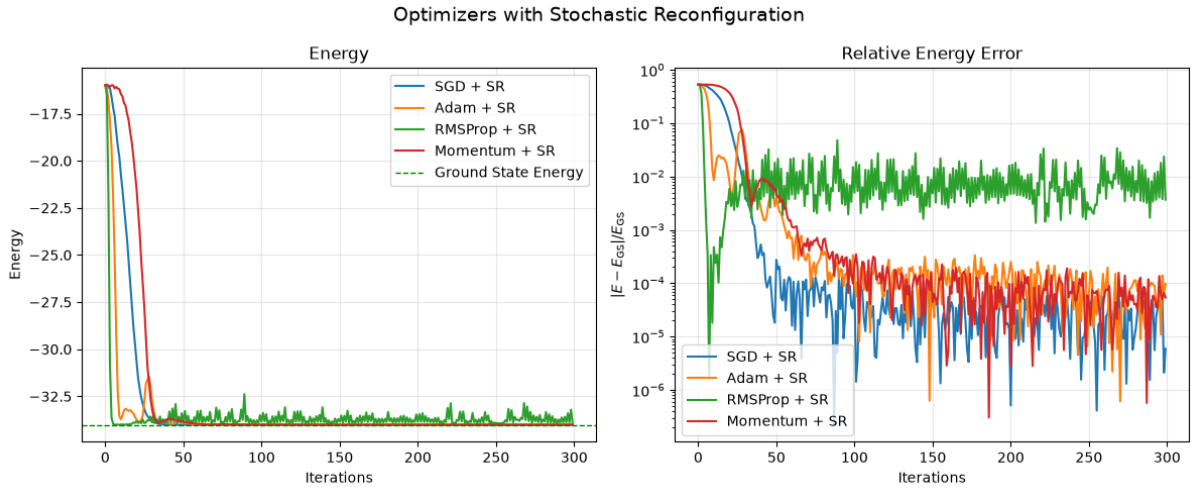


Figure 5.11: Plots of the energy E and relative energy errors (in semi-log scale) $|E - E_{\text{GS}}|/E_{\text{GS}}$ using VMC with a Stochastic Reconfiguration as preconditioner. We look at Stochastic Gradient Descent (SGD), Adam, RMSProp and SGD with Momentum.

Modifying the Learning Rate: Let us now investigate the effect of the learning rate on a SGD with Stochastic Reconfiguration update. We choose this model as we have seen in the previous point that it yields the best relative error rates, leading to a finer estimation of the ground-state energy. We plot the energy and relative energy errors for learning rate values $\{0.1, 0.05, 0.01, 0.005, 0.001\}$ over $m = 3000$ iterations, where the results are displayed in Figure (5.12). As one should expect, smaller learning rate values lead to slower convergence, thus requiring considerably more iterations before a convergence in the energy may be observed. Moreover, we find that considering small learning rates in SGD + SR does not necessarily imply an improvement in the relative energy errors. We thus conclude that, for SGD + SR, a learning rates as small as $\eta = 0.001$ or $\eta = 0.005$ are not optimal for energy estimation. For an improved clarity in the curves, Figure (5.13) shows the relative energy errors with an implementation of a Gaussian filter to smooth the curves. Although each of the values of η produces similar rate of convergence in the energy errors, it does seem that values of $\eta \in \{0.1, 0.05, 0.01\}$ are

reasonable choices for the simulations. Indeed, we have generated the median of the smoothed error over the final 20% of the data. This median for $\eta \in \{0.1, 0.05, 0.01\}$ is of the order of $\mathcal{O}(10^{-6})$ while for $\eta \in \{0.005, 0.001\}$ the median sits around $\mathcal{O}(10^{-5})$. Remark that the full simulation took ~ 20 minutes, performed on an Apple Silicon M5 Pro chip on 48Gb of RAM, 15-core CPU and 16-core GPU. We finally note that, in practice, one could adopt a strategy of varying the learning rate *on-the-fly* during the optimization procedure. In particular, one could start with a *large* learning rate to accelerate the initial convergence, and gradually decrease it towards the end of the optimization to allow for more precise convergence towards the ground-state energy.

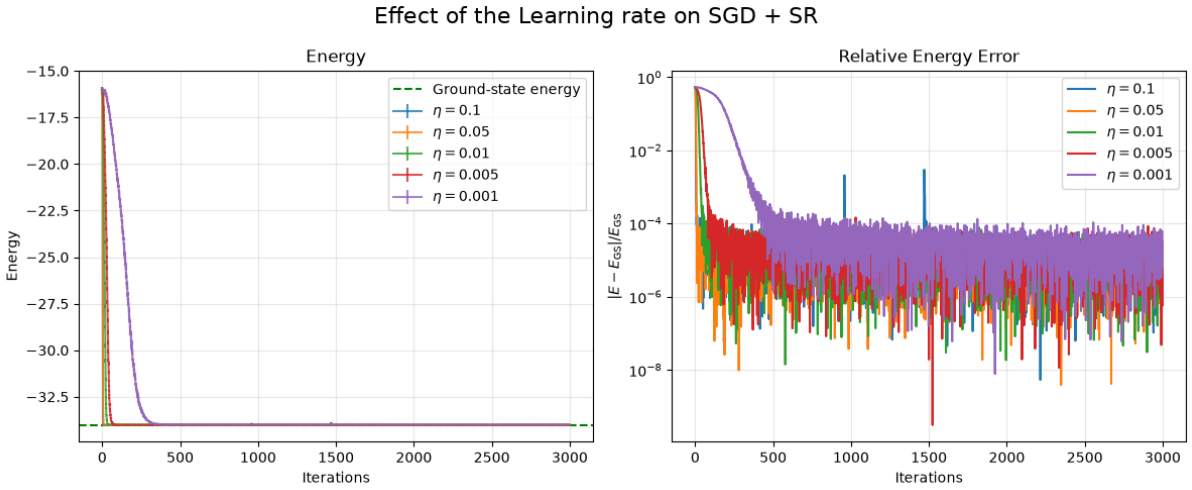


Figure 5.12: Plots of the energy E and relative energy errors (in semi-log scale) $|E - E_{\text{GS}}|/E_{\text{GS}}$ using VMC with SGD update on $N_S = 10000$ samples and a Stochastic Reconfiguration as preconditioner, over $m = 3000$ iterations.

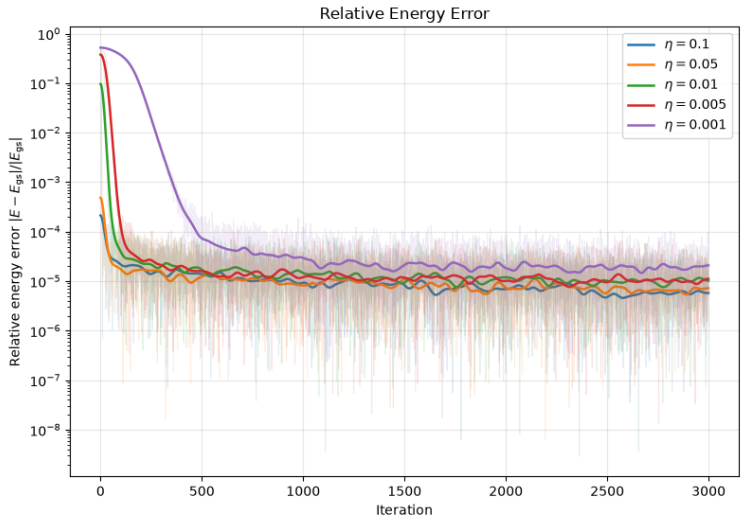


Figure 5.13: Plots of the relative energy errors (in semi-log scale) $|E - E_{\text{GS}}|/E_{\text{GS}}$ using VMC with SGD update on $N_S = 10000$ samples and a Stochastic Reconfiguration as preconditioner, over $m = 3000$ iterations.

Modifying the Model Parameters: We finally end the study of this *simple* TFIM by observing the effect of modifying the architecture of the neural network by varying the number of neurons and number of hidden layers in the model. For benchmarking, we use six different models, all referenced in Table (5.1). We present the results on the energy and relative energy errors in Figures (5.14) and (5.15). The simulations have been executed on a machine with an Apple Silicon M5 Pro chip on 48Gb of RAM, 15-core CPU and 16-core GPU.

Neural Network Model	# Neurons	# Hidden Layers	# Variational Parameters
FNN1	32	1	544
FNN2	32	2	1600
FNN3	32	3	2656
FNN4	16	2	544
FNN5	32	2	1600
FNN6	64	2	5248

Table 5.1: Comparison of feedforward neural network (FNN) models. Models {FNN1, FNN2, FNN3} investigate the fixed width with varying depth case, while the models {FNN4, FNN5, FNN6} investigate the case with fixed depth with varying width.

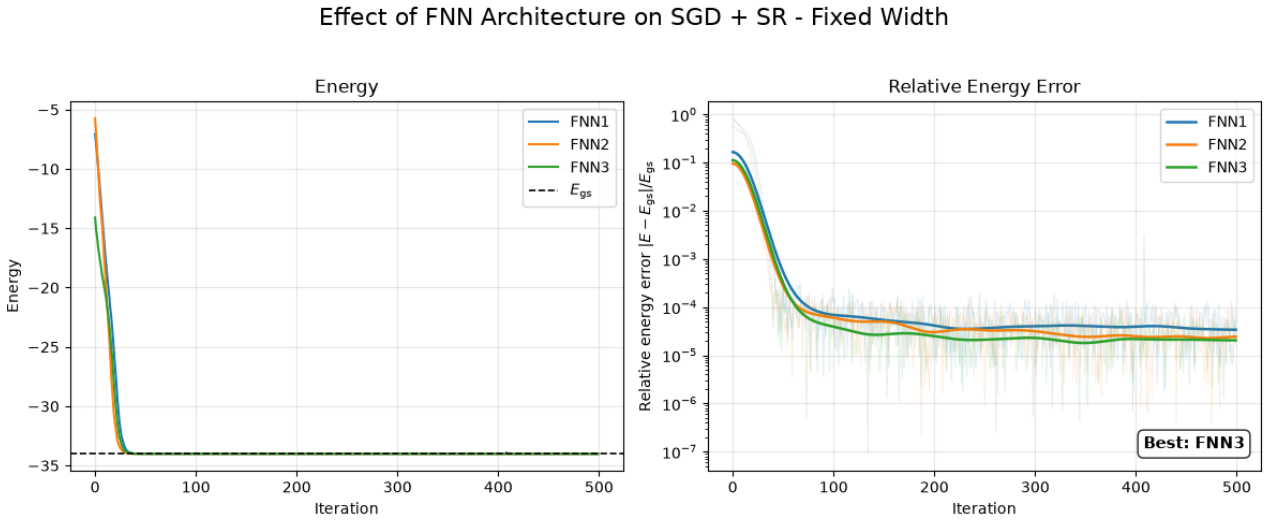


Figure 5.14: Plots of the energy and relative energy errors (in semi-log scale) $|E - E_{\text{GS}}|/E_{\text{GS}}$ using VMC with SGD update and a Stochastic Reconfiguration as preconditioner for fixed width neural network models, and a ReLU activation function. To be able to interpret the results, we smooth the errors with a Gaussian filter, with parameters $N_S = 10000$ samples over $m = 500$ iterations.

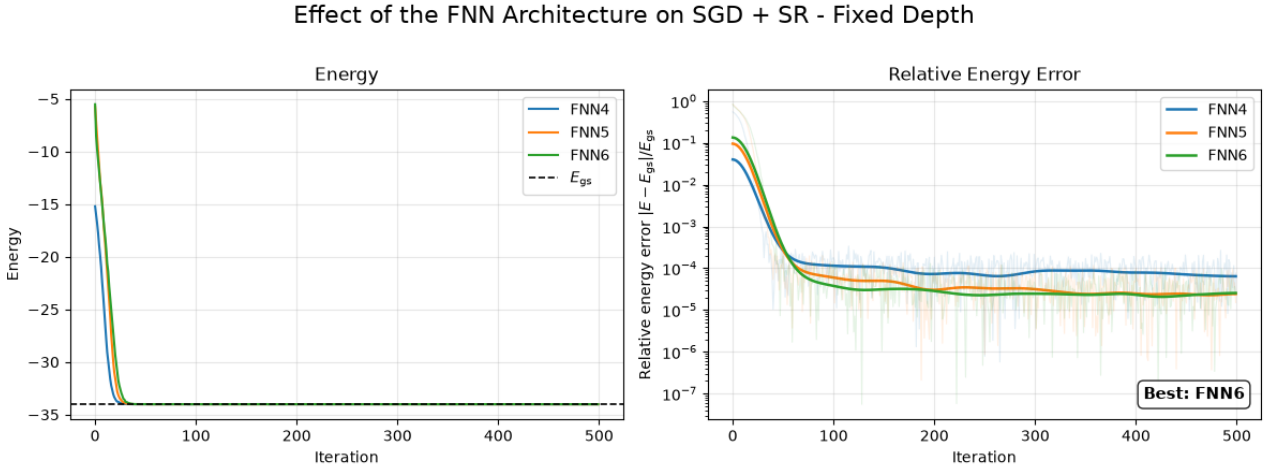


Figure 5.15: Plots of the energy and relative energy errors (in semi-log scale) $|E - E_{\text{GS}}|/E_{\text{GS}}$ using VMC with SGD update and a Stochastic Reconfiguration as preconditioner for fixed depth neural network models, and a ReLU activation function. To be able to interpret the results, we smooth the errors with a Gaussian filter, with parameters $N_S = 10000$ samples over $m = 500$ iterations.

As usual, we can see that all the energies have converged to the ground-state with a varying degree of precision. As a matter of clarity, the true dynamics of the relative errors are displayed with partial transparency, while we have smoothed out the curves with a *Gaussian filter*. Such a filter applies a weighted moving average for some weights following a Gaussian distribution. This procedure gives more weight to points in a small neighbourhood of a reference point and less to those further away. This filter allows to interpret the dynamics of the errors from a global point of view, discarding sharp spikes and delaying sudden transitions in the curves. Let us now investigate the effect of the architecture on the estimation of the ground state energy. For a fixed width, here chosen to be 32 neurons, we remark some improvements in adding more hidden layers to the network. Although the order of magnitude for the errors remain at $\mathcal{O}(10^{-5})$, we still observe an advantage to input more layers. For instance, the median of the smoothed error over the final 20% of the data yields $3.64 \cdot 10^{-5}$ for FNN1, $2.38 \cdot 10^{-5}$ for FNN2 and $2.11 \cdot 10^{-5}$ for FNN3, indicating that we expect improved results in the errors for a feedforward neural network with an increasing number of hidden layers. With similar observations, we can also remark improved results in the relative energy errors for neural networks with a fixed depth, here chosen to be 2 hidden layers. Again, while the errors remain in orders of magnitude of $\mathcal{O}(10^{-5})$, we still observe an advantage to input more neurons. Again, the median of the smoothed error over the final 20% of the data yields $7.08 \cdot 10^{-5}$ for FNN4, $2.39 \cdot 10^{-5}$ for FNN5 and $2.27 \cdot 10^{-5}$ for FNN6, indicating that we expect improved results in the errors for a feedforward neural network with an increasing number of neurons. As a result, we conclude that one can expect some improved performance in the relative energy errors by increasing the complexity of the networks. Considering more general deep neural networks with tens or hundreds of layers and thousands of neurons will almost surely improve error rates. Remark that the number of variational parameters N_p in our models do not surpass the number of samples $N_S = 10000$ in the VMC simulation. As a result, NetKet used the quantum geometric tensor (QGT) based formulation of SR, where we recall that for variational parameters $\mathbf{W}$, the QGT S quantifies an infinitesimal change in the parameters via $D(\Psi_{\mathbf{W}}, \Psi_{\mathbf{W}+\delta\mathbf{W}}) = (\delta\mathbf{W})^\dagger S(\delta\mathbf{W})$ for D the Fubini-Study metric. However, for a feedforward neural network with 3 layers and 128 neurons on each layer, the corresponding number of variational parameters is $N_p = 13650 > 10000 = N_S$ and thus in this case it uses a `minSR` method (see Proposition (5.4.5) for more details). For a flexible implementation, one is invited to use the driver `netket.driver.VMC_SR` for a fully automatised selection of the optimal method relevant to a specific scenario, choosing between `QGT` and `minSR`.

We finally mention that it is still to be investigated whether increasing the number of neurons / hidden layers yields highly advantageous results, as the computational power for deep neural networks grows significantly with an increase in the complexity. Indeed, simulation times for the above neural network models ranged from 30 seconds to 6 minutes. However, O. Sharir et al. in [80] have performed similar calculations on a (21×21) TFIM in two dimensions, with a network depth of 20 leading to around one million variational parameters, which took hundreds of hours of computational time to complete.

5.5 Advanced Topics in Neural Quantum States

In this final section, we briefly describe a few topics on extensions of Neural Quantum States to more complex systems. We will look at NQSs for systems of fermions, symmetry-aware NQSs, results on the representation of tensor networks (MPS, PEPS) with NQSs, and finally an application of NQSs to the study of open quantum systems. We mention an article providing a complete and detailed review of the applicability of NQSs in [81].

5.5.1 Symmetry-Aware Neural Quantum States

In practice, to study a quantum mechanical system with neural quantum states, one can in principle use any neural network architecture and use it for the problem at hand. However, in some instances, it may be advantageous to construct some models that are said to be *symmetry-aware*. This is in fact a popular field of study, named *Physics-informed Machine Learning*, in which one constructs *Physics-informed Neural Networks* to

encode a certain amount of physical information about a system. For instance, most of the lattice Hamiltonians contain lattice symmetries such as translational symmetries. Thus, in order for a neural network to be aware of these symmetries, such an architecture is required to *learn* to gives us back some translationally invariant solution. However, one major question is: *Why does it need to learn the symmetry, and why cannot we simply enforce it instead?* We can always start from an arbitrary non-symmetric (NS) neural network $\Psi_{\text{NS}}(\mathbf{x})$, and if one is interested in providing a symmetric version of this NN, say with respect to the group $\mathcal{G}$ of translations, then we can consider:

$$\Psi_{\text{SYMM}}(\mathbf{x}) = \sum_{g\in\mathcal{G}} \Psi_{\text{NS}}\big(T_g \cdot \mathbf{x}\big), \tag{5.97}$$

where $\{T_g\}_{g\in\mathcal{G}}$ are all the translations of the state.

Example 5.5.1. *Considering a 4-spin system, summing over all the elements of the group $\mathcal{G}$, the symmetric neural network counterpart for the configuration $\mathbf{x} = \uparrow\downarrow\uparrow\uparrow$ is given by:*

$$\Psi_{\text{SYMM}}(\mathbf{x}) = \Psi_{\text{NS}}(\uparrow\downarrow\uparrow\uparrow) + \Psi_{\text{NS}}(\downarrow\uparrow\uparrow\uparrow) + \Psi_{\text{NS}}(\uparrow\uparrow\uparrow\downarrow) + \Psi_{\text{NS}}(\uparrow\uparrow\downarrow\uparrow), \tag{5.98}$$

for some translations $\{T_{g_0} \equiv \mathbb{I}, T_{g_1}, T_{g_2}, T_{g_3}\}$.

Of course, this is a useful method if the representation of the symmetry group is polynomial in the space. Although it is still a costly operation to perform, where for instance the size of the group of translations in a two-dimensional lattice is $\dim(\mathcal{G}_{2d}) = L^2$ for L the size of the lattice, the symmetrisation in Eq. (5.97) is always well-defined. To encode symmetries, a smarter alternative is to consider neural networks that are already permutationally equivariant with respect to some symmetry group. For instance, *Convolutional Neural Networks* (CNNs) are already equivariant under translations. However, a CNN cannot analyse certain groups of symmetries such as *point group symmetries* (i.e. mirror symmetries and rotations), or even continuous ones such as $SU(2)$ or $SO(3)$, particularly relevant for Quantum Chemistry. Nonetheless, the specific advantage is that point group symmetries do not grow with the system size (e.g. in two-dimensional systems, we have two mirror symmetries and four rotations for any lattice size L). Thus, in place of the symmetrised neural network in Eq. (5.97), we can choose:

$$\Psi_{\text{SYMM}}(\mathbf{x}) = \sum_{g\in\mathcal{G}_{\text{point}}} \Psi_{\text{CNN}}\big(T_g \cdot \mathbf{x}\big), \tag{5.99}$$

where $\mathcal{G}_{\text{point}}$ is the group of *point group symmetries* of a system and Ψ_{CNN} is a convolutional neural network. More formally, this generalisation of CNNs to more general symmetry groups bears the name *Group Convolutional Neural Network (GCNN)*[82], although originally designed to be invariant they can can also be modified to transform under an arbitrary irreducible representation of a group G using the projection operator

$$|\Psi_\chi\rangle = \frac{d_\chi}{|G|} \sum_{g\in G} \chi_g^* \cdot g\,|\Psi\rangle\,, \tag{5.100}$$

where g runs over all symmetry operations in G with corresponding characters χ_g[75]. There are many more architectures that can be used to encode relevant physical symmetries, and the interested reader may find more details on the subject of Transformers in VMC following [83] and [84].

5.5.2 Neural Quantum States for Fermionic Systems

So far, we only have considered systems of spins. Though, for many models in Quantum Chemistry, one often needs to consider fermionic systems to analyse molecules, proteins, ligands, etc. However for fermions, we know from the Pauli exclusion principle that the many-body wavefunction is antisymmetric, i.e. exchanging two fermions in the wavefunction Ψ produces a negative sign, where for an N-fermionic system: $\Psi(\mathbf{x}_1, \ldots, \mathbf{x}_i, \ldots, \mathbf{x}_j, \ldots, \mathbf{x}_N) = -\Psi(\mathbf{x}_1, \ldots, \mathbf{x}_j, \ldots, \mathbf{x}_i, \ldots, \mathbf{x}_N)$. The most simplest parametrisation for a per-

mutationally antisymmetric wavefunction is given by the so-called *Slater determinant*:

$$\Psi(\mathbf{x}_1,\dots,\mathbf{x}_N) = \det\begin{pmatrix} \phi_1(\mathbf{x}_1) & \dots & \phi_1(\mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ \phi_N(\mathbf{x}_1) & \dots & \phi_N(\mathbf{x}_N) \end{pmatrix} = \langle \mathbf{x}_1 \dots \mathbf{x}_N | \phi_{\text{Slater}} \rangle \,, \tag{5.101}$$

where the functions $\{\phi_j\}_{j=1}^N$ are single-particle orbitals. We note that this parametrisation does not encode any quantum correlations beyond the enforcement of antisymmetry since it only depends on single-particle orbitals. Hence, we can view a Slater determinant as a Mean-Field parametrisation of a many-body wavefunction. Thus, we are then looking for a way to inject a neural network without breaking the antisymmetry of the Slater determinant and so that we can represent more general quantum states. Now, the intuition for an answer to this problem can be viewed as follows: it is known that we cannot always express the ground-state of a Hamiltonian as a product state; however we can always find a unitary rotation U to a basis of the system such that the ground-state is a separable state. Consider such a rotation U, then we are looking for the case where:

$$\langle \mathbf{x}_1 \dots \mathbf{x}_N | \phi_{\text{Slater}} \rangle = \langle \mathbf{x}_1 \dots \mathbf{x}_N | \, U \, | \phi_{\text{Slater}} \rangle \equiv \big\langle U^{-1}(\mathbf{x}_1 \dots \mathbf{x}_N) \big| \phi_{\text{Slater}} \big\rangle \,. \tag{5.102}$$

However finding such a rotation is a hard problem in general, and as a result it is a perfect candidate to be a neural network. Then, we obtain the same Slater determinant rotated to another basis, such that the action of the rotation on the many-body wavefunction produces a transformation $\mathcal{F}$ of the complete configuration $\mathbf{x} = \mathbf{x}_1 \dots \mathbf{x}_N$, thus giving us:

$$\Psi_U(\mathbf{x}_1,\dots,\mathbf{x}_N) = \det\begin{pmatrix} \phi_1\big(\mathcal{F}_1(\mathbf{x}_1 \dots \mathbf{x}_N)\big) & \dots & \phi_1\big(\mathcal{F}_N(\mathbf{x}_1 \dots \mathbf{x}_N)\big) \\ \vdots & \ddots & \vdots \\ \phi_N\big(\mathcal{F}_1(\mathbf{x}_1 \dots \mathbf{x}_N)\big) & \dots & \phi_N\big(\mathcal{F}_N(\mathbf{x}_1 \dots \mathbf{x}_N)\big) \end{pmatrix}, \tag{5.103}$$

where now instead of a parametrisation solely dependent on single-particle orbitals, the wavefunction depends on all the coordinates in a configuration, and hence generalises the *plain* Mean-Field parametrisation. This approach is named *Slater Backflow* and in practice requires the learning of the *backflow transformation* $\mathcal{F}$, where a complete account of the method is described by B.K. Clark and D. Luo in [85]. A *Neural Network Backflow* uses a Feedforward Neural Network (FNN) to modify the single-particle orbital matrix Φ, thus promoted to a parametrized many-body function $\widetilde{\Phi}$, depending on all the occupation numbers $\{n(\mathbf{x})\}$ and parameters $\boldsymbol{\theta}$, such that:

$$\widetilde{\Phi}_{ij}(\boldsymbol{\theta}) = \Phi_{ij}(\boldsymbol{\theta}) + \Delta_{ij}\big(n(\mathbf{x}); \boldsymbol{\theta}\big). \tag{5.104}$$

Here, Δ corresponds to a correction of the single-particle orbitals Φ, and is parametrised by a FNN with the electronic occupation numbers as input and a many-body correction to the matrix Φ as output. We emphasize once again that this approach allows the orbitals to dynamically change depending on the positions $\{\mathbf{x}\}$ of the electrons, thus allowing one to include true many-body correlations in the antisymmetric part of the wavefunction[86]. Another powerful method to deal with a fermionic framework is named *Hidden Fermions*, for which an account of the method can be found in [87]. There are several studies of fermionic systems that use NQSs where the interested reader may refer, among many others, to [88, 89, 90, 91, 92, 80, 93].

5.5.3 Representation of Tensor Networks with Neural Quantum States

See Appendix (A.2) for a brief description of the Tensor Network (TN) terminology and refer the reader to [94] for an introduction to MPS and TNs. Now, it turns out that NQSs have a direct connection with tensor network representations of quantum states. Indeed, O. Sharir, A. Shashua and G. Carleo in [61] have constructed neural-network layers that efficiently perform tensor contractions. As a result, in the context of many-body quantum states, they established that NQSs have strictly the same or higher expressive power than practical

variational tenor networks. *Expressivity* means that a specific method has the ability to explore and generate different quantum states across the full Hilbert space. In particular, they demonstrate that all matrix product states (MPSs) can be efficiently written as NQSs with a number of edges polynomial in the bond dimension and logarithmic depth in the system size. However, the converse is not true in general, and they show that there exists some states that are not efficiently expressible in terms of MPSs or Projected Entangled Pair States (PEPs), but are efficiently expressible with neural network states. Here, PEPS are a generalization of MPSs to two- or higher-dimensional systems. This yields a picture of the expressive power of classically tractable variational quantum states (see Figure (5.16)[61] below).

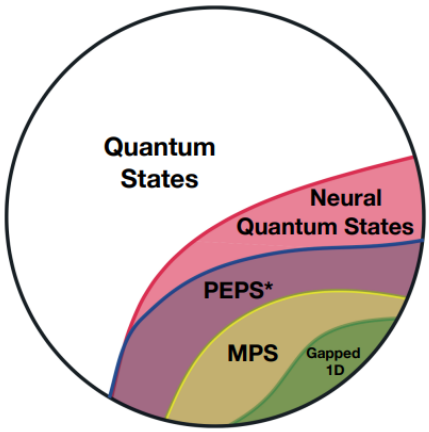


Figure 5.16: Note that PEPS* denotes the Projected Entangled Pair States of bond dimension χ that are exactly or approximately contracted in poly(N, χ) time on a classical computer.

Moreover, a second paper on the subject written by D. Wu, R. Rossi, F. Vicentini and G. Carleo in [95] shows that any MPS can be exactly represented by a recurrent neural network (RNN) with a linear memory update. In particular, numerical implementations indicate that it can encode wavefunctions using a bond dimension lower by orders of magnitude when compared to MPS, with an increase in the accuracy by tuning the bond dimension. These results suggest that Neural Quantum States are more versatile decompositions compared to general tensor networks for the representation of many-body quantum states.

5.5.4 Neural Quantum States for Open Quantum Systems

We end the extensions of Neural Quantum States with a completely different framework, where now we consider open quantum systems, i.e. systems that interact with an external environment causing an exchange of matter or information. This framework is trickier to work with than closed systems due to these interactions. We follow [81] for the description. The state of an open quantum system is described by its density operator $\hat{\rho}$ whose dynamics for Markovian systems is described by a *Linblad Master equation*:

$$\frac{d}{dt}\hat{\rho} = -i[\hat{H}, \hat{\rho}] + \sum_j \gamma_j \left(\hat{L}_j^\dagger \hat{\rho} \hat{L}_j - \frac{1}{2}\{\hat{L}_j^\dagger \hat{L}_j, \hat{\rho}\} \right), \tag{5.105}$$

where $[A, B] = AB - BA$ denotes the commutator and $\{A, B\} = AB + BA$ denotes the anticommutator of operators A and B. Moreover, $\hat{H}$ is the Hamiltonian of the system, $\{\gamma_j\}_j$ are non-negative real coefficients named *damping rates*, and $\{\hat{L}_j\}_j$ are the so-called *Linblad operators*. Conceptually, they are also named *jump operators* since these operators will produce jumps between states and thus modify the internal quantum numbers of the system. The first term in Eq. (5.105) describes the unitary dynamics of the system given by $\hat{H}$, while the second term describes non-unitary dynamics due to the dissipation of the environment with strength γ. In particular, in the second bloc, terms of the form $\hat{L}_j^\dagger \hat{\rho} \hat{L}_j$ account for the possible quantum jumps, while the anticommutator term is needed to normalise the case in which no jumps occur. We note that Eq. (5.105) can be re-written in a

superoperator form such that

$$\frac{d}{dt}\hat{\rho} = \hat{\mathcal{L}}(\hat{\rho}), \tag{5.106}$$

with $\hat{\mathcal{L}}$ the so-called *Liouvillian superoperator*, since the above equation resembles the classical Liouville equation $\partial_t \hat{\rho} = \{\hat{H}, \hat{\rho}\}$. The fact that the state of a quantum system is dictated by a density operator makes the simulation of open systems even more challenging than for closed systems since for density matrices, the curse of dimensionality is even more pronounced as for wavefunctions, e.g. for a system of N spin-1/2 particles, the number of coefficients to parametrise $\hat{\rho}$ scales as 4^N coefficients[96]. In general, it is common to use the superoperator formulation of the Linblad Master equation, Eq. (5.106), to determine the solution $\hat{\rho}$ using NQSs. We thus need a neural representation of the density operator. We now mention one of many possibilities to simulate open systems with Neural Quantum States, by using a Positive Operator Valued Measures (POVMs) approach. As described in [97], the density matrix is represented by a probability distribution over measurement outcomes $\{\alpha\}$ of an Informationally Complete (IC) set of measurement operators $\mathbf{M} := \{\hat{M}_\alpha\}_\alpha$ given by the Born rule:

$$\mathbb{P}_{\boldsymbol{\theta}}(\alpha) = \mathrm{Tr}\big(\hat{M}_\alpha \hat{\rho}_{\mathrm{POVM}}\big). \tag{5.107}$$

Here, POVMs $\mathbf{M} := \{\hat{M}_\alpha\}_\alpha$ describe the most general type of measurements allowed by Quantum Theory beyond the notion of projective measurements, while Information Completeness (IC) dictates the fact that measurement statistics contain all of the information about the state and thus specifies it univocally[97]. As a result, this leads to the representation:

$$\hat{\rho}_{\mathrm{POVM}} = \sum_{\alpha,\alpha'} \mathbb{P}_{\boldsymbol{\theta}}(\alpha) T^{-1}_{\alpha,\alpha'} \hat{M}_{\alpha'}, \tag{5.108}$$

with an overlap matrix $(T_{\alpha,\alpha'})_{\alpha,\alpha'} = \big(\mathrm{Tr}\big[\hat{M}_\alpha \hat{M}_{\alpha'}\big]\big)_{\alpha,\alpha'}$ and measurement operators $\{\hat{M}\}$. Thus the advantage of an IC-POVM representation such as Eq. (5.108) is that only the positive amplitudes $\{\mathbb{P}_{\boldsymbol{\theta}}(\alpha)\}_\alpha$ have to be modelled by a neural network. Although this is a powerful method for the problem at hand we might still have some issues, for instance from the fact that the density operator $\hat{\rho}_{\mathrm{POVM}}$ is not always positive-definite. Other approaches are also possible, where the density operator is expressed in terms of Restricted Boltzmann Machines (RBMs)[98], Convolutional Neural Networks (CNNs)[99,100] or also as autoregressive networks[101]. Finally, as for Neural Quantum States in closed systems, one can define stochastic sampling procedures and a Stochastic Reconfiguration (SR) method for a general Liouvillian $\hat{\mathcal{L}}$ (see [101] and [98] for the details).

A | Further Topics

This appendix is devoted to some topics that I started investigating but never decided to push further, either from time constraints or from a lack of vision on my part. Nonetheless, I reference them here to highlight their existence. I also provide some details on the construction of the Rayleigh quotient for the energy of a quantum system.

A.1 Rayleigh Quotient for Energy Minimization

In the sequel, we have used several times a Rayleigh quotient to define the energy of a quantum system. This first appendix is dedicated to the derivation of this form for the energy. For a complex Hermitian matrix H and non-zero vector $\boldsymbol{x}$, the Rayleigh quotient is defined as

$$R(H, \boldsymbol{x}) := \frac{\boldsymbol{x}^* H \boldsymbol{x}}{\boldsymbol{x}^* \boldsymbol{x}} \equiv \frac{\sum_{j=1}^N \lambda_j \boldsymbol{y}_j^2}{\sum_{j=1}^N \boldsymbol{y}_j^2}, \tag{A.1}$$

where $(\lambda_j, \boldsymbol{v}_j)$ is the j-th eigenpair and $\boldsymbol{y}_j = \boldsymbol{v}_j^* \boldsymbol{x}$ is the j-th coordinate of $\boldsymbol{x}$ in the eigenbasis. The fact that the quotient is a weighted average of the eigenvalues can be used to identify the eigenvalues (in decreasing order) $\lambda_{\max} := \lambda_1 \geq \lambda_2 \geq ... \geq \lambda_N =: \lambda_{\min}$, and in general it is clear that $R(H, \boldsymbol{x}) \in [\lambda_{\min}, \lambda_{\max}]$ (Rayleigh-Ritz principle). Let us then see how we can use the Rayleigh quotient for energy minimization from the point of view of Lagrange multipliers. We seek to minimise the energy of H (i.e. find the critical points) given by $E_\Psi := \langle \Psi | H | \Psi \rangle$ subject to a normalization constraint of the quantum states $\langle \Psi | \Psi \rangle = 1$. In others words, we need the critical points of the Lagrangian $\mathcal{L}(\Psi) = \langle \Psi | H | \Psi \rangle - \lambda(\langle \Psi | \Psi \rangle - 1)$, for a Lagrange multiplier λ. It is not difficult to see that, in order for $\partial_\Psi \mathcal{L}(\Psi) = 0$ we require $H | \Psi \rangle = \lambda | \Psi \rangle$. Furthermore, in such a case,

$$R(H, \Psi) := \frac{\langle \Psi | H | \Psi \rangle}{\langle \Psi | \Psi \rangle} = \lambda. \tag{A.2}$$

As a result, the eigenvectors of H are the critical points of the Rayleigh quotient and their corresponding eigenvalues are the stationary values of $\mathcal{L}$, hence giving the energies of the system.

A.2 Density Matrix Embedding Theory (DMET)

In this appendix, we aim to present a method for molecular fragmentation named *Density Matrix Embedding Theory (DMET)*. In order to fragment the molecule into multiple subsystems, DMET employs a self-consistent construction of smaller and/or less complex many-body problems by leveraging an approximate, computationally inexpensive (*low-level*) descriptor of the original system which are then solved with highly accurate (*high-level*) methods[102]. These multiple fragments are not disjoint from the original system, rather they are still entangled to the system of the molecule. Then, an interesting idea could be to use SQD in each fragment so as to find the lowest energy and the corresponding ground-state. We can then recover the total information for the molecule by putting all the fragments back together. In our case, we will use a one-shot DMET procedure. One-shot DMET is the fragmentation of a system into multiple fragments where each fragment has its own bath and is

treated with its own fragment solver. In other words in the multi-fragment formulation of DMET, the total system is separated into N_{frag} fragments such that each fragment is treated as its own embedding problem where the orbitals that span the fragment are defined as a subsystem and the rest of the system is set to be the environment[103]. Now, let us analyse one of the fragments and suppose that the system is separated into a fragment along with an environment such that the global Hilbert space yields $\mathcal{H} = \mathcal{H}_f \otimes \mathcal{H}_e$, for $\mathcal{H}_e$ (resp. $\mathcal{H}_f$) the Hilbert space of the environment (resp. fragment). Thus, any state $|\Psi\rangle \in \mathcal{H}$ may be decomposed as

$$|\Psi\rangle = \sum_{i=1}^{\dim(\mathcal{H}_f)} \sum_{j=1}^{\dim(\mathcal{H}_e)} \Psi_{ij} |f_i\rangle \otimes |e_j\rangle , \tag{A.3}$$

for basis states $\{f_i\}_i$ (resp. $\{e_j\}_j$) of the Hilbert space $\mathcal{H}_f$ (resp. $\mathcal{H}_e$). Using a Singular Value Decomposition of the matrix of coefficients Ψ, the state $|\Psi\rangle$ may be written as a Schmidt decomposition

$$|\Psi\rangle = \sum_{p=1}^{\dim(\mathcal{H}_f)} \lambda_p |f_p\rangle \otimes |b_p\rangle , \tag{A.4}$$

such that $\{\lambda_p\}_p$ are the singular values and $\{|b_p\rangle\}_p$ are the so-called bath states. As a result, if $|\Psi\rangle$ is a ground state of $\hat{H}$ in either a low or high level of the theory, then the embedding Hamiltonian $\hat{H}_{\text{emb}} = \hat{P}\hat{H}\hat{P}$ obtained by projecting the original Hamiltonian $\hat{H}$ onto the Schmidt space with operator

$$\hat{P} = \sum_{p,q=1}^{\dim(\mathcal{H}_f)\wedge\dim(\mathcal{H}_e)} |f_p\rangle \langle f_p| \otimes |b_q\rangle \langle b_q| \tag{A.5}$$

will share the same ground state as $\hat{H}$ at the exact same level of the theory[104]. This method introduces a global chemical potential μ which is then optimized in order to ensure that DMET predicts the correct particle number N_{tot}. The Hamiltonian of a fragment plus bath is in the form

$$\hat{H}_{\text{emb}}(\mu) = \hat{H}_{\text{emb}} - \mu \sum_{p\in A} a_p^\dagger a_p \tag{A.6}$$

where A is the set of fragment orbitals, B is the set of bath orbitals, $E = \overline{A \cup B}$ is the set of environment orbitals and Γ_{rs}^{env} is the 1-RDM (Reduced Density Matrix) $\Gamma_{rs} := \langle a_r^\dagger a_s \rangle$ constructed from the fully occupied environment orbitals only. The embedded Hamiltonian $\hat{H}_{\text{emb}}$ is set to be

$$\hat{H}_{\text{emb}} = \sum_{p,q\in A\cup B} \left(h_{pq} + \sum_{r,s\in E} \big(h_{pqrs} - h_{psrq}\big)\Gamma_{rs}^{\text{env}} \right) a_p^\dagger a_q + \frac{1}{2} \sum_{p,q,r,s\in A\cup B} h_{pqrs} a_p^\dagger a_r^\dagger a_s a_q. \tag{A.7}$$

Using an optimization procedure, we need to ensure that the constraint

$$\sum_f \langle \Psi_f(\mu)| \hat{N}_f |\Psi_f(\mu)\rangle = N_{\text{tot}} \tag{A.8}$$

is satisfied. Here $\Psi_f(\mu)$ is the high-level ground state of fragment f calculated using the solver, $\hat{N}_f = \sum_{p\in A_f} a_p^\dagger a_p$ is the particle number operator of fragment f, and N_{tot} is the total number of electrons in the system. Finally, to avoid a double count of the environmental terms we modify $\hat{H}_{\text{emb}}$ slightly, and the total electronic energy is calculated as a sum of the (non-overlapping) fragments using an energy operator $\hat{E}_{\text{emb},f}$ for each fragment:

$$\langle \hat{H} \rangle = \sum_f \langle \Psi_f(\mu)| \hat{E}_{\text{emb},f} |\Psi_f(\mu)\rangle$$

$$= \sum_f \left[\sum_{p \in A_f} \left(\sum_{q \in A_f \cup B} \left(h_{pq} + \frac{1}{2} \sum_{r,s \in E} (h_{pqrs} - h_{psrq}) \Gamma^{\text{env}}_{rs} \right) \langle a_p^\dagger a_q \rangle_f + \frac{1}{2} \sum_{q,r,s \in A_f \cup B} h_{pqrs} \langle a_p^\dagger a_r^\dagger a_s a_q \rangle_f \right) \right],$$

where $\langle a_p^\dagger a_q \rangle_f$ and $\langle a_p^\dagger a_r^\dagger a_s a_q \rangle_f$ are respectively the 1-RDM and 2-RDM constructed from the fragment orbitals[105]. Note in practice, however, that the absence of high symmetry makes the uniqueness of a non-overlapping partition difficult (even impossible in some instances). So, different partitions often lead to different results and there is no apparent way to evaluate the quality of those different choices[104].

In the above construction, a crucial step is the computation of the ground state energy in a specific fragment f. In the early stages of this theory, a natural approach via a VQE has been made which is a practical method for relatively small fragments (e.g. [106]). However on can generalise the fragmentation procedure to other types of solvers subject to the size of the corresponding fragments.

A.3 Tensor Decompositions in a Nutshell

A **tensor** is simply an ordered collection of elements such that its *dimension* specifies the size of the collection and its *order* represents the number of different dimensions. In this chapter, specifying an index along each tensor *mode* (i.e. mode-1 is the column direction, mode-2 is the row direction, ...) defines an element of the tensor. Common examples of tensors are scalars (order 0), vectors (order 1) and matrices (order 2). We will denote a general tensor of order n with dimension $s_1 \times \ldots \times s_n$ by $\boldsymbol{\mathcal{T}} \in \mathbb{R}^{s_1 \times \ldots \times s_n}$ with elements $t_{i_1 \ldots i_n}$ for $i_1 \in \{1, \ldots, s_1\}$ up to $i_n \in \{1, \ldots, s_n\}$. For completeness, we recall that *folding* a tensor yields a higher-order tensor with the same elements, while *unfolding* a tensor yields lower-order tensor with the same elements. Important unfolding examples, especially relevant in quantum mechanics, are the concepts of vectorization and matricization. For a tensor $\boldsymbol{\mathcal{T}} \in \mathbb{R}^{s_1 \times s_2 \times s_3}$ of order 3, vectorization produces an unfolding to a tensor of order 1 (i.e. a vector) $v = \text{vec}(\boldsymbol{\mathcal{T}}) \in \mathbb{R}^{s_1 s_2 s_3}$ with $t_{ijk} = v_{i+(j-1)s_1+(k-1)s_1 s_2}$, while matricizations of the tensor produces a set of new tensors $\boldsymbol{T}_{(1)} \in \mathbb{R}^{s_1 \times s_2 s_3}$, $\boldsymbol{T}_{(2)} \in \mathbb{R}^{s_2 \times s_1 s_3}$ and $\boldsymbol{T}_{(3)} \in \mathbb{R}^{s_3 \times s_1 s_2}$. Now that we have seen how to reshape a tensor, we still need to investigate the notion of multiplication of tensors. We can encounter such operations in inner products, outer products, tensor products, Hadamard products or matrix multiplication. This task is defined as a *tensor contraction*, where these operations can always be reduced to product of matrices and/or vectors by transposing modes and matricizing both operands, then folding and transposing the product. Thus, for a tensor $\boldsymbol{\mathcal{U}}$ of order $s+v$ and another tensor $\boldsymbol{\mathcal{V}}$ of order $v+t$ we have the tensor contraction $\boldsymbol{\mathcal{W}}$ over v modes given by

$$w_{i_1 \ldots i_s j_1 \ldots j_t} = \sum_q u_{i_1 \ldots i_s k_1 \ldots k_v} v_{k_1 \ldots k_v j_1 \ldots j_t}. \tag{A.9}$$

We note that other contractions may be mapped to this form after a transposition, and also that if all the tensor modes are of dimension n we recover matrix multiplication $\mathbf{C} = \mathbf{AB}$ such that $\mathbf{C} \in \mathbb{R}^{n^s \times n^t}$, $\mathbf{A} \in \mathbb{R}^{n^s \times n^v}$ and $\mathbf{B} \in \mathbb{R}^{n^v \times n^t}$.

As a close concept to tensor contraction, *tensor decomposition* expresses a tensor as a contraction of factors. Although these decompositions have many applications, we will use them to provide a mechanism to approximate tensor datasets with a smaller number of degrees of freedom. Then, a **tensor network** is a diagram which tells us how to combine several small tensors into a single larger composite tensor. One of these decompositions is the so-called **tensor train decomposition** for which the aim is to express a tensor of order d as a chain of products of order 2 or order 3 tensors. A general tensor train decomposition is given by

$$t_{i_1 \ldots i_d} = \sum_{r_1=1}^{R_1} \cdots \sum_{r_{d-1}=1}^{R_{d-1}} u^{(1)}_{i_1 r_1} \left(\prod_{j=2}^{d-1} u^{(j)}_{r_{j-1} i_j r_j} \right) u^{(d)}_{r_{d-1} i_d}, \tag{A.10}$$

such that the Tucker ranks $(R_1, \ldots, R_{d-1})$ are all bounded by the respective tensor dimensions. In quantum physics, tensor train decompositions are usually known as **matrix product states (MPS)** since they can be

written in the following form:

$$t_{i_1\dots i_d} = \left\langle \mathbf{u}_{i_1}^{(1)}, \mathbf{U}_{i_2}^{(2)} \dots \mathbf{U}_{i_{d-1}}^{(d-1)} \mathbf{u}_{i_d}^{(d)} \right\rangle. \tag{A.11}$$

We finally note the following important remarks (assume wlog a tensor $\boldsymbol{\mathcal{T}} \in \mathbb{R}^{n\times\dots\times n}$ of order d):
(i) Tensor trains are useful whenever the tensor is (approximately) low-rank (i.e. $R_j R_{j+1} < n^{d-1}$ for all $j < d-1$).
(ii) The MPS is usually optimized relative to an implicit operator referred to as the *matrix product operator*.
(iii) The tensor train decomposition is not unique and approximations with MPS is NP hard.
(iv) If an exact decomposition exists, it can be computed by *tensor train SVD (TTSVD)* by performing $d-1$ singular value decompositions (SVDs). This procedure can be accomplished with a cost $\mathcal{O}(n^{d+1})$ (reducible to $\mathcal{O}(n^d R)$ via faster low-rank SVD).
(v) Iterative (alternating) optimization is generally used when optimizing tensor train relative to an implicit operator or to refine TTSVD.
Let us finally end this section by defining the notion of a *matrix product operator* (MPO), which is just a tensor train decomposition where every factor tensor is assigned a pair of modes of the original tensor. For instance, if we consider a Hamiltonian H of the form $H = \sum_{j=1}^{N} \mathbb{I}^{\otimes(j-1)} \otimes A_j \otimes \mathbb{I}^{\otimes(N-j)}$ for some local operators $\{A_j\}_{j=1}^N$, then a MPO for this Hamiltonian can be formed by the set of unfolding matrices:

$$U_1^{(H)} = \begin{pmatrix} A_1 & \mathbb{I} & \dots & \mathbb{I} \end{pmatrix}_{1\times N}, \quad V_j^{(H)} = \operatorname{diag}\big(\mathbb{I}, \dots, \mathbb{I}, A_j, \mathbb{I}, \dots, \mathbb{I}\big)_{N\times N}, \quad W_N^{(H)} = \begin{pmatrix} \mathbb{I} & \dots & \mathbb{I} & A_N \end{pmatrix}_{1\times N}^{T}. \tag{A.12}$$

We note that in the construction, the structure of an MPO is similar to the one of an MPS except that MPS has a single physical index per site while MPO has two, where the top one acts on kets and the bottom one on bras. We refer to [107] for a general theory of Tensor Networks (MPS, PEPS), and for instance to [108, 109, 110, 111] for applications of TNs in Quantum Chemistry (DMRG, MPO Hamiltonian, MPOs for time-evolution).

A.4 An Automatic Reduction in the Space of Active Orbitals

Recall that in order to describe a molecule in second quantisation, we have encoded the information of the Slater determinant through the occupation of the spin-orbitals (see Section (1.2)). Although this is a convenient representation on a practical perspective, in practice however this data can become computationally expensive. Indeed for a spin system with a number N_{orb} of molecular orbitals (i.e. combination of atomic orbitals during the formation of a molecule), the corresponding number of qubits N_{q} is given by $N_{\text{q}} = 2N_{\text{orb}}$. As a result, for a water molecule such as H_2O already requires 14 qubits and on the other hand carbon dioxide CO_2 demands 30 qubits which, in this latter case, is already posing some difficulties on current hardware. Thus, in order to make molecular simulation possible, a common practice is to restrict the space of orbitals to a smaller region known as the **active space**. In practice, however, active space selection is a delicate task that is usually done manually and is based on a subjective vision of the problem that will likely differ due to different chemical interpretations. A common, relatively simple choice, is to select the active space around the highest occupied molecular orbital (HOMO) and the lowest unoccupied molecular orbital (LUMO) so that the space is given by the energy gap $[\text{LUMO} - \alpha, \text{HOMO} + \alpha]$ for some natural number α (relevant for static correlations). In general, it is desirable to consider an active space as compact as possible and including all statistically correlated orbitals to clearly discriminate between static and dynamic correlations[112]. At a heuristic level, one usually states that in systems with (strong) static correlation the wavefunction differs qualitatively from the reference Slater determinant, while strong dynamic correlation gives a wavefunction including a large number of excited determinants, all with comparable, small occupations[113]. The importance of considering static correlation is that it is present in the transition states of chemical reactions which involve breaking or forming bonds, and also in most excited states (e.g. in atomic systems with small HOMO-LUMO gap)[112]. The aim of this section is to present an automated method for active space selection based on a quantum information measure, the **orbital entanglement entropy**. This will allow for an approximate but fast calculation of a large active space of

orbitals chosen around the Fermi level (i.e. midpoint of the HOMO-LUMO gap) for the identification of highly entangled orbitals.

As a first approach, we introduce a multi-configurational wavefunction based method tailored to recover static correlation, the **Complete Active Space** (CAS), which is often combined with a simultaneous Self-Consistent Field (SCF) optimization of the orbital basis, yielding the CASSCF method. A CAS classifies orbitals in three classes: *core* orbitals always holding two electrons, *active* orbitals being partially occupied orbitals, and *virtual* orbitals that hold no electrons. Note that we will denote by CAS(N, M) the Complete Active Space such that N specifies the number of active electrons and M the number of active orbitals. The key idea of a CASSCF approach is the selection of CAS(N, M) for a which an approximation of the wavefunction is given by the full configuration interaction (FCI) expansion of the wavefunction

$$|\Psi_{\text{CAS-CI}}\rangle = \sum_{n_1,\ldots,n_M} C_{n_1\ldots n_M} |n_1 \ldots n_M\rangle , \tag{A.13}$$

where $|n_1 \ldots n_M\rangle$ is an occupation number vector corresponding to an orthonormal basis state (e.g. a Slater determinant)[114]. However, in addition to the fact that we only obtain an approximation of the wavefunction, this FCI expansion scales exponentially with respect to the number of active electrons and orbitals with a current limit at a CAS size of CAS$(18, 18)$[115]. To improve on this method, we turn to the development of the Density Matrix Renormalization Group (DMRG) algorithm that provides the advantage of having an iterative convergence to the exact solution in a given active space with polynomial cost rather than exponential[116], also allowing for complete active spaces of up to 100 orbitals. As described in [114], DMRG was introduced by S.R. White[117,118] proposing to select states according to their weight in the density matrix, which improved the performance of the renormalization group. As presented above, although DMRG is known to be the most successful numerical method to solve one- or quasi-one-dimensional systems, the higher performance coming from the truncation of the density matrix by only retaining the m states with the highest weight comes at the price of assessing the accuracy of the approximation *a posteriori*. This can be achieved by analyzing the weights of the discarded density matrix eigenstates and by performing calculations with different values of m (that may be subject to an extrapolation towards $m \to \infty$)[114]. The core principle of (second-generation) DMRG[114] is the variational optimization of a class of *ansatz* states called **matrix product states** (MPSs),

$$|\Psi_{\text{MPS}}\rangle = \sum_{n_1,\ldots,n_M} M^{n_1} \ldots M^{n_M} |n_1 \ldots n_M\rangle , \tag{A.14}$$

where $\{M^{n_j}\}_{j=1}^{M}$ are matrices for the j-th orbital whose product yields the CI coefficient $C_{n_1\ldots n_M}$ and $\{n_j\}_{j=1}^{M} = \{|\uparrow\downarrow\rangle , |\uparrow\rangle , |\downarrow\rangle , |0\rangle\}_{j=1}^{M}$. Of course, it must be understood that the DMRG algorithm attempts to variationally find the MPS wavefunction (A.14) of the system. Note that, given an approximation $|\tilde{\Psi}\rangle$ of the true wavefunction $|\Psi\rangle$, we can quantify the accuracy of the approximation method by evaluating a truncation error[119]

$$\epsilon(m) := \big\| \, |\Psi\rangle - |\tilde{\Psi}\rangle \, \big\|^2 = 1 - \sum_{\alpha=1}^{m} \omega_\alpha, \tag{A.15}$$

where m corresponds to the number of renormalized states (i.e. the maximum size of the wavefunction expansion) and $\{\omega_\alpha\}_\alpha$ are the eigenvalues of the reduced density matrix of the active subsystem. Now that we have determined a method to approximate the wavefunction of the system, we still need to define a (low computational cost) measure to select specific orbitals in our final DMRG (or CASSCF) expansion. To achieve this, [9] uses orbital entanglement entropy[120] as the measure quantifying the importance of a given orbital in the final approximated wavefunction for the description of static correlations. The single-orbital entropy $S_i^{(1)}$ is defined as the Von-Neumann entropy of the density matrix ρ_i of the system, quantifies the deviation from a pure state

measured by the eigenvalues $\{\omega_{k,i}\}_k$ of a grand-canonical one-orbital reduced density matrix (1o-RDM)[10],

$$S_i^{(1)} := -\mathrm{Tr}(\rho_i \ln(\rho_i)) = -\sum_{k=1}^{4} \omega_{k,i} \ln(\omega_{k,i}), \tag{A.16}$$

such that the four states of orbital i belongs to the collection $\{|\uparrow\downarrow\rangle, |\uparrow\rangle, |\downarrow\rangle, |0\rangle\}$. In a similar way, the two-orbital entropies $S_{ij}^{(2)}$ is defined as the Von-Neumann entropy of the two-orbital reduced density matrix ρ_{ij} (2o-RDM),

$$S_{ij}^{(2)} := -\mathrm{Tr}(\rho_{ij} \ln(\rho_{ij})) = -\sum_{k=1}^{16} \eta_{k,ij} \ln(\eta_{k,ij}), \tag{A.17}$$

such that $\{\eta_{k,ij}\}_k$ are the eigenvalues of the 2o-RDM with all 16 combinations of occupation/spin. The two-orbital entropy of an orbital pair describes by how much the occupation of states defined on this orbital pair are affected by the presence of all other orbitals[10]. For reference, we note that the one- and two-orbital entropies may be combined to give the orbital mutual information[121] I_{ij} which measures the degree of entanglement between the two orbitals (i, j),

$$I_{ij} = \frac{1}{2}\left(S_i^{(1)} + S_j^{(1)} - S_{ij}^{(2)}\right)(1 - \delta_{ij}). \tag{A.18}$$

Finally, to set a meaningful algorithmic measure, we may define a multi-configurational diagnostic $Z_{S^{(1)}}$ solely based on the one-orbital entropy giving us the indications $Z_{S^{(1)}} = 0$ implies no entanglement (i.e. selection of all orbitals for final converged calculation) and $Z_{S^{(1)}} = 1$ implies full entanglement (i.e. no orbitals for final converged calculation). Such a diagnostic may be given by

$$Z_{S^{(1)}} = \frac{1}{M \ln(4)} \sum_{i=1}^{M} S_i^{(1)}, \tag{A.19}$$

where M corresponds to the number of orbitals considered for the evaluation of $Z_{S^{1)}}$. This measure is well defined in the sense that maximum entanglement corresponds to the situation where all occupations are equally likely (i.e. $\omega_{k,i} = 0.25$ for all k) and is therefore equal to $\ln(4) \approx 1.4$ distributed over the M orbitals[122]. Let us finally present the full algorithm by first presenting its workflow and then proceed by providing the key ideas, where one may see [10] for the full details.

We first generate initial orbitals of the desired molecule, where an initial guess for the first DMRG calculation with a very large active space is required until one sweep is completed. For AutoCAS with more than 30 orbitals, it is advised to make a guess with CI-DEAS[123] and $m = 500$ as it was argued that the entanglement measures from this method were hardly distinguishable from actual converged results[9]. The goal of this step is to include highly entangled orbitals from the very beginning of the algorithm, then expanding the active space dynamically, in order for DMRG to not get stuck in any local minima and also to avoid a potentially slow convergence in the subsequent steps[120]. Once the initial step is completed, the entanglement measures are calculated and a global threshold has to be appointed. A possible choice is to appoint a threshold based on a fraction of the theoretical single-orbital entropy maximum in the form $\rho_{\mathrm{thr}} := c \cdot \max_i S_i^{(1)}$ for some $c \in [0, 1]$. In [9], after analysis of several molecules, they concluded on a threshold with $c = 0.1$. In the first iteration of DMRG, the maximal theoretical value of the single-orbital entropy $\left(S^{(1)}\right)_{\max} = \ln(4) \approx 1.4$ is chosen. In all subsequent iterations however, to account for the varying degree of orbital entanglement in different molecules[9], the selection criteria is made with respect to the current calculation rather than the theoretical maximum. We thus consider a conservative threshold of $\rho_{\mathrm{thr}}^{(1)} = 0.14$ for the first iteration and $\rho_{\mathrm{thr}}^{(n)} = 0.1 \cdot \max_i S_i^{(1)}$ for iterations $n \geq 2$. The first threshold $\rho_{\mathrm{thr}}^{(1)}$ has the important task of deciding whether a molecule requires a multi-configuration setting (i.e. description of the molecule using multiple configurations/Slater determinants) or a single-configuration method (e.g. Hartree-Fock, Density Functional Theory). The wavefunction will be of multi-configuration character if at least one single-orbital entropy is greater than $\rho_{\mathrm{thr}}^{(1)}$. After this first step,

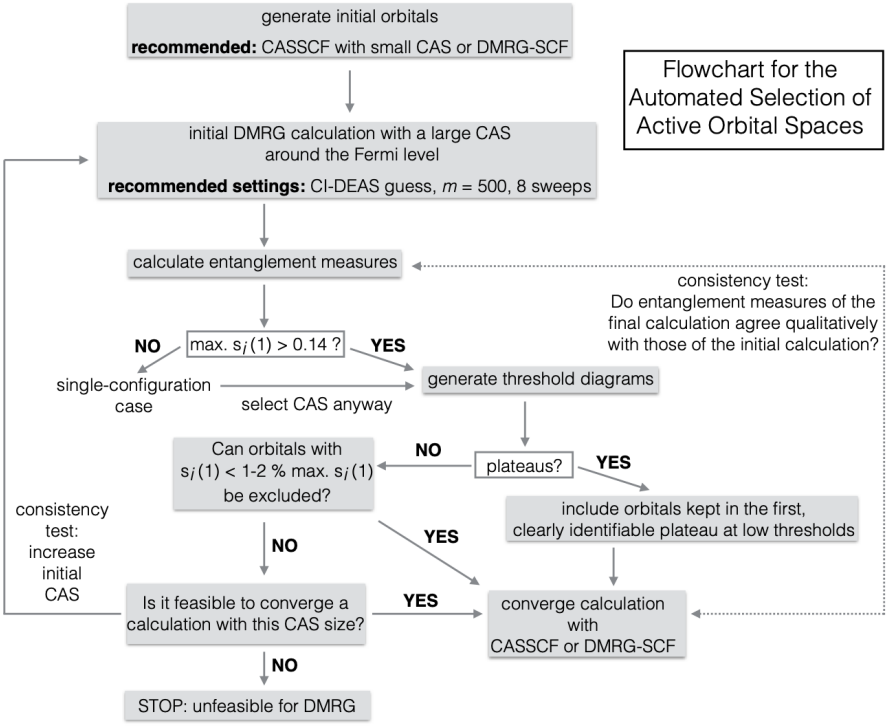


Figure A.1: Flowchart for the Automated Selection of Active Orbital Spaces (AutoCAS)[9,10].

one generates threshold diagrams and investigates for the presence of plateaus. In these diagrams, the x-axis is dedicated to the orbital discarding threshold which is varied from $0-100\%$ of the largest single-orbital entropy (or mutual information matrix) $\max_i S_i^{(1)}$, while the y-axis represents the number of selected orbitals according to a specific threshold. Since these diagrams contain in general one or more plateaus, a threshold of 0% leads to a selection of all the orbitals while a threshold of 100% does not select any orbitals for the final converged calculation. In other words, if we observe plateaus then the corresponding highly entangled orbitals will be identified and included in the final active space, whilst no plateau structure implies that we exclude orbitals with a single-orbital entropy lower than $1-2\%$ of the maximal single-orbital entropy[9]. In the event where all orbitals are selected, we produce a larger CAS in the initial DMRG calculation to know if more orbitals are required for the final active space. A final comparison of the entanglement information for the output converged complete active space with the initial calculation is made in order to assess the quality of the selection procedure. Note that AutoCAS (by the *Reiher Research Group*, ETH Zürich) is implemented with QCMaquis[124] DMRG program[110] and the quantum chemistry software package openMolcas[125,126,127].

References

[1] James C. L. Chow. "Quantum Computing in Medicine". In: *Medical Sciences* 12.4 (2024), p. 67. DOI: 10.3390/medsci12040067. URL: https://doi.org/10.3390/medsci12040067.

[2] P A M Casares, Roberto Campos, and M A Martin-Delgado. "QFold: quantum walks and deep learning to solve protein folding". In: *Quantum Science and Technology* 7.2 (Mar. 2022), p. 025013. ISSN: 2058-9565. DOI: 10.1088/2058-9565/ac4f2f. URL: http://dx.doi.org/10.1088/2058-9565/ac4f2f.

[3] Aaron Mark Thomas et al. *QCA-MolGAN: Quantum Circuit Associative Molecular GAN with Multi-Agent Reinforcement Learning*. 2025. arXiv: 2509.05051 [quant-ph]. URL: https://arxiv.org/abs/2509.05051.

[4] Josh J. M. Kirsopp et al. "Quantum computational quantification of protein–ligand interactions". In: *International Journal of Quantum Chemistry* 122.22 (Aug. 2022). ISSN: 1097-461X. DOI: 10.1002/qua.26975. URL: http://dx.doi.org/10.1002/qua.26975.

[5] Trygve Helgaker, Poul Jørgensen, and Jeppe Olsen. *Molecular Electronic-Structure Theory*. John Wiley Sons, Ltd, 2000. ISBN: 9781119019572. DOI: https://doi.org/10.1002/9781119019572.ch2. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119019572.ch2.

[6] Sam McArdle et al. "Quantum computational chemistry". In: *Reviews of Modern Physics* 92.1 (Mar. 2020). ISSN: 1539-0756. DOI: 10.1103/revmodphys.92.015003. URL: http://dx.doi.org/10.1103/RevModPhys.92.015003.

[7] E. U. Condon and G. H. Shortley. *The Theory of Atomic Spectra*. Cambridge University Press, 1967.

[8] Björn O. Roos, Peter R. Taylor, and Per E.M. Sigbahn. "A complete active space SCF method (CASSCF) using a density matrix formulated super-CI approach". In: *Chemical Physics* 48.2 (1980), pp. 157–173. ISSN: 0301-0104. DOI: https://doi.org/10.1016/0301-0104(80)80045-0. URL: https://www.sciencedirect.com/science/article/pii/0301010480800450.

[9] Christopher J. Stein and Markus Reiher. "Automated Selection of Active Orbital Spaces". In: *Journal of Chemical Theory and Computation* 12.4 (Mar. 2016), pp. 1760–1771. ISSN: 1549-9626. DOI: 10.1021/acs.jctc.6b00156. URL: http://dx.doi.org/10.1021/acs.jctc.6b00156.

[10] Christopher J. Stein and Markus Reiher. "autoCAS: A Program for Fully Automated Multiconfigurational Calculations". In: *Journal of Computational Chemistry* 40.25 (2019), pp. 2216–2226. DOI: https://doi.org/10.1002/jcc.25869. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/jcc.25869. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/jcc.25869.

[11] Rodney J. Bartlett, Stanislaw A. Kucharski, and Jozef Noga. "Alternative coupled-cluster ansätze II. The unitary coupled-cluster method". In: *Chemical Physics Letters* 155.1 (1989), pp. 133–140. ISSN: 0009-2614. DOI: https://doi.org/10.1016/S0009-2614(89)87372-5. URL: https://www.sciencedirect.com/science/article/pii/S0009261489873725.

[12] Mark R. Hoffmann and Jack Simons. "A unitary multiconfigurational coupled-cluster method: Theory and applications". In: *The Journal of Chemical Physics* 88.2 (Jan. 1988), pp. 993–1002. ISSN: 0021-9606. DOI: 10.1063/1.454125. eprint: https://pubs.aip.org/aip/jcp/article-pdf/88/2/993/18969241/993_1_online.pdf. URL: https://doi.org/10.1063/1.454125.

[13] Mario Motta et al. *Subspace methods for electronic structure simulations on quantum computers*. 2023. arXiv: 2312.00178 [quant-ph]. URL: https://arxiv.org/abs/2312.00178.

[14] Ian D. Kivlichan et al. "Quantum Simulation of Electronic Structure with Linear Depth and Connectivity". In: *Physical Review Letters* 120.11 (Mar. 2018). ISSN: 1079-7114. DOI: 10.1103/physrevlett.120.110501. URL: http://dx.doi.org/10.1103/PhysRevLett.120.110501.

[15] Guorui Zhu et al. *Quantum Circuit for Non-Unitary Linear Transformation of Basis Sets*. 2025. arXiv: 2502.08962 [quant-ph]. URL: https://arxiv.org/abs/2502.08962.

[16] Frank Arute et al. "Hartree-Fock on a superconducting qubit quantum computer". In: *Science* 369.6507 (Aug. 2020), pp. 1084–1089. ISSN: 1095-9203. DOI: 10.1126/science.abb9811. URL: http://dx.doi.org/10.1126/science.abb9811.

[17] Ruben Van der Stichelen et al. "The role of the parameter landscape in Hartree–Fock quantum computing benchmarks". In: *The Journal of Chemical Physics* 164.7 (Feb. 2026), p. 074110. ISSN: 0021-9606. DOI: 10.1063/5.0301030. eprint: https://pubs.aip.org/aip/jcp/article-pdf/doi/10.1063/5.0301030/20910241/074110_1_5.0301030.pdf. URL: https://doi.org/10.1063/5.0301030.

[18] Gavin E. Crooks. "Gates, States, and Circuits: Notes on the circuit model of quantum computation". In: 2019. URL: https://api.semanticscholar.org/CorpusID:211266793.

[19] Trygve Helgaker, Poul Jørgensen, and Jeppe Olsen. "Orbital Rotations". In: *Molecular Electronic-Structure Theory*. John Wiley Sons, Ltd, 2000. Chap. 3, pp. 80–106. ISBN: 9781119019572. DOI: https://doi.org/10.1002/9781119019572.ch3. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119019572.ch3. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119019572.ch3.

[20] Dave Wecker et al. "Solving strongly correlated electron models on a quantum computer". In: *Physical Review A* 92.6 (Dec. 2015). ISSN: 1094-1622. DOI: 10.1103/physreva.92.062318. URL: http://dx.doi.org/10.1103/PhysRevA.92.062318.

[21] Han Xu, Kazuhiro Seki, and Seiji Yunoki. "Preparing the Gutzwiller wave function for attractive SU(3) fermions on a quantum computer". In: *Physical Review Research* 7.3 (Sept. 2025). ISSN: 2643-1564. DOI: 10.1103/rc6z-837q. URL: http://dx.doi.org/10.1103/rc6z-837q.

[22] Kazuhiro Seki and Seiji Yunoki. "Spatial, spin, and charge symmetry projections for a Fermi-Hubbard model on a quantum computer". In: *Physical Review A* 105.3 (Mar. 2022). ISSN: 2469-9934. DOI: 10.1103/physreva.105.032419. URL: http://dx.doi.org/10.1103/PhysRevA.105.032419.

[23] Zhang Jiang et al. "Quantum Algorithms to Simulate Many-Body Physics of Correlated Fermions". In: *Physical Review Applied* 9.4 (Apr. 2018). ISSN: 2331-7019. DOI: 10.1103/physrevapplied.9.044036. URL: http://dx.doi.org/10.1103/PhysRevApplied.9.044036.

[24] James D. Whitfield, Jacob Biamonte, and Alán Aspuru-Guzik. "Simulation of electronic structure Hamiltonians using quantum computers". In: *Molecular Physics* 109.5 (Mar. 2011), pp. 735–750. ISSN: 1362-3028. DOI: 10.1080/00268976.2011.552441. URL: http://dx.doi.org/10.1080/00268976.2011.552441.

[25] Karl Svozil. *Experimental realization of any discrete operator*. 1996. arXiv: quant-ph/9612010 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9612010.

[26] Adam Holmes et al. "Impact of qubit connectivity on quantum algorithm performance". In: *Quantum Science and Technology* 5.2 (Mar. 2020), p. 025009. ISSN: 2058-9565. DOI: 10.1088/2058-9565/ab73e0. URL: http://dx.doi.org/10.1088/2058-9565/ab73e0.

[27] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Second. Society for Industrial and Applied Mathematics, 2003. DOI: 10.1137/1.9780898718003. eprint: https://epubs.siam.org/doi/pdf/10.1137/1.9780898718003. URL: https://epubs.siam.org/doi/abs/10.1137/1.9780898718003.

[28] Qiaohong Wang et al. *Sample-based quantum diagonalization as parallel fragment solver for the localized active space self-consistent field method*. 2025. arXiv: 2512.14936 [physics.chem-ph]. URL: https://arxiv.org/abs/2512.14936.

[29] Akhil Shajan et al. *Towards quantum-centric simulations of extended molecules: sample-based quantum diagonalization enhanced with density matrix embedding theory*. 2024. arXiv: 2411.09861 [quant-ph]. URL: https://arxiv.org/abs/2411.09861.

[30] Javier Robledo-Moreno et al. "Chemistry beyond the scale of exact diagonalization on a quantum-centric supercomputer". In: *Science Advances* 11.25 (2025). ISSN: 2375-2548. DOI: 10.1126/sciadv.adu9991. URL: http://dx.doi.org/10.1126/sciadv.adu9991.

[31] Giuseppe Carleo and Matthias Troyer. "Solving the quantum many-body problem with artificial neural networks". In: *Science* 355.6325 (Feb. 2017), pp. 602–606. ISSN: 1095-9203. DOI: 10.1126/science.aag2302. URL: http://dx.doi.org/10.1126/science.aag2302.

[32] David Ceperley and Berni Alder. "Quantum Monte Carlo". In: *Science* 231.4738 (1986), pp. 555–560. DOI: 10.1126/science.231.4738.555. eprint: https://www.science.org/doi/pdf/10.1126/science.231.4738.555. URL: https://www.science.org/doi/abs/10.1126/science.231.4738.555.

[33] W. M. C. Foulkes et al. "Quantum Monte Carlo simulations of solids". In: *Rev. Mod. Phys.* 73 (1 Jan. 2001), pp. 33–83. DOI: 10.1103/RevModPhys.73.33. URL: https://link.aps.org/doi/10.1103/RevModPhys.73.33.

[34] J. Carlson et al. "Quantum Monte Carlo methods for nuclear physics". In: *Rev. Mod. Phys.* 87 (3 Sept. 2015), pp. 1067–1118. DOI: 10.1103/RevModPhys.87.1067. URL: https://link.aps.org/doi/10.1103/RevModPhys.87.1067.

[35] Steven R. White. "Density matrix formulation for quantum renormalization groups". In: *Phys. Rev. Lett.* 69 (19 Nov. 1992), pp. 2863–2866. DOI: 10.1103/PhysRevLett.69.2863. URL: https://link.aps.org/doi/10.1103/PhysRevLett.69.2863.

[36] Stefan Rommer and Stellan Östlund. "Class of ansatz wave functions for one-dimensional spin systems and their relation to the density matrix renormalization group". In: *Physical Review B* 55.4 (Jan. 1997), pp. 2164–2181. ISSN: 1095-3795. DOI: 10.1103/physrevb.55.2164. URL: http://dx.doi.org/10.1103/PhysRevB.55.2164.

[37] Ulrich Schollwöck. "The density-matrix renormalization group in the age of matrix product states". In: *Annals of Physics* 326.1 (Jan. 2011), pp. 96–192. ISSN: 0003-4916. DOI: 10.1016/j.aop.2010.09.012. URL: http://dx.doi.org/10.1016/j.aop.2010.09.012.

[38] Román Orús. "A practical introduction to tensor networks: Matrix product states and projected entangled pair states". In: *Annals of Physics* 349 (Oct. 2014), pp. 117–158. ISSN: 0003-4916. DOI: 10.1016/j.aop.2014.06.013. URL: http://dx.doi.org/10.1016/j.aop.2014.06.013.

[39] F. Verstraete, V. Murg, and J.I. Cirac. "Matrix product states, projected entangled pair states, and variational renormalization group methods for quantum spin systems". In: *Advances in Physics* 57.2 (Mar. 2008), pp. 143–224. ISSN: 1460-6976. DOI: 10.1080/14789940801912366. URL: http://dx.doi.org/10.1080/14789940801912366.

[40] W. L. McMillan. "Ground State of Liquid He^4". In: *Phys. Rev.* 138 (2A Apr. 1965), A442–A451. DOI: 10.1103/PhysRev.138.A442. URL: https://link.aps.org/doi/10.1103/PhysRev.138.A442.

[41] Giuseppe Carleo et al. "Localization and Glassy Dynamics Of Many-Body Quantum Systems". In: *Scientific Reports* 2.1 (Feb. 2012). ISSN: 2045-2322. DOI: 10.1038/srep00243. URL: http://dx.doi.org/10.1038/srep00243.

[42] Giuseppe Carleo et al. "Light-cone effect and supersonic correlations in one- and two-dimensional bosonic superfluids". In: *Physical Review A* 89.3 (Mar. 2014). ISSN: 1094-1622. DOI: 10.1103/physreva.89.031602. URL: http://dx.doi.org/10.1103/PhysRevA.89.031602.

[43] Fabio Nobile. *Lecture notes in Stochastic Simulation*. Dec. 2025.

[44] Matthias Troyer and Uwe-Jens Wiese. "Computational Complexity and Fundamental Limitations to Fermionic Quantum Monte Carlo Simulations". In: *Physical Review Letters* 94.17 (May 2005). ISSN: 1079-7114. DOI: 10.1103/physrevlett.94.170201. URL: http://dx.doi.org/10.1103/PhysRevLett.94.170201.

[45] Gaopei Pan and Zi Yang Meng. "The sign problem in quantum Monte Carlo simulations". In: *Encyclopedia of Condensed Matter Physics*. Elsevier, 2024, pp. 879–893. ISBN: 9780323914086. DOI: 10.1016/b978-0-323-90800-9.00095-0. URL: http://dx.doi.org/10.1016/B978-0-323-90800-9.00095-0.

[46] Kenny Choo, Antonio Mezzacapo, and Giuseppe Carleo. "Fermionic neural-network states for ab-initio electronic structure". In: *Nature Communications* 11.1 (May 12, 2020), p. 2368. ISSN: 2041-1723. DOI: 10.1038/s41467-020-15724-9. URL: https://doi.org/10.1038/s41467-020-15724-9.

[47] W. K. Hastings. "Monte Carlo Sampling Methods Using Markov Chains and Their Applications". In: *Biometrika* 57.1 (1970), pp. 97–109. ISSN: 00063444, 14643510. URL: http://www.jstor.org/stable/2334940 (visited on 06/17/2026).

[48] Charles J. Geyer. "Practical Markov Chain Monte Carlo". In: *Statistical Science* 7.4 (1992), pp. 473–483. ISSN: 08834237. URL: http://www.jstor.org/stable/2246094 (visited on 08/10/2026).

[49] Andrew Gelman and Donald B. Rubin. "Inference from Iterative Simulation Using Multiple Sequences". In: *Statistical Science* 7.4 (1992), pp. 457–472. DOI: 10.1214/ss/1177011136. URL: https://doi.org/10.1214/ss/1177011136.

[50] James M. Flegal, Murali Haran, and Galin L. Jones. "Markov Chain Monte Carlo: Can We Trust the Third Significant Figure?" In: *Statistical Science* 23.2 (May 2008). ISSN: 0883-4237. DOI: 10.1214/08-sts257. URL: http://dx.doi.org/10.1214/08-STS257.

[51] Anna Dawid et al. *Machine Learning in Quantum Sciences*. Cambridge University Press, 2025. ISBN: 9781009504935. DOI: 10.1017/9781009504942. URL: http://dx.doi.org/10.1017/9781009504942.

[52] Marvin Minsky and Seymour A. Papert. *Perceptrons: An Introduction to Computational Geometry*. The MIT Press, Sept. 2017. ISBN: 9780262343930. DOI: 10.7551/mitpress/11301.001.0001. URL: https://doi.org/10.7551/mitpress/11301.001.0001.

[53] Ken-Ichi Funahashi. "On the approximate realization of continuous mappings by neural networks". In: *Neural Networks* 2.3 (1989), pp. 183–192. ISSN: 0893-6080. DOI: https://doi.org/10.1016/0893-6080(89)90003-8. URL: https://www.sciencedirect.com/science/article/pii/0893608089900038.

[54] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. "Multilayer feedforward networks are universal approximators". In: *Neural Networks* 2.5 (1989), pp. 359–366. DOI: 10.1016/0893-6080(89)90020-8.

[55] George V. Cybenko. "Approximation by superpositions of a sigmoidal function". In: *Mathematics of Control, Signals and Systems* 2 (1989), pp. 303–314. URL: https://api.semanticscholar.org/CorpusID:3958369.

[56] Moshe Leshno et al. "Original Contribution: Multilayer feedforward networks with a nonpolynomial activation function can approximate any function". In: *Neural Netw.* 6.6 (June 1993), pp. 861–867. ISSN: 0893-6080. DOI: 10.1016/S0893-6080(05)80131-5. URL: https://doi.org/10.1016/S0893-6080(05)80131-5.

[57] Allan Pinkus. "Approximation theory of the MLP model in neural networks". In: *Acta Numerica* 8 (1999), pp. 143–195. URL: https://api.semanticscholar.org/CorpusID:16800260.

[58] A. K. Kolmogorov. "On the Representation of Continuous Functions of Many Variables by Superposition of Continuous Functions of One Variable and Addition". In: *Doklady Akademii Nauk SSSR. Russian Academy of Sciences* 114 (1957), pp. 953–956.

[59] Zhou Lu et al. *The Expressive Power of Neural Networks: A View from the Width*. 2017. arXiv: 1709.02540 [cs.LG]. URL: https://arxiv.org/abs/1709.02540.

[60] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: https://arxiv.org/abs/1412.6980.

[61] Or Sharir, Amnon Shashua, and Giuseppe Carleo. "Neural tensor contractions and the expressive power of deep neural quantum states". In: *Physical Review B* 106.20 (Nov. 2022). ISSN: 2469-9969. DOI: 10.1103/physrevb.106.205136. URL: http://dx.doi.org/10.1103/PhysRevB.106.205136.

[62] F. Verstraete and J. I. Cirac. *Renormalization algorithms for Quantum-Many Body Systems in two and higher dimensions*. 2004. arXiv: cond-mat/0407066 [cond-mat.str-el]. URL: https://arxiv.org/abs/cond-mat/0407066.

[63] Ken Kreutz-Delgado. *The Complex Gradient Operator and the CR-Calculus*. 2009. arXiv: 0906.4835 [math.OC]. URL: https://arxiv.org/abs/0906.4835.

[64] Atilim Gunes Baydin et al. *Automatic differentiation in machine learning: a survey*. 2018. arXiv: 1502.05767 [cs.SC]. URL: https://arxiv.org/abs/1502.05767.

[65] Michael Betancourt. *A Geometric Theory of Higher-Order Automatic Differentiation*. 2018. arXiv: 1812.11592 [stat.CO]. URL: https://arxiv.org/abs/1812.11592.

[66] Sandro Sorella, Michele Casula, and Dario Rocca. "Weak binding between two aromatic rings: Feeling the van der Waals attraction by quantum Monte Carlo methods". In: *The Journal of Chemical Physics* 127.1 (2007). ISSN: 1089-7690. DOI: 10.1063/1.2746035. URL: http://dx.doi.org/10.1063/1.2746035.

[67] Xiao Yuan et al. "Theory of variational quantum simulation". In: *Quantum* 3 (Oct. 2019), p. 191. ISSN: 2521-327X. DOI: 10.22331/q-2019-10-07-191. URL: http://dx.doi.org/10.22331/q-2019-10-07-191.

[68] Chae-Yeun Park and Michael J. Kastoryano. "Geometry of learning neural quantum states". In: *Physical Review Research* 2.2 (May 2020). ISSN: 2643-1564. DOI: 10.1103/physrevresearch.2.023232. URL: http://dx.doi.org/10.1103/PhysRevResearch.2.023232.

[69] James Stokes et al. "Quantum Natural Gradient". In: *Quantum* 4 (May 2020), p. 269. ISSN: 2521-327X. DOI: 10.22331/q-2020-05-25-269. URL: http://dx.doi.org/10.22331/q-2020-05-25-269.

[70] Lucas Hackl et al. "Geometry of variational methods: dynamics of closed quantum systems". In: *SciPost Physics* 9.4 (Oct. 2020). ISSN: 2542-4653. DOI: 10.21468/scipostphys.9.4.048. URL: http://dx.doi.org/10.21468/SciPostPhys.9.4.048.

[71] Alessandro Sinibaldi et al. "Unbiasing time-dependent Variational Monte Carlo by projected quantum evolution". In: *Quantum* 7 (Oct. 2023), p. 1131. ISSN: 2521-327X. DOI: 10.22331/q-2023-10-10-1131. URL: http://dx.doi.org/10.22331/q-2023-10-10-1131.

[72] Filippo Vicentini et al. "NetKet 3: Machine Learning Toolbox for Many-Body Quantum Systems". In: *SciPost Physics Codebases* (Aug. 2022). DOI: 10.21468/scipostphyscodeb.7. URL: http://dx.doi.org/10.21468/SciPostPhysCodeb.7.

[73] David Pfau et al. "Ab initio solution of the many-electron Schrödinger equation with deep neural networks". In: *Physical Review Research* 2.3 (2020). ISSN: 2643-1564. DOI: 10.1103/physrevresearch.2.033429. URL: http://dx.doi.org/10.1103/PhysRevResearch.2.033429.

[74] Ao Chen and Markus Heyl. "Empowering deep neural quantum states through efficient optimization". In: *Nature Physics* 20.9 (2024), pp. 1476–1481. ISSN: 1745-2481. DOI: 10.1038/s41567-024-02566-1. URL: http://dx.doi.org/10.1038/s41567-024-02566-1.

[75] Filippo Vicentini et al. "NetKet 3: Machine Learning Toolbox for Many-Body Quantum Systems". In: *SciPost Phys. Codebases* (2022), p. 7. DOI: 10.21468/SciPostPhysCodeb.7. URL: https://scipost.org/10.21468/SciPostPhysCodeb.7.

[76] Giuseppe Carleo et al. "NetKet: A machine learning toolkit for many-body quantum systems". In: *SoftwareX* 10 (2019), p. 100311. DOI: 10.1016/j.softx.2019.100311. URL: https://www.sciencedirect.com/science/article/pii/S2352711019300974.

[77] James Bradbury et al. *JAX: Composable Transformations of Python+NumPy Programs*. 2018.

[78] Jonathan Heek et al. *Flax: A Neural Network Library and Ecosystem for JAX*. 2024.

[79] Ning Qian. "On the momentum term in gradient descent learning algorithms". In: *Neural Networks* 12.1 (1999), pp. 145–151. ISSN: 0893-6080. DOI: https://doi.org/10.1016/S0893-6080(98)00116-6. URL: https://www.sciencedirect.com/science/article/pii/S0893608098001166.

[80] Or Sharir et al. "Deep Autoregressive Models for the Efficient Variational Simulation of Many-Body Quantum Systems". In: *Physical Review Letters* 124.2 (Jan. 2020). ISSN: 1079-7114. DOI: 10.1103/physrevlett.124.020503. URL: http://dx.doi.org/10.1103/PhysRevLett.124.020503.

[81] Hannah Lange et al. *From Architectures to Applications: A Review of Neural Quantum States*. 2024. arXiv: 2402.09402 [cond-mat.dis-nn]. URL: https://arxiv.org/abs/2402.09402.

[82] Taco S. Cohen and Max Welling. *Group Equivariant Convolutional Networks*. 2016. arXiv: 1602.07576 [cs.LG]. URL: https://arxiv.org/abs/1602.07576.

[83] Luciano Loris Viteritti, Riccardo Rende, and Federico Becca. "Transformer Variational Wave Functions for Frustrated Quantum Spin Systems". In: *Physical Review Letters* 130.23 (2023). ISSN: 1079-7114. DOI: 10.1103/physrevlett.130.236401. URL: http://dx.doi.org/10.1103/PhysRevLett.130.236401.

[84] Riccardo Rende et al. "A simple linear algebra identity to optimize large-scale neural network quantum states". In: *Communications Physics* 7.1 (Aug. 2024). ISSN: 2399-3650. DOI: 10.1038/s42005-024-01732-4. URL: http://dx.doi.org/10.1038/s42005-024-01732-4.

[85] Di Luo and Bryan K. Clark. "Backflow Transformations via Neural Networks for Quantum Many-Body Wave Functions". In: *Physical Review Letters* 122.22 (2019). ISSN: 1079-7114. DOI: 10.1103/physrevlett.122.226401. URL: http://dx.doi.org/10.1103/PhysRevLett.122.226401.

[86] Jan Hermann et al. "Ab initio quantum chemistry with neural-network wavefunctions". In: *Nature Reviews Chemistry* 7.10 (Aug. 2023), pp. 692–709. ISSN: 2397-3358. DOI: 10.1038/s41570-023-00516-8. URL: http://dx.doi.org/10.1038/s41570-023-00516-8.

[87] Javier Robledo Moreno et al. "Fermionic wave functions from neural-network constrained hidden states". In: *Proceedings of the National Academy of Sciences* 119.32 (Aug. 2022). ISSN: 1091-6490. DOI: 10.1073/pnas.2122059119. URL: http://dx.doi.org/10.1073/pnas.2122059119.

[88] P. López Ríos et al. "Inhomogeneous backflow transformations in quantum Monte Carlo calculations". In: *Phys. Rev. E* 74 (6 Dec. 2006), p. 066701. DOI: 10.1103/PhysRevE.74.066701. URL: https://link.aps.org/doi/10.1103/PhysRevE.74.066701.

[89] Ingrid von Glehn, James S. Spencer, and David Pfau. *A Self-Attention Ansatz for Ab-initio Quantum Chemistry*. 2023. arXiv: 2211.13672 [physics.chem-ph]. URL: https://arxiv.org/abs/2211.13672.

[90] James S. Spencer et al. *Better, Faster Fermionic Neural Networks*. 2020. arXiv: 2011.07125 [physics.comp-ph]. URL: https://arxiv.org/abs/2011.07125.

[91] Adam Foster et al. *An ab initio foundation model of wavefunctions that accurately describes chemical bond breaking*. 2025. arXiv: 2506.19960 [physics.chem-ph]. URL: https://arxiv.org/abs/2506.19960.

[92] Thomas D. Barrett, Aleksei Malyshev, and A. I. Lvovsky. *Autoregressive neural-network wavefunctions for ab initio quantum chemistry*. 2022. arXiv: 2109.12606 [physics.chem-ph]. URL: https://arxiv.org/abs/2109.12606.

[93] Tianchen Zhao, James Stokes, and Shravan Veerapaneni. *Scalable neural quantum states architecture for quantum chemistry*. 2022. arXiv: 2208.05637 [physics.chem-ph]. URL: https://arxiv.org/abs/2208.05637.

[94] Grégoire Misguich. *Introduction to matrix-product states and tensor networks*. 2026. arXiv: 2606.24803 [cond-mat.str-el]. URL: https://arxiv.org/abs/2606.24803.

[95] Dian Wu et al. "From tensor-network quantum states to tensorial recurrent neural networks". In: *Physical Review Research* 5.3 (2023). ISSN: 2643-1564. DOI: 10.1103/physrevresearch.5.l032001. URL: http://dx.doi.org/10.1103/PhysRevResearch.5.L032001.

[96] Moritz Reh, Markus Schmitt, and Martin Gärttner. "Time-Dependent Variational Principle for Open Quantum Systems with Artificial Neural Networks". In: *Physical Review Letters* 127.23 (Dec. 2021). ISSN: 1079-7114. DOI: 10.1103/physrevlett.127.230501. URL: http://dx.doi.org/10.1103/PhysRevLett.127.230501.

[97] Juan Carrasquilla et al. "Reconstructing quantum states with generative models". In: *Nature Machine Intelligence* 1.3 (Mar. 2019), pp. 155–161. ISSN: 2522-5839. DOI: 10.1038/s42256-019-0028-1. URL: http://dx.doi.org/10.1038/s42256-019-0028-1.

[98] Michael J. Hartmann and Giuseppe Carleo. "Neural-Network Approach to Dissipative Quantum Many-Body Dynamics". In: *Physical Review Letters* 122.25 (2019). ISSN: 1079-7114. DOI: 10.1103/physrevlett.122.250502. URL: http://dx.doi.org/10.1103/PhysRevLett.122.250502.

[99] Johannes Mellak, Enrico Arrigoni, and Wolfgang von der Linden. "Deep neural networks as variational solutions for correlated open quantum systems". In: *Communications Physics* 7.1 (Aug. 2024). ISSN: 2399-3650. DOI: 10.1038/s42005-024-01757-9. URL: http://dx.doi.org/10.1038/s42005-024-01757-9.

[100] Luis E. Herrera Rodriguez and Alexei A. Kananenka. *Convolutional neural networks for long-time dissipative quantum dynamics*. 2020. arXiv: 2012.11009 [physics.comp-ph]. URL: https://arxiv.org/abs/2012.11009.

[101] Filippo Vicentini, Riccardo Rossi, and Giuseppe Carleo. *Positive-definite parametrization of mixed quantum states with deep neural networks*. 2022. arXiv: 2206.13488 [quant-ph]. URL: https://arxiv.org/abs/2206.13488.

[102] Alicia Negre et al. *New perspectives on Density-Matrix Embedding Theory*. 2025. arXiv: 2503.09881 [cond-mat.str-el]. URL: https://arxiv.org/abs/2503.09881.

[103] Dariia Yehorova and Joshua S. Kretchmer. "A multi-fragment real-time extension of projected density matrix embedding theory: Non-equilibrium electron dynamics in extended systems". In: *The Journal of Chemical Physics* 158.13 (Apr. 2023). ISSN: 1089-7690. DOI: 10.1063/5.0146973. URL: http://dx.doi.org/10.1063/5.0146973.

[104] Hong-Zhou Ye et al. "Incremental embedding: A density matrix embedding scheme for molecules". In: *The Journal of Chemical Physics* 149.19 (Nov. 2018). ISSN: 1089-7690. DOI: 10.1063/1.5053992. URL: http://dx.doi.org/10.1063/1.5053992.

[105] Yukio Kawashima et al. *Optimizing Electronic Structure Simulations on a Trapped-ion Quantum Computer using Problem Decomposition*. 2021. arXiv: 2102.07045 [quant-ph]. URL: https://arxiv.org/abs/2102.07045.

[106] Naoki Iijima et al. *Towards Accurate Quantum Chemical Calculations on Noisy Quantum Computers*. 2023. arXiv: 2311.09634 [quant-ph]. URL: https://arxiv.org/abs/2311.09634.

[107] J. Ignacio Cirac et al. "Matrix product states and projected entangled pair states: Concepts, symmetries, theorems". In: *Reviews of Modern Physics* 93.4 (Dec. 2021). ISSN: 1539-0756. DOI: 10.1103/revmodphys.93.045003. URL: http://dx.doi.org/10.1103/RevModPhys.93.045003.

[108] G. Catarina and Bruno Murta. "Density-matrix renormalization group: a pedagogical introduction". In: *The European Physical Journal B* 96.8 (Aug. 2023). ISSN: 1434-6036. DOI: 10.1140/epjb/s10051-023-00575-2. URL: http://dx.doi.org/10.1140/epjb/s10051-023-00575-2.

[109] Alberto Baiardi and Markus Reiher. "The density matrix renormalization group in chemistry and molecular physics: Recent developments and new challenges". In: *The Journal of Chemical Physics* 152.4 (Jan. 2020). ISSN: 1089-7690. DOI: 10.1063/1.5129672. URL: http://dx.doi.org/10.1063/1.5129672.

[110] Sebastian Keller et al. "An efficient matrix product operator representation of the quantum chemical Hamiltonian". In: *The Journal of Chemical Physics* 143.24 (Dec. 2015). ISSN: 1089-7690. DOI: 10.1063/1.4939000. URL: http://dx.doi.org/10.1063/1.4939000.

[111] Maarten Van Damme et al. *Efficient higher-order matrix product operators for time evolution*. 2023. arXiv: 2302.14181 [cond-mat.str-el]. URL: https://arxiv.org/abs/2302.14181.

[112] Sebastian Keller et al. "Selection of active spaces for multiconfigurational wavefunctions". In: *The Journal of Chemical Physics* 142.24 (June 2015), p. 244104. ISSN: 0021-9606. DOI: 10.1063/1.4922352. eprint: https://pubs.aip.org/aip/jcp/article-pdf/doi/10.1063/1.4922352/15496177/244104_1_online.pdf. URL: https://doi.org/10.1063/1.4922352.

[113] Carlos L. Benavides-Riveros, Nektarios N. Lathiotakis, and Miguel A. L. Marques. "Towards a formal definition of static and dynamic electronic correlations". In: *Physical Chemistry Chemical Physics* 19.20 (2017), pp. 12655–12664. ISSN: 1463-9084. DOI: 10.1039/c7cp01137g. URL: http://dx.doi.org/10.1039/C7CP01137G.

[114] Stefan Knecht et al. "New Approaches for ab initio Calculations of Molecules with Strong Electron Correlation". In: *CHIMIA* 70.4 (Apr. 2016), p. 244. ISSN: 0009-4293. DOI: 10.2533/chimia.2016.244. URL: http://dx.doi.org/10.2533/chimia.2016.244.

[115] Francesco Aquilante et al. "Molcas 8: New capabilities for multiconfigurational quantum chemical calculations across the periodic table". In: *Journal of Computational Chemistry* 37.5 (2016), pp. 506–541. DOI: https://doi.org/10.1002/jcc.24221. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/jcc.24221. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/jcc.24221.

[116] Ulrich Schollwöck. "The density-matrix renormalization group in the age of matrix product states". In: *Annals of Physics* 326.1 (Jan. 2011), pp. 96–192. ISSN: 0003-4916. DOI: 10.1016/j.aop.2010.09.012. URL: http://dx.doi.org/10.1016/j.aop.2010.09.012.

[117] Steven R. White. "Density matrix formulation for quantum renormalization groups". In: *Phys. Rev. Lett.* 69 (19 Nov. 1992), pp. 2863–2866. DOI: 10.1103/PhysRevLett.69.2863. URL: https://link.aps.org/doi/10.1103/PhysRevLett.69.2863.

[118] Steven R. White. "Density-matrix algorithms for quantum renormalization groups". In: *Phys. Rev. B* 48 (14 Oct. 1993), pp. 10345–10356. DOI: 10.1103/PhysRevB.48.10345. URL: https://link.aps.org/doi/10.1103/PhysRevB.48.10345.

[119] Ö. Legeza, J. Röder, and B. A. Hess. "Controlling the accuracy of the density-matrix renormalization-group method: The dynamical block state selection approach". In: *Physical Review B* 67.12 (Mar. 2003). ISSN: 1095-3795. DOI: 10.1103/physrevb.67.125114. URL: http://dx.doi.org/10.1103/PhysRevB.67.125114.

[120] Ö. Legeza and J. Sólyom. "Optimizing the density-matrix renormalization group method using quantum information entropy". In: *Phys. Rev. B* 68 (19 Nov. 2003), p. 195116. DOI: 10.1103/PhysRevB.68.195116. URL: https://link.aps.org/doi/10.1103/PhysRevB.68.195116.

[121] Jörg Rissler, Reinhard M. Noack, and Steven R. White. "Measuring orbital interaction using quantum information theory". In: *Chemical Physics* 323.2-3 (Apr. 2006), pp. 519–531. ISSN: 0301-0104. DOI: 10.1016/j.chemphys.2005.10.018. URL: http://dx.doi.org/10.1016/j.chemphys.2005.10.018.

[122] Christopher J. Stein and Markus Reiher. "Measuring multi-configurational character by orbital entanglement". In: *Molecular Physics* 115.17-18 (Feb. 2017), pp. 2110–2119. ISSN: 1362-3028. DOI: 10.1080/00268976.2017.1288934. URL: http://dx.doi.org/10.1080/00268976.2017.1288934.

[123] G. Barcza et al. “Quantum-information analysis of electronic states of different molecular structures”. In: *Physical Review A* 83.1 (Jan. 2011). ISSN: 1094-1622. DOI: 10.1103/physreva.83.012508. URL: http://dx.doi.org/10.1103/PhysRevA.83.012508.

[124] Kalman Szenes et al. “QCMaquis 4.0: Multipurpose Electronic, Vibrational, and Vibronic Structure and Dynamics Calculations with the Density Matrix Renormalization Group”. In: *The Journal of Physical Chemistry A* (Aug. 2025). ISSN: 1520-5215. DOI: 10.1021/acs.jpca.5c02970. URL: http://dx.doi.org/10.1021/acs.jpca.5c02970.

[125] Ignacio Fdez. Galván et al. “OpenMolcas: From Source Code to Insight”. In: *Journal of Chemical Theory and Computation* 15.11 (2019). PMID: 31509407, pp. 5925–5964. DOI: 10.1021/acs.jctc.9b00532. eprint: https://doi.org/10.1021/acs.jctc.9b00532. URL: https://doi.org/10.1021/acs.jctc.9b00532.

[126] Francesco Aquilante et al. “Modern quantum chemistry with [Open]Molcas”. In: *The Journal of Chemical Physics* 152.21 (June 2020), p. 214117. ISSN: 0021-9606. DOI: 10.1063/5.0004835. eprint: https://pubs.aip.org/aip/jcp/article-pdf/doi/10.1063/5.0004835/16669993/214117_1_online.pdf. URL: https://doi.org/10.1063/5.0004835.

[127] Giovanni Li Manni et al. “The OpenMolcas Web: A Community-Driven Approach to Advancing Computational Chemistry”. In: *Journal of Chemical Theory and Computation* 19.20 (2023). PMID: 37216210, pp. 6933–6991. DOI: 10.1021/acs.jctc.3c00182. eprint: https://doi.org/10.1021/acs.jctc.3c00182. URL: https://doi.org/10.1021/acs.jctc.3c00182.